

PROYECTO DEVOPS EN KUBERNETES

Diseño e Implementación de una Plataforma Cloud Native con CI/CD, Blue-Green Deployment y Observabilidad

Memoria Técnica del Proyecto

Contenerización, Orquestación, Integración Continua, Entrega Continua, Seguridad y Monitorización en Kubernetes

Autor

Fernando Martínez Altolaguirre

Tokio School

Curso DevOps

1 Resumen Ejecutivo

El presente proyecto tiene como finalidad el diseño, implementación y validación de una plataforma DevOps basada en tecnologías Cloud Native, aplicando principios de automatización, contenerización y orquestación de aplicaciones para mejorar la calidad, velocidad y fiabilidad del ciclo de vida del software.

La solución desarrollada, denominada **TechWave DevOps Platform**, ha sido implementada utilizando **Node.js** y **Express** como tecnologías de desarrollo, **Docker** para la contenerización de la aplicación y **Kubernetes** como plataforma de orquestación. Asimismo, se ha utilizado **GitHub** como repositorio central de código fuente, **GitHub Actions** para la automatización de los procesos de integración y entrega continua (CI/CD) y **GitHub Container Registry (GHCR)** para el almacenamiento y distribución de imágenes Docker.

Como parte de la estrategia de despliegue se ha implementado un modelo **Blue-Green Deployment**, permitiendo la coexistencia simultánea de dos versiones de la aplicación dentro del clúster Kubernetes, reduciendo riesgos durante las actualizaciones y facilitando los procesos de recuperación ante incidencias.

La plataforma incorpora mecanismos de gestión de configuración mediante **ConfigMaps** y **Secrets de Kubernetes**, permitiendo la separación entre configuración, credenciales y lógica de negocio. Adicionalmente, se han implementado mecanismos de supervisión mediante la exposición de métricas compatibles con **Prometheus** a través del endpoint `/metrics`, proporcionando una base sólida para futuras integraciones con plataformas de observabilidad.

Durante la fase de validación se verificó el correcto funcionamiento de los distintos componentes de la solución, incluyendo el despliegue automatizado de contenedores, la ejecución de la estrategia Blue-Green, la disponibilidad de los servicios desplegados y la correcta exposición de métricas operativas desde Kubernetes.

Los resultados obtenidos demuestran la viabilidad de una arquitectura moderna basada en prácticas DevOps, permitiendo automatizar procesos, mejorar la trazabilidad de los despliegues y aumentar la capacidad de escalado y mantenimiento de aplicaciones contenerizadas.

Finalmente, el proyecto establece una base tecnológica preparada para futuras evoluciones, incluyendo la integración de plataformas avanzadas de observabilidad, control de tráfico mediante Ingress Controllers, automatización de infraestructura mediante Terraform y despliegue sobre plataformas Kubernetes gestionadas en entornos cloud.

1.1 Situación de partida

TechWave Solutions se encuentra en una fase de crecimiento en la que la velocidad de desarrollo y despliegue de nuevas funcionalidades constituye un factor estratégico de diferenciación.

La organización presenta necesidades típicas de una empresa en expansión:

- Incremento continuo de la demanda.
- Necesidad de reducir tiempos de entrega.
- Dependencia de procesos manuales.
- Dificultad para reproducir entornos.
- Escalabilidad limitada.
- Falta de trazabilidad operativa.
- Ausencia de mecanismos avanzados de monitorización.

Aunque la aplicación actual satisface los requisitos funcionales del negocio, la plataforma tecnológica requiere evolucionar hacia un modelo más automatizado, seguro y resiliente.

1.2 Problema de negocio

La ausencia de una estrategia DevOps integral genera varios riesgos operativos:

Riesgos de Desarrollo

- Diferencias entre entornos.
- Errores de configuración.
- Dificultad para reproducir incidencias.

Riesgos Operativos

- Despliegues manuales.
- Interrupciones del servicio.
- Dependencia de personal clave.

Riesgos de Seguridad

- Gestión manual de credenciales.
- Escasa trazabilidad.
- Ausencia de políticas homogéneas.

Riesgos de Escalabilidad

- Crecimiento limitado.
- Saturación de recursos.
- Incremento de costes operativos.

1.3 Objetivos Estratégicos

La implantación de la plataforma TechWave DevOps persigue la consecución de una serie de objetivos estratégicos orientados a mejorar la eficiencia, calidad y trazabilidad del ciclo de vida del software mediante la adopción de prácticas DevOps y tecnologías Cloud Native.

O1. Reducir el Time to Market

Disminuir el tiempo necesario para transformar una modificación de código en una versión desplegada y operativa, mediante la automatización de los procesos de construcción, validación y publicación de la aplicación.

O2. Incrementar la estabilidad operativa

Garantizar despliegues repetibles, consistentes y controlados a través del uso de contenedores Docker, pipelines automatizados y entornos gestionados mediante Kubernetes.

O3. Aumentar la disponibilidad del servicio

Implementar mecanismos que permitan minimizar el impacto de los cambios sobre los usuarios finales, favoreciendo la continuidad operativa mediante estrategias de despliegue Blue-Green y capacidades de recuperación rápida.

O4. Mejorar la observabilidad

Proporcionar visibilidad sobre el comportamiento de la aplicación mediante la exposición de métricas operativas que permitan supervisar el consumo de recursos, el rendimiento y el estado general del servicio.

O5. Establecer una base de crecimiento sostenible

Diseñar una plataforma flexible y escalable que pueda evolucionar hacia entornos cloud gestionados e incorporar capacidades avanzadas de automatización, observabilidad y gestión operativa.

1.4 Solución Propuesta

Para alcanzar los objetivos establecidos se propone la construcción de una plataforma DevOps moderna basada en tecnologías de código abierto y principios Cloud Native.

La solución se estructura en los siguientes componentes:

- **Contenerización** La aplicación es empaquetada mediante Docker, garantizando portabilidad, reproducibilidad y consistencia entre los distintos entornos de ejecución.

- **Orquestación** Kubernetes actúa como plataforma de orquestación, proporcionando:
 - Gestión declarativa de aplicaciones.
 - Alta disponibilidad.
 - Escalabilidad horizontal.
 - Recuperación automática ante fallos.
 - Gestión centralizada de despliegues.
- **Integración y Entrega Continua** GitHub Actions automatiza las actividades de:
 - Construcción de la aplicación.
 - Ejecución de pruebas.
 - Generación de imágenes Docker.
 - Publicación en GitHub Container Registry (GHCR).
 - Preparación del despliegue para Kubernetes.
- **Gestión de Configuración** La solución utiliza recursos nativos de Kubernetes para separar la configuración de la aplicación de su código fuente:
 - ConfigMaps para parámetros de configuración.
 - Secrets para credenciales e información sensible.
- **Observabilidad** La plataforma incorpora instrumentación mediante Prometheus Client para Node.js, permitiendo la exposición de métricas operativas a través del endpoint /metrics.
Esta aproximación sienta las bases para futuras integraciones con herramientas de monitorización y observabilidad más avanzadas.
- **Seguridad** Durante el desarrollo se aplican principios básicos de seguridad orientados a:
 - Separación de configuración y secretos.
 - Ejecución de contenedores bajo usuarios no privilegiados.
 - Automatización de despliegues para reducir errores manuales.
 - Gestión controlada de versiones y trazabilidad de cambios.

1.5 Beneficios Esperados

- **Beneficios Técnicos**
 - Entornos consistentes y reproducibles.
 - Despliegues automatizados.
 - Mayor capacidad de escalado.
 - Reducción de errores derivados de tareas manuales.
 - Mejor capacidad de recuperación ante incidencias.
- **Beneficios Operacionales**
 - Simplificación de tareas de despliegue.
 - Incremento de la productividad.

- Mayor trazabilidad en cambios y versiones.
- Reducción del tiempo invertido en operaciones repetitivas.
- **Beneficios Estratégicos**
 - Mayor capacidad de adaptación a nuevas necesidades.
 - Aceleración de los ciclos de entrega.
 - Preparación para futuras evoluciones tecnológicas.
 - Base sólida para la adopción de plataformas cloud gestionadas.

1.6 Indicadores de Éxito

La implantación se considerará satisfactoria cuando se alcancen los siguientes objetivos de referencia:

Indicador	Objetivo
Tiempo medio de despliegue	Menor de 10 minutos
Automatización del ciclo de entrega	Superior al 90 %
Disponibilidad de la aplicación	Superior al 99 %
Tiempo de recuperación	Menor de 15 minutos
Exposición de métricas operativas	100 % de la aplicación
Despliegues automatizados	Superior al 90 %

1.7 Alcance

Incluido

- Desarrollo de una aplicación web basada en Node.js.
- Contenerización mediante Docker.
- Almacenamiento de imágenes en GHCR.
- Plataforma Kubernetes.
- GitHub Actions para automatización CI/CD.
- Estrategia Blue-Green Deployment.
- ConfigMaps y Secrets.
- Endpoint de métricas compatibles con Prometheus.
- Documentación técnica del proyecto.
- Validación funcional de la solución implementada.

No incluido

- Desarrollo de funcionalidades avanzadas de negocio.

- Integraciones con sistemas corporativos externos.
- Despliegues multi-región.
- Arquitecturas multicloud.
- Infraestructura cloud gestionada.
- Implementación completa de plataformas de observabilidad avanzadas.

1.8 Visión Futura

La arquitectura desarrollada ha sido diseñada para permitir la evolución de la plataforma sin necesidad de cambios significativos en la aplicación.

- **Corto Plazo**
 - Integración completa de Ingress Controller.
 - Incorporación de Prometheus como plataforma de monitorización.
 - Implementación de dashboards operativos.
- **Medio Plazo**
 - Grafana para visualización avanzada.
 - Loki para gestión centralizada de logs.
 - OpenTelemetry para trazabilidad distribuida.
 - Helm para empaquetado de aplicaciones.
- **Largo Plazo**
 - GitOps.
 - ArgoCD.
 - Terraform para automatización de infraestructura.
 - Azure Kubernetes Service (AKS).
 - Service Mesh.
 - FinOps.
 - SRE avanzado.

2 Contexto Estratégico

Antes de definir cualquier arquitectura tecnológica es necesario comprender el contexto organizativo en el que dicha arquitectura debe operar.

La implantación de una plataforma DevOps no constituye únicamente un cambio tecnológico. Supone una transformación de los procesos de desarrollo, operación, gobierno, seguridad y gestión del ciclo de vida del software.

Por este motivo, el presente capítulo analiza la situación actual de TechWave Solutions desde una perspectiva estratégica y operativa, identificando los factores internos y externos que justifican la evolución hacia una plataforma basada en prácticas DevOps, infraestructura como código, contenerización y automatización del ciclo de entrega.

2.1 Descripción de la Organización

TechWave Solutions es una empresa tecnológica emergente especializada en el desarrollo de soluciones digitales para clientes corporativos y consumidores finales.

La organización se encuentra en una fase de crecimiento caracterizada por:

- Incremento progresivo de la base de usuarios.
- Necesidad constante de incorporar nuevas funcionalidades.
- Evolución continua del producto.
- Competencia creciente dentro del sector tecnológico.
- Elevada exigencia en términos de disponibilidad y experiencia de usuario.

El principal activo de la organización es su capacidad para innovar y entregar nuevas funcionalidades con rapidez al mercado.

Sin embargo, el crecimiento del negocio exige evolucionar la infraestructura tecnológica para evitar cuellos de botella derivados de procesos manuales y arquitecturas difíciles de escalar.

2.1.1 Misión

Desarrollar soluciones tecnológicas innovadoras capaces de aportar valor tangible a los usuarios mediante servicios digitales modernos, seguros y escalables.

2.1.2 Visión

Convertirse en una organización altamente digitalizada, capaz de desplegar nuevas capacidades de negocio de forma rápida, segura y fiable mediante una plataforma tecnológica cloud-native basada en principios DevOps.

2.1.3 Objetivos Estratégicos

La organización persigue los siguientes objetivos estratégicos:

OE-01 Incrementar la velocidad de entrega

Reducir el tiempo transcurrido entre el desarrollo de una funcionalidad y su disponibilidad para los usuarios finales.

OE-02 Mejorar la estabilidad operativa

Disminuir el número de incidencias producidas por errores de configuración y despliegues manuales.

OE-03 Garantizar escalabilidad

Proporcionar una infraestructura capaz de adaptarse dinámicamente a las necesidades del negocio.

OE-04 Mejorar el control y la trazabilidad

Disponer de mecanismos que permitan conocer en todo momento el estado de la infraestructura, aplicaciones y servicios.

OE-05 Aumentar la seguridad

Incorporar controles de seguridad en todas las fases del ciclo de vida de las aplicaciones.

2.2 Identificación de Stakeholders

La implantación de la plataforma propuesta afecta a múltiples actores dentro y fuera de la organización.

2.2.1 Dirección General

Intereses

- Reducción de costes.
- Mejora de la competitividad.
- Incremento de la velocidad de entrega.
- Mitigación de riesgos operativos.

Nivel de impacto

- Alto.

Nivel de influencia

- Muy alto.

2.2.2 Área de Desarrollo

Intereses

- Automatización de despliegues.
- Entornos homogéneos.
- Reducción de tareas manuales.
- Mayor productividad.

Nivel de impacto

- Muy alto.

Nivel de influencia

- Alto.

2.2.3 Área de Operaciones

Intereses

- Estabilidad.
- Escalabilidad.
- Observabilidad.
- Facilidad de mantenimiento.

Nivel de impacto

- Muy alto.

Nivel de influencia

- Alto.

2.2.4 Área de Seguridad

Intereses

- Protección de datos.
- Gestión de credenciales.
- Auditorías.
- Cumplimiento normativo.

Nivel de impacto

- Alto.

Nivel de influencia

- Medio.

2.2.5 Usuarios Finales

Intereses

- Disponibilidad.
- Rendimiento.
- Fiabilidad.

Nivel de impacto

- Muy alto.

Nivel de influencia

- Bajo.

2.2.6 Proveedores Cloud

Intereses

- Disponibilidad de servicios.
- Consumo de recursos.

Nivel de impacto

- Medio.

Nivel de influencia

- Medio.

2.3 Modelo Operativo Actual (AS-IS)

2.3.1 Situación General

Actualmente la organización utiliza un modelo operativo tradicional donde gran parte de las actividades relacionadas con el despliegue y mantenimiento de aplicaciones dependen de acciones manuales.

Aunque este modelo resulta suficiente en fases iniciales de crecimiento, comienza a mostrar limitaciones cuando el número de desarrolladores, despliegues y usuarios aumenta de forma significativa.

2.3.2 Ciclo Actual de Desarrollo

Fase 1 Desarrollo de funcionalidades.

Fase 2 Pruebas locales.

Fase 3 Generación manual de artefactos.

Fase 4 Despliegue manual.

Fase 5 Validación funcional.

Fase 6 Monitorización reactiva.

2.3.3 Principales Limitaciones

Dependencia de tareas manuales Los despliegues requieren intervención humana.

Ausencia de automatización integral Los procesos de compilación, pruebas y publicación no se encuentran completamente automatizados.

Trazabilidad limitada Resulta complejo identificar:

- Qué versión está desplegada.
- Quién realizó el cambio.
- Cuándo se realizó.
- Qué componentes fueron modificados.

Observabilidad insuficiente No existe una plataforma centralizada de métricas, logs y trazas.

Escalabilidad limitada La capacidad de crecimiento depende en gran medida de intervenciones manuales.

2.4 Identificación de Problemas

P1. Lentitud en los despliegues Cada despliegue requiere múltiples tareas manuales que incrementan el tiempo necesario para publicar nuevas funcionalidades.

Impacto Alto.

Prioridad Alta.

P2. Riesgo de errores humanos La repetición de tareas manuales aumenta la probabilidad de errores operativos.

Impacto Alto.

Prioridad Alta.

P3. Baja capacidad de observación La organización carece de una visión unificada sobre:

- Estado de la aplicación.
- Estado de la infraestructura.
- Rendimiento.
- Disponibilidad.

Impacto Alto.

Prioridad Alta.

P4. Gestión compleja de configuraciones La configuración de los sistemas no está completamente centralizada ni versionada.

Impacto Medio.

Prioridad Media.

P5. Dificultad para escalar La infraestructura actual presenta limitaciones para responder rápidamente a incrementos de carga.

Impacto Alto.

Prioridad Alta.

2.5 Análisis DAFO

Debilidades • Procesos de despliegue manuales. • Escasa observabilidad. • Dependencia de personas clave. • Gestión descentralizada de configuraciones. • Escalabilidad limitada.	Amenazas • Incremento de la competencia tecnológica. • Evolución continua de las amenazas de ciberseguridad. • Incremento inesperado de la demanda. • Dependencia de proveedores externos. • Cambios regulatorios.
Fortalezas • Equipo técnico capacitado. • Aplicación desarrollada con tecnologías modernas. • Capacidad de adaptación tecnológica. • Cultura orientada a la innovación. • Flexibilidad organizativa.	Oportunidades • Adopción de DevOps. • Arquitecturas Cloud Native. • Automatización de procesos. • Optimización de costes. • Incremento de la calidad del servicio.

El análisis DAFO realizado pone de manifiesto la necesidad de evolucionar hacia un modelo operativo basado en principios DevOps que permita mejorar la

eficiencia, escalabilidad y capacidad de respuesta de la organización ante los retos tecnológicos actuales.

Por un lado, se identifican diversas **debilidades internas**, principalmente relacionadas con la existencia de procesos manuales de despliegue, una limitada capacidad de observabilidad, la dependencia de personas clave para determinadas tareas operativas, la gestión descentralizada de configuraciones y ciertas limitaciones en materia de escalabilidad. Estas circunstancias incrementan el riesgo operativo y dificultan la implantación de procesos ágiles y repetibles.

En el ámbito externo, las **amenazas** detectadas evidencian un entorno tecnológico altamente competitivo y en constante evolución. El aumento de las amenazas de ciberseguridad, los cambios regulatorios, la dependencia de proveedores externos y la posibilidad de incrementos bruscos de demanda obligan a disponer de plataformas más resilientes, automatizadas y preparadas para adaptarse con rapidez a nuevos escenarios.

No obstante, la organización cuenta con importantes **fortalezas** que facilitan la transformación propuesta. Entre ellas destacan la capacitación técnica del equipo, el uso de tecnologías modernas en el desarrollo de aplicaciones, la capacidad de adaptación al cambio, una cultura orientada a la innovación y una estructura organizativa flexible. Estos factores constituyen una base sólida para la adopción de nuevas metodologías y herramientas.

Adicionalmente, existen relevantes **oportunidades** derivadas de la adopción de prácticas DevOps y arquitecturas Cloud Native. La automatización de procesos, la mejora de la calidad del servicio, la optimización de costes y la posibilidad de acelerar los ciclos de entrega representan elementos clave para incrementar la competitividad y la capacidad de crecimiento de la organización.

En consecuencia, el análisis realizado justifica la implantación de una plataforma DevOps basada en contenedores, integración y entrega continua, Kubernetes y mecanismos de observabilidad, permitiendo reducir la complejidad operativa, aumentar la fiabilidad de los despliegues y establecer una base tecnológica preparada para futuras evoluciones y procesos de modernización.

2.6 Necesidades de Transformación

A partir del análisis realizado se identifican las siguientes necesidades de evolución tecnológica.

NT-01 Automatización Automatizar completamente el ciclo de vida de las aplicaciones desde la construcción hasta el despliegue.

NT-02 Escalabilidad Implementar una plataforma capaz de crecer horizontalmente en función de la demanda.

NT-03 Observabilidad Incorporar capacidades avanzadas de:

- Métricas.
- Logs.
- Trazas.
- Alertas.

NT-04 Seguridad Integrar la seguridad dentro del proceso de desarrollo y operación.

NT-05 Resiliencia Reducir el impacto de incidencias mediante:

- Alta disponibilidad.
- Rollback automático.
- Recuperación rápida.

NT-06 Gobierno Tecnológico Implantar mecanismos que permitan:

- Versionado.
- Auditoría.
- Trazabilidad.
- Gestión del cambio.

2.7 Justificación de la Transformación DevOps

La implantación de una estrategia DevOps en TechWave Solutions no debe entenderse como una actualización tecnológica aislada, sino como una iniciativa de transformación organizativa orientada a mejorar la capacidad de entrega de valor al negocio.

La adopción de Docker, Kubernetes, Terraform, GitHub Actions y herramientas avanzadas de observabilidad permitirá construir una plataforma moderna, escalable y segura capaz de responder a las necesidades actuales y futuras de la organización. Esta evolución reducirá tiempos de despliegue, mejorará la estabilidad operativa, incrementará la capacidad de monitorización y facilitará la adopción de futuros modelos de operación basados en GitOps, SRE y automatización avanzada.

3 Objetivos Corporativos

La transformación DevOps propuesta para TechWave Solutions no constituye únicamente una evolución tecnológica, sino una iniciativa estratégica alineada con los objetivos corporativos de crecimiento, eficiencia operativa y excelencia en la prestación de servicios digitales.

Los objetivos definidos en este capítulo servirán como referencia para la toma de decisiones arquitectónicas y tecnológicas que serán descritas en los capítulos posteriores. Cada componente de la solución propuesta deberá contribuir de forma directa al cumplimiento de uno o varios de estos objetivos.

3.1 Objetivo General

Diseñar e implantar una plataforma DevOps moderna, automatizada, segura y escalable que permita a TechWave Solutions acelerar la entrega de valor al negocio, incrementar la calidad de sus servicios digitales y garantizar la sostenibilidad operativa a medida que la organización crece.

3.2 Objetivos Estratégicos de Negocio

OE-01 Reducir el Time To Market

Descripción

Uno de los principales factores competitivos en el sector tecnológico es la capacidad de transformar una necesidad de negocio en una funcionalidad disponible para el usuario final en el menor tiempo posible.

Actualmente, los ciclos de desarrollo y despliegue pueden verse afectados por tareas manuales, validaciones repetitivas y procesos operativos que ralentizan la puesta en producción.

La organización necesita reducir significativamente el tiempo transcurrido entre:

- La identificación de una necesidad.
- El desarrollo de una solución.
- Su despliegue en producción.
- La obtención de feedback del usuario.

Beneficios esperados

- Mayor velocidad de innovación.
- Respuesta rápida a cambios del mercado.
- Mayor competitividad.
- Incremento de la satisfacción del cliente.
- Reducción del tiempo de obtención de valor.

Contribución de la plataforma DevOps

- Automatización completa del ciclo CI/CD.
- Estandarización de entornos.
- Reducción de tareas manuales.
- Despliegues repetibles y automatizados.
- Disminución del riesgo asociado a cada release.

Indicadores asociados

Indicador	Situación actual	Objetivo
Tiempo medio de despliegue	Variable	< 10 minutos
Frecuencia de despliegues	Baja	Varias veces por semana
Tiempo desde desarrollo a producción	Días	Horas

OE-02 Incrementar la Disponibilidad del Servicio

Descripción

La disponibilidad de los servicios digitales constituye un requisito fundamental para garantizar la continuidad de negocio y la satisfacción de los usuarios.

A medida que aumenta la dependencia de la plataforma tecnológica, cualquier interrupción del servicio puede generar pérdidas económicas, impacto reputacional y disminución de la confianza de los clientes.

La organización necesita disponer de una infraestructura resiliente que minimice las interrupciones y permita recuperaciones rápidas ante incidencias.

Beneficios esperados

- Mayor continuidad operativa.
- Mejora de la experiencia de usuario.
- Reducción de incidencias críticas.
- Mayor confianza de clientes y socios.

Contribución de la plataforma DevOps

- Kubernetes como plataforma de alta disponibilidad.
- Escalado automático.

- Despliegues Blue-Green.
- Rollback automatizado.
- Monitorización proactiva.
- Alertas tempranas.

Indicadores asociados

Indicador	Objetivo
Disponibilidad anual	≥ 99,5 %
Tiempo medio de recuperación (MTTR)	< 15 minutos
Incidencias críticas por despliegue	Tendencia a cero

OE-03 Mejorar la Calidad del Software

Descripción

La calidad del software impacta directamente en la estabilidad operativa, los costes de mantenimiento y la satisfacción de los usuarios.

La organización requiere establecer mecanismos que permitan detectar errores cuanto antes, garantizando que cada nueva versión cumpla unos estándares mínimos de calidad antes de su puesta en producción.

Beneficios esperados

- Reducción de defectos.
- Mayor estabilidad.
- Menor retrabajo.
- Incremento de la satisfacción del usuario.

Contribución de la plataforma DevOps

- Automatización de pruebas.
- Validaciones continuas.
- Integración continua.
- Observabilidad avanzada.
- Trazabilidad de cambios.

Indicadores asociados

Indicador	Objetivo
Defectos detectados en producción	Descendente
Porcentaje de despliegues exitosos	> 95 %
Tiempo de corrección de incidencias	Reducido

OE-04 Reducir los Costes Operativos

Descripción

La eficiencia operativa constituye uno de los pilares fundamentales para asegurar la sostenibilidad económica de la organización.

Las tareas manuales, los procesos repetitivos y la gestión reactiva de incidencias generan costes operativos que aumentan a medida que la plataforma crece.

La automatización y estandarización de procesos permiten optimizar el uso de recursos técnicos y humanos.

Beneficios esperados

- Menor carga operativa.
- Mejor aprovechamiento de recursos.
- Reducción de errores humanos.
- Optimización del esfuerzo de los equipos.

Contribución de la plataforma DevOps

- Infraestructura como Código (IaC).
- Automatización CI/CD.
- Gestión centralizada de configuraciones.
- Escalabilidad dinámica.
- Observabilidad centralizada.

Indicadores asociados

Indicador	Objetivo
Tareas manuales de despliegue	Reducción > 90 %
Tiempo de operación rutinaria	Descendente
Coste de gestión por despliegue	Descendente

OE-05 Facilitar la Escalabilidad del Negocio

Descripción

La previsión de crecimiento de TechWave Solutions exige que la infraestructura tecnológica sea capaz de adaptarse de forma ágil a nuevas demandas sin requerir rediseños continuos.

La plataforma debe permitir absorber incrementos de usuarios, aplicaciones y servicios manteniendo niveles adecuados de rendimiento y estabilidad.

Beneficios esperados

- Preparación para el crecimiento.
- Mayor flexibilidad tecnológica.
- Optimización de recursos.
- Reducción de cuellos de botella.

Contribución de la plataforma DevOps

- Arquitectura basada en contenedores.
- Orquestación mediante Kubernetes.
- Escalado horizontal.
- Infraestructura declarativa.
- Automatización de aprovisionamiento mediante Terraform.

Indicadores asociados

Indicador	Objetivo
Tiempo para ampliar capacidad	Minutos
Escalado horizontal automatizado	Sí
Impacto de crecimiento sobre operación	Mínimo

3.3 Relación entre Objetivos y Capacidades Tecnológicas

Objetivo Corporativo	Docker	Kubernetes	Terraform	GitHub Actions	Observabilidad
Reducir Time To Market	✓	✓	✓	✓	✓
Incrementar disponibilidad	✓	✓	✓	✓	✓
Mejorar calidad	✓	✓	✗	✓	✓
Reducir costes operativos	✓	✓	✓	✓	✓
Facilitar escalabilidad	✓	✓	✓	✗	✓

3.4 Factores Críticos de Éxito

Para lograr los objetivos definidos será necesario garantizar los siguientes factores:

FCE-01 Automatización Integral Minimizar la intervención manual en los procesos de construcción, despliegue y mantenimiento.

FCE-02 Observabilidad Completa Disponer de métricas, logs y trazas que permitan conocer en todo momento el estado de la plataforma.

FCE-03 Seguridad Integrada Incorporar controles de seguridad desde las fases iniciales del ciclo de vida del software.

FCE-04 Gobierno Tecnológico Mantener una gestión controlada del cambio mediante procesos documentados y trazables.

FCE-05 Evolución Continua Diseñar la plataforma con capacidad para incorporar futuras mejoras tecnológicas sin necesidad de rediseños estructurales.

Conclusión

Los objetivos corporativos definidos establecen el marco estratégico que guiará todo el proyecto. La arquitectura propuesta no se justifica únicamente por criterios tecnológicos, sino por su capacidad para contribuir directamente a los

objetivos de negocio de TechWave Solutions: acelerar la entrega de valor, mejorar la disponibilidad, incrementar la calidad del software, optimizar los costes operativos y asegurar la escalabilidad futura de la organización. A partir de estos objetivos, el siguiente capítulo definirá los requisitos funcionales y no funcionales que deberán satisfacerse para materializar la solución propuesta.

4 Análisis de Requisitos

El presente capítulo define los requisitos que deberá satisfacer la plataforma DevOps propuesta para TechWave Solutions. Estos requisitos constituyen la base sobre la que se diseñará la arquitectura objetivo (TO-BE) y permitirán validar posteriormente que la solución implantada cumple los objetivos estratégicos identificados en los capítulos anteriores.

Los requisitos se dividen en dos grandes categorías:

- **Requisitos funcionales**, que describen los servicios y capacidades que debe proporcionar la plataforma.
- **Requisitos no funcionales**, que establecen las características de calidad, rendimiento, seguridad y operación que debe cumplir la solución.

4.1 Requisitos Funcionales

RF-01 Gestión de la Aplicación Contenerizada

Descripción La plataforma deberá permitir la ejecución de aplicaciones mediante contenedores estandarizados basados en Docker.

Objetivo Garantizar la portabilidad y reproducibilidad de los entornos.

Criterio de aceptación

- La aplicación debe ejecutarse mediante una imagen Docker personalizada.
- La imagen debe poder desplegarse de forma idéntica en cualquier entorno compatible.

Prioridad Alta.

RF-02 Orquestación de Contenedores

Descripción La plataforma deberá gestionar automáticamente el ciclo de vida de los contenedores mediante Kubernetes.

Objetivo Asegurar la disponibilidad y escalabilidad de los servicios.

Criterio de aceptación

- Los pods deben desplegarse automáticamente.
- Kubernetes debe gestionar reinicios y recuperación ante fallos.
- Los servicios deben mantenerse disponibles independientemente del estado de instancias individuales.

Prioridad Alta.

RF-03 Gestión Centralizada de Configuración

Descripción La configuración de la aplicación deberá mantenerse separada del código fuente.

Objetivo Facilitar la promoción entre entornos y simplificar la administración.

Criterio de aceptación

- Uso de ConfigMaps para parámetros de configuración.
- Uso de Secrets para información sensible.

Prioridad Alta.

RF-04 Integración Continua

Descripción La plataforma deberá ejecutar automáticamente procesos de integración continua cuando se produzcan cambios en el repositorio de código.

Objetivo Detectar errores en fases tempranas del ciclo de desarrollo.

Criterio de aceptación

- Ejecución automática de validaciones.
- Construcción automática de artefactos.
- Generación automática de imágenes Docker.

Prioridad Alta.

RF-05 Entrega Continua

Descripción La solución deberá automatizar el despliegue de nuevas versiones.

Objetivo Reducir tiempos de entrega y errores manuales.

Criterio de aceptación

- Publicación automática de imágenes en el registro.
- Despliegue automatizado en Kubernetes.

Prioridad Alta.

RF-06 Despliegues Blue-Green

Descripción La plataforma deberá soportar despliegues Blue-Green para minimizar interrupciones durante la publicación de nuevas versiones.

Objetivo Garantizar continuidad de servicio.

Criterio de aceptación

- Existencia simultánea de entornos Blue y Green.
- Conmutación controlada del tráfico.
- Validación previa antes de la exposición al usuario final.

Prioridad Alta.

RF-07 Recuperación Automática (Rollback)

Descripción La plataforma deberá permitir revertir automáticamente despliegues defectuosos.

Objetivo Reducir el impacto de errores en producción.

Criterio de aceptación

- Detección de fallos mediante probes y validaciones.
- Restauración automática de la versión estable.

Prioridad Alta.

RF-08 Monitorización de Infraestructura

Descripción La solución deberá recopilar métricas de los componentes de infraestructura.

Objetivo Facilitar la supervisión operativa.

Criterio de aceptación

- Recolección de métricas de nodos.
- Recolección de métricas de contenedores.
- Visualización centralizada.

Prioridad Alta.

RF-09 Gestión Centralizada de Logs

Descripción Todos los componentes deberán centralizar sus registros de actividad.

Objetivo Facilitar auditorías y análisis de incidencias.

Criterio de aceptación

- Envío automático de logs.

- Almacenamiento centralizado.
- Búsqueda y consulta unificada.

Prioridad Alta.

RF-10 Observabilidad Distribuida

Descripción La solución deberá proporcionar trazabilidad extremo a extremo de las operaciones realizadas por la aplicación.

Objetivo Facilitar el diagnóstico de problemas complejos.

Criterio de aceptación

- Recogida de trazas mediante OpenTelemetry.
- Correlación entre eventos, logs y métricas.

Prioridad Media-Alta.

RF-11 Gestión de Alertas

Descripción La plataforma deberá generar alertas automáticas ante eventos relevantes.

Objetivo Reducir el tiempo de detección de incidencias.

Criterio de aceptación

- Alertas por disponibilidad.
- Alertas por consumo de recursos.
- Alertas por errores de aplicación.

Prioridad Alta.

4.2 Requisitos No Funcionales

Los requisitos no funcionales definen los niveles mínimos de calidad que deberá alcanzar la solución.

Disponibilidad

RNF-01 Alta Disponibilidad

Descripción La plataforma deberá diseñarse bajo principios de alta disponibilidad.

Objetivo Garantizar la continuidad del servicio.

Requisito Disponibilidad mínima anual objetivo: $\geq 99,5 \%$

Medición Disponibilidad calculada sobre el tiempo total de operación del servicio.

Justificación La aplicación constituye un activo estratégico para el negocio y debe minimizarse el tiempo de inactividad no planificado.

Escalabilidad

RNF-02 Escalabilidad Horizontal

Descripción La plataforma deberá permitir el incremento de capacidad mediante la incorporación de nuevas instancias de servicio.

Objetivo Adaptarse a incrementos de demanda sin rediseños arquitectónicos.

Requisito Escalado horizontal de pods.

- Posibilidad de ampliación de nodos.
- Arquitectura desacoplada.

Justificación El crecimiento futuro de TechWave Solutions exige una infraestructura flexible y preparada para absorber cargas variables.

Observabilidad

RNF-03 Supervisión Integral

Descripción La solución deberá proporcionar una visión completa del estado de la plataforma.

Objetivo Permitir una operación basada en datos y evidencias.

Alcance

Logs Registro detallado de eventos de aplicación e infraestructura.

Métricas Información cuantitativa sobre:

- CPU
- Memoria
- Red
- Almacenamiento
- Estado de servicios

Trazas Seguimiento de peticiones a través de múltiples componentes.

Herramientas objetivo

- Loki
- Prometheus
- Grafana
- OpenTelemetry

Justificación La observabilidad es un requisito imprescindible para garantizar estabilidad y capacidad de diagnóstico.

Seguridad

RNF-04 Principio de Mínimo Privilegio

Descripción

Todos los componentes deberán operar exclusivamente con los permisos estrictamente necesarios.

Objetivo

Reducir la superficie de ataque de la plataforma.

Requisitos

- Contenedores ejecutados como usuarios no privilegiados.
- Uso de Service Accounts específicas.
- Restricción de permisos innecesarios.
- Gestión segura de credenciales.

Justificación

La adopción del principio de mínimo privilegio constituye una práctica fundamental de seguridad en entornos cloud-native.

RNF-05 Protección de Secretos

Descripción

La información sensible deberá almacenarse y gestionarse mediante mecanismos seguros.

Requisitos

- Uso de Kubernetes Secrets.
- Exclusión de credenciales del código fuente.
- Control de acceso restringido.

Justificación

La protección adecuada de credenciales es esencial para preservar la confidencialidad e integridad de los sistemas.

Recuperación

RNF-06 Rollback Automático

Descripción La plataforma deberá ser capaz de restaurar automáticamente una versión estable tras detectar fallos en una nueva implantación.

Objetivo Reducir el impacto de despliegues defectuosos.

Requisitos

- Validación automática post-despliegue.
- Detección de errores críticos.
- Recuperación automatizada.

Tiempo objetivo < 15 minutos

Justificación La continuidad operativa constituye un requisito fundamental para la organización.

Rendimiento

RNF-07 Tiempo de Respuesta

Descripción La aplicación deberá ofrecer tiempos de respuesta adecuados para la experiencia de usuario.

Objetivo Garantizar una experiencia fluida.

Requisito objetivo 95 % de las peticiones < 2 segundos

Mantenibilidad

RNF-08 Infraestructura como Código

Descripción Toda la infraestructura deberá definirse mediante código versionado.

Objetivo Facilitar mantenimiento, auditoría y repetibilidad.

Requisitos

- Terraform como herramienta IaC.
- Repositorio Git centralizado.
- Control de versiones.

4.3 Matriz de Trazabilidad

Objetivo Corporativo	Requisitos Asociados
Reducir Time To Market	RF-04, RF-05, RNF-08
Incrementar Disponibilidad	RF-06, RF-07, RNF-01, RNF-06
Mejorar Calidad del Software	RF-04, RF-10, RNF-03
Reducir Costes Operativos	RF-02, RF-08, RNF-08
Facilitar Escalabilidad	RF-02, RNF-02

5 Análisis de Alternativas Tecnológicas

Tras el análisis de las diferentes alternativas tecnológicas disponibles, se concluye que la combinación formada por **Kubernetes**, **GitHub Actions**, **Docker** y **GitHub Container Registry (GHCR)** constituye la opción más adecuada para satisfacer los objetivos establecidos para la plataforma TechWave DevOps.

Esta combinación proporciona una solución moderna, escalable y alineada con las prácticas DevOps actuales, permitiendo automatizar el ciclo completo de integración, construcción y despliegue de aplicaciones contenerizadas.

Asimismo, la arquitectura seleccionada mantiene la flexibilidad necesaria para evolucionar en el futuro hacia entornos cloud gestionados como Azure Kubernetes Service (AKS), Amazon Elastic Kubernetes Service (EKS) o Google Kubernetes Engine (GKE), sin requerir cambios significativos en la aplicación ni en los procesos definidos.

La selección de tecnologías constituye uno de los factores críticos para el éxito de cualquier iniciativa DevOps. La decisión adoptada debe responder no solamente a criterios técnicos, sino también a consideraciones relacionadas con la facilidad de operación, escalabilidad, costes, adopción por la industria y sostenibilidad a largo plazo.

Con el objetivo de construir una plataforma moderna y alineada con los principios Cloud Native, se han evaluado distintas alternativas para los componentes principales de la solución, incluyendo la plataforma de orquestación, la herramienta de integración y entrega continua y la infraestructura de ejecución.

En los apartados siguientes se analizan las tecnologías consideradas y se justifican las decisiones finalmente adoptadas.

5.1 Selección de la Plataforma de Orquestación

La contenerización mediante Docker constituye un requisito fundamental de la solución propuesta. Sin embargo, para gestionar aplicaciones distribuidas en entornos modernos resulta necesario disponer de una plataforma de orquestación capaz de automatizar los despliegues, el escalado y la recuperación ante fallos.

Las alternativas evaluadas fueron:

- Docker Swarm
- Kubernetes

5.1.1 Decisión Adoptada

Tecnología seleccionada: Kubernetes

Justificación

La elección de Kubernetes responde directamente a los objetivos definidos para el proyecto:

Objetivo	Contribución de Kubernetes
Reducir Time To Market	Automatización de despliegues
Incrementar disponibilidad	Alta disponibilidad y autorecuperación
Mejorar estabilidad	Entornos homogéneos
Facilitar crecimiento	Escalabilidad horizontal
Simplificar operaciones	Gestión declarativa

Además, Kubernetes permite implementar estrategias avanzadas de despliegue, como Blue-Green Deployment, y constituye el estándar de facto para arquitecturas basadas en contenedores.

5.2 Selección de la Plataforma CI/CD

La automatización del ciclo de vida del software constituye uno de los pilares fundamentales de la metodología DevOps.

Se han evaluado las siguientes alternativas:

- Jenkins
- GitLab CI/CD
- GitHub Actions

5.2.1 Decisión Adoptada

Tecnología seleccionada: GitHub Actions

Justificación

GitHub Actions proporciona una integración nativa con el repositorio utilizado en el proyecto, reduciendo significativamente la complejidad operativa y facilitando la automatización del proceso de construcción y despliegue.

Mediante GitHub Actions se ha automatizado:

- Instalación de dependencias.
- Ejecución de pruebas.

- Construcción de imágenes Docker.
- Publicación en GitHub Container Registry (GHCR).

Esta aproximación permite disponer de un pipeline CI/CD completo sin necesidad de mantener infraestructura adicional.

5.3 Selección del Registro de Contenedores

Una vez construidas las imágenes Docker, resulta necesario almacenarlas en un registro centralizado desde el que puedan ser descargadas por Kubernetes.

Se evaluaron las siguientes alternativas:

- Docker Hub
- GitHub Container Registry (GHCR)
- Azure Container Registry (ACR)

5.3.1 Docker Hub

Ventajas

- Amplia adopción.
- Facilidad de uso.
- Gran comunidad.

Desventajas

- Limitaciones en planes gratuitos.
- Menor integración con GitHub.
- Restricciones en algunas funcionalidades avanzadas.

5.3.2 Azure Container Registry

Ventajas

- Integración con Azure.
- Seguridad empresarial.
- Gestión avanzada de permisos.

Desventajas

- Dependencia de Azure.
- Complejidad adicional para proyectos pequeños.

5.3.3 GitHub Container Registry

Ventajas

- Integración nativa con GitHub Actions.

- Gestión unificada junto al código fuente.
- Control centralizado de versiones.
- Facilidad de configuración.

Desventajas

- Dependencia del ecosistema GitHub.

5.3.4 Decisión Adoptada

Tecnología seleccionada: GitHub Container Registry (GHCR)

Justificación

La utilización de GHCR permite simplificar la arquitectura y centralizar la gestión del código fuente, la automatización CI/CD y las imágenes Docker dentro del mismo ecosistema.

Esta integración ha permitido automatizar completamente la publicación de imágenes y su consumo posterior por Kubernetes.

5.4 Infraestructura de Ejecución

La validación funcional del proyecto se realizó sobre un clúster Kubernetes ejecutado en Docker Desktop.

La solución ha sido diseñada siguiendo principios Cloud Native, permitiendo su futura migración a plataformas gestionadas como:

- Azure Kubernetes Service (AKS)
- Amazon Elastic Kubernetes Service (EKS)
- Google Kubernetes Engine (GKE)

Sin embargo, para el alcance del presente proyecto, la infraestructura utilizada se limita a un entorno Kubernetes local completamente funcional y suficiente para validar todas las capacidades implementadas.

5.5 Conclusión

Tras el análisis realizado, se considera que la combinación formada por **Docker**, **Kubernetes**, **GitHub Actions** y **GitHub Container Registry** ofrece el mejor equilibrio entre simplicidad, automatización, escalabilidad y alineación con las prácticas DevOps modernas.

La arquitectura resultante ha permitido implementar y validar satisfactoriamente una plataforma capaz de automatizar el ciclo completo de construcción, publicación y despliegue de aplicaciones contenerizadas, estableciendo

además una base tecnológica sólida para futuras evoluciones hacia entornos cloud y soluciones avanzadas de observabilidad.

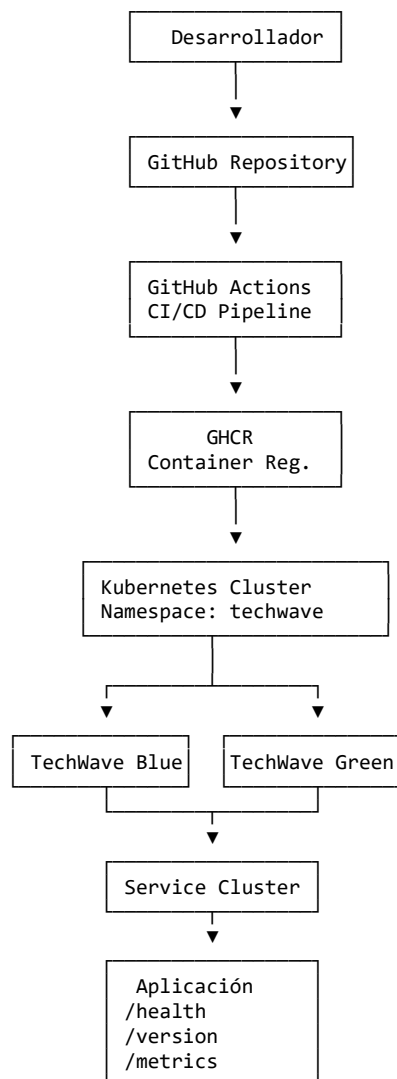
6 Diseño Arquitectónico

La arquitectura de la plataforma TechWave DevOps ha sido diseñada siguiendo principios Cloud Native y buenas prácticas DevOps, con el objetivo de proporcionar una solución automatizada, escalable y fácilmente mantenible.

La solución implementada integra un repositorio GitHub como fuente oficial del código, GitHub Actions como plataforma de integración y entrega continua, GitHub Container Registry (GHCR) como repositorio de imágenes Docker y Kubernetes como plataforma de orquestación para la ejecución de la aplicación.

La arquitectura incorpora además una estrategia de despliegue Blue-Green, permitiendo la ejecución simultánea de múltiples versiones de la aplicación y facilitando procesos seguros de actualización y recuperación.

Figura 6.1. Arquitectura implementada de la plataforma TechWave DevOps



6.1 Introducción

Una vez definidos los objetivos estratégicos, los requisitos de negocio y las decisiones tecnológicas adoptadas, se diseña una arquitectura objetivo capaz de soportar el ciclo completo de desarrollo, integración y despliegue de aplicaciones contenerizadas.

La arquitectura propuesta se apoya en tecnologías ampliamente adoptadas dentro del ecosistema DevOps y Cloud Native, garantizando:

- Escalabilidad.
- Disponibilidad.
- Automatización.
- Trazabilidad.
- Reproducibilidad.
- Capacidad de evolución futura.

El diseño prioriza la simplicidad operativa y la automatización de procesos, minimizando la intervención manual durante el ciclo de vida de la aplicación.

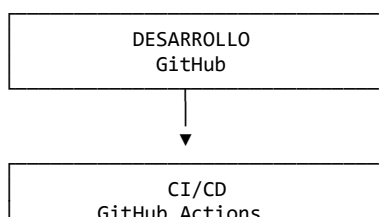
6.2 Arquitectura de Alto Nivel

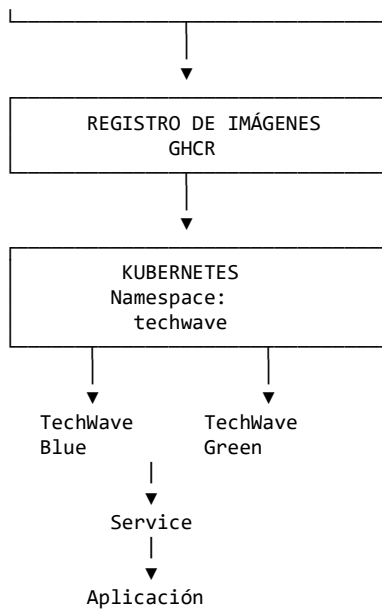
6.2.1 Visión Ejecutiva

Desde una perspectiva funcional, la solución se estructura en cinco dominios principales:

- **Desarrollo** Responsable de la creación y mantenimiento del código fuente mediante GitHub.
- **Integración Continua** Responsable de validar automáticamente el código fuente y generar nuevas versiones de la aplicación mediante GitHub Actions.
- **Registro de Artefactos** Gestiona el almacenamiento de imágenes Docker mediante GitHub Container Registry (GHCR).
- **Plataforma de Ejecución** Entorno Kubernetes encargado de ejecutar y gestionar la aplicación.
- **Observabilidad** Mecanismos de supervisión implementados mediante la exposición de métricas operativas compatibles con Prometheus.

6.2.2 Arquitectura Ejecutiva





6.3 Arquitectura Lógica

6.3.1 Principios de Diseño

La arquitectura lógica se ha construido siguiendo los siguientes principios:

- **Separación de responsabilidades** Cada componente realiza una función claramente definida.
- **Automatización** Las operaciones repetitivas deben ejecutarse de forma automática siempre que sea posible.
- **Desacoplamiento** Las distintas capas interactúan mediante interfaces estandarizadas.
- **Observabilidad** La aplicación debe exponer información suficiente para facilitar la supervisión de su comportamiento.
- **Recuperabilidad** La plataforma debe facilitar la recuperación rápida ante incidencias.

6.3.2 Componentes Lógicos

Repositorio GitHub

Responsable de:

- Gestión del código fuente.
- Control de versiones.
- Gestión de ramas.
- Trazabilidad de cambios.

GitHub Actions

Responsable de:

- Integración continua.
- Ejecución de pruebas.
- Construcción de imágenes Docker.
- Publicación automatizada.

GitHub Container Registry (GHCR)

Responsable de:

- Almacenamiento de imágenes Docker.
- Versionado.
- Distribución de artefactos.

Kubernetes

Responsable de:

- Orquestación.
- Gestión de Deployments.
- Alta disponibilidad.
- Recuperación automática.

Aplicación Web

Responsable de:

- Exposición de APIs.
- Endpoint de salud.
- Endpoint de versión.
- Exposición de métricas.

ConfigMaps y Secrets

Responsables de:

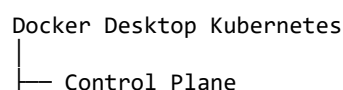
- Gestión de configuración.
- Separación de secretos.
- Parametrización de despliegues.

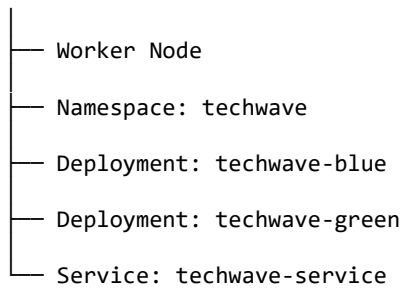
6.4 Arquitectura Física

6.4.1 Infraestructura Implementada

Las validaciones funcionales del proyecto se realizaron sobre un entorno Kubernetes ejecutado mediante Docker Desktop.

La arquitectura física utilizada está formada por:





6.4.2 Distribución Física de Servicios

Capa de Aplicación

- Pods Blue.
- Pods Green.
- Service Kubernetes.

Capa de Plataforma

- ConfigMaps.
- Secrets.
- Deployments.
- ReplicaSets.

Capa de Observabilidad

- Endpoint /metrics.
- Prometheus Client para Node.js.

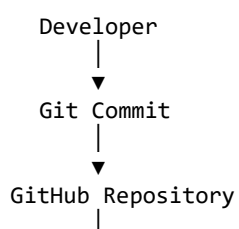
6.5 Arquitectura de Red

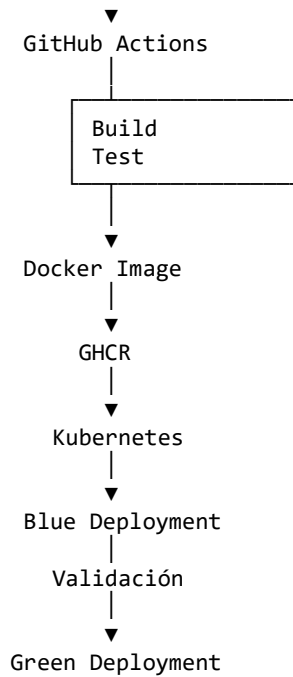
La solución utiliza la red interna proporcionada por Kubernetes para la comunicación entre componentes.

Objetivos

- Aislamiento lógico.
- Simplicidad operativa.
- Escalabilidad.
- Disponibilidad.

6.6 Flujo de Despliegue





6.7 Observabilidad

La aplicación expone métricas compatibles con Prometheus mediante el endpoint:

/metrics

Las métricas recopiladas incluyen:

- Consumo de CPU.
- Consumo de memoria.
- Información del Event Loop.
- Estadísticas de Garbage Collector.
- Métricas internas de Node.js.

La arquitectura ha sido diseñada para facilitar futuras integraciones con:

- Prometheus.
- Grafana.
- Loki.
- OpenTelemetry.

aunque dichas plataformas quedan consideradas como evolución futura de la solución.

7 Gobierno DevOps

El modelo de Gobierno DevOps propuesto permite establecer una estructura organizativa clara, definiendo responsabilidades, mecanismos de coordinación y criterios de operación. La combinación de equipos especializados, principios DevSecOps y una matriz RACI formal proporciona una base sólida para operar la plataforma de forma escalable, segura y alineada con los objetivos corporativos de TechWave Solutions. Este marco de gobierno servirá como referencia para los capítulos posteriores de diseño técnico, despliegue y operación de la plataforma.

7.1 Introducción

La implantación de una plataforma DevOps no debe limitarse a la adopción de herramientas tecnológicas. Numerosos proyectos fracasan no por problemas de infraestructura o software, sino por la ausencia de un modelo de gobierno que defina claramente responsabilidades, procesos de decisión y mecanismos de coordinación entre los distintos equipos.

El Gobierno DevOps tiene como objetivo establecer un marco organizativo que permita a TechWave Solutions operar su plataforma de forma eficiente, segura y alineada con los objetivos de negocio.

Este modelo busca equilibrar tres factores fundamentales:

- Velocidad de entrega.
- Estabilidad operativa.
- Seguridad.

La colaboración entre las áreas de Desarrollo, Operaciones y Seguridad constituye uno de los pilares principales de la estrategia propuesta.

7.2 Principios de Gobierno

El modelo de gobierno se fundamenta en los siguientes principios.

PG-01 Responsabilidad Compartida

La calidad y estabilidad de la plataforma no son responsabilidad exclusiva de un único departamento.

Todos los equipos participan en el éxito o fracaso del servicio.

PG-02 Automatización por Defecto

Cualquier actividad repetitiva deberá evaluarse para su automatización.

El objetivo es reducir:

- Errores humanos.
- Costes operativos.
- Dependencia de conocimiento individual.

PG-03 Seguridad Integrada

La seguridad no se considera una fase final del proyecto.

Los controles de seguridad deben incorporarse desde las primeras etapas del ciclo de vida de desarrollo.

PG-04 Observabilidad Integral

Todas las decisiones operativas deben basarse en métricas objetivas.

La plataforma deberá generar información suficiente para facilitar la toma de decisiones basada en evidencias.

PG-05 Mejora Continua

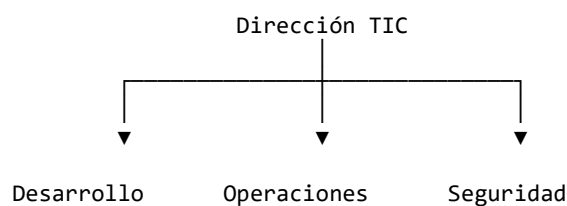
Los procesos deberán revisarse periódicamente para identificar oportunidades de mejora.

Se promoverá una cultura basada en:

- Aprendizaje continuo.
- Automatización.
- Optimización.
- Gestión del conocimiento.

7.3 Estructura Organizativa

La operación de la plataforma se apoyará en tres dominios funcionales principales.



Cada equipo mantiene responsabilidades específicas, aunque existe una elevada colaboración transversal.

7.4 Equipo de Desarrollo

7.4.1 Misión

Diseñar, desarrollar y evolucionar las aplicaciones de negocio garantizando que cumplan los estándares de calidad, seguridad y mantenibilidad establecidos por la organización.

7.4.2 Responsabilidades

Desarrollo de funcionalidades

Implementación del software de acuerdo con los requisitos de negocio.

Gestión del código fuente

Administración de repositorios Git, ramas y revisiones de código.

Calidad del software

Ejecución de:

- Pruebas unitarias.
- Pruebas de integración.
- Validaciones funcionales.

Instrumentación de observabilidad

Incorporación de:

- Logs estructurados.
- Métricas.
- Trazas OpenTelemetry.

Definición de despliegues

Mantenimiento de:

- Dockerfiles.
- Configuración de aplicación.
- Artefactos de despliegue.

Corrección de incidencias

Análisis y resolución de defectos detectados durante la operación.

7.4.3 Indicadores Asociados

- Número de despliegues exitosos.
- Cobertura de pruebas.
- Tasa de defectos.

- Tiempo de corrección de incidencias.

7.5 Equipo de Operaciones

7.5.1 Misión

Garantizar la disponibilidad, estabilidad y rendimiento de la plataforma tecnológica.

7.5.2 Responsabilidades

Gestión de Kubernetes

Administración de:

- Clúster AKS.
- Nodos.
- Namespaces.
- Recursos compartidos.

Gestión de infraestructura

Administración de recursos desplegados mediante Terraform.

Gestión de despliegues

Supervisión de:

- Releases.
- Despliegues Blue-Green.
- Rollbacks.

Observabilidad

Administración de:

- Prometheus.
- Grafana.
- Loki.
- OpenTelemetry.

Gestión de capacidad

Monitorización y planificación de recursos.

Continuidad operativa

Gestión de incidencias y recuperación ante fallos.

7.5.3 Indicadores Asociados

- Disponibilidad.

- MTTR.
- Consumo de recursos.
- Tasa de éxito de despliegues.

7.6 Equipo de Seguridad

7.6.1 Misión

Garantizar que la plataforma cumple los requisitos de seguridad establecidos por la organización y las buenas prácticas de la industria.

7.6.2 Responsabilidades

Gestión de accesos

Supervisión de identidades y permisos.

Auditoría

Revisión periódica de:

- Configuraciones.
- Accesos.
- Actividades críticas.

Gestión de vulnerabilidades

Identificación y seguimiento de riesgos de seguridad.

Gestión de secretos

Supervisión del uso de:

- Kubernetes Secrets.
- GitHub Secrets.
- Credenciales corporativas.

Cumplimiento normativo

Verificación del cumplimiento de políticas internas y requisitos regulatorios.

Respuesta ante incidentes

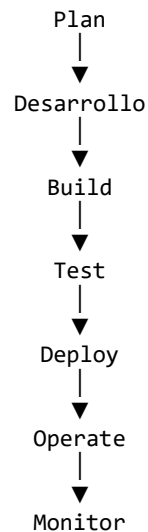
Colaboración en la gestión de eventos de seguridad.

7.6.3 Indicadores Asociados

- Vulnerabilidades detectadas.
- Tiempo de resolución.
- Auditorías completadas.
- Incidentes de seguridad.

7.7 Modelo DevSecOps

La organización adoptará un enfoque DevSecOps donde la seguridad se integra a lo largo de todo el ciclo de vida.



La seguridad participa en todas las fases del proceso.

7.8 Modelo RACI

Para evitar ambigüedades se define una matriz RACI que establece el nivel de responsabilidad de cada equipo.

Definiciones

R (Responsible) Ejecuta la actividad.

A (Accountable) Responsable último del resultado.

C (Consulted) Participa mediante asesoramiento.

I (Informed) Debe ser informado.

7.8.1 Matriz RACI Operativa

Actividad	Desarrollo	Operaciones	Seguridad
Desarrollo de funcionalidades	R/A	I	I
Revisión de código	R/A	I	C
Gestión de repositorios Git	R	C	I
Definición de Dockerfile	R	C	C
Gestión de Terraform	C	R/A	C

Actividad	Desarrollo	Operaciones	Seguridad
Administración de AKS	I	R/A	C
Gestión de Kubernetes	C	R/A	C
Configuración CI/CD	R	R	C
Gestión de GitHub Actions	C	R/A	C
Gestión de secretos	I	C	R/A
Escaneo de vulnerabilidades	C	C	R/A
Gestión de alertas	I	R/A	C
Monitorización de plataforma	I	R/A	C
Respuesta a incidencias operativas	C	R/A	I
Respuesta a incidentes de seguridad	I	C	R/A
Auditorías de seguridad	I	C	R/A
Gestión de rollback	C	R/A	I
Gestión del cambio	C	R/A	C

7.9 Comité de Gobierno DevOps

Se recomienda establecer un comité de seguimiento con reuniones periódicas.

Participantes

- Responsable de Desarrollo.
- Responsable de Operaciones.
- Responsable de Seguridad.
- Responsable de Proyecto.

Objetivos

Seguimiento operativo Revisar disponibilidad, incidencias y rendimiento.

Seguimiento de despliegues Evaluar releases y cambios relevantes.

Seguimiento de seguridad Revisar vulnerabilidades y riesgos abiertos.

Seguimiento de evolución Priorizar mejoras de la plataforma.

Periodicidad Recomendada

Reunión	Frecuencia
Operación diaria	Diaria
Seguimiento DevOps	Semanal
Comité técnico	Mensual
Comité estratégico	Trimestral

7.10 KPIs de Gobierno

La madurez del modelo DevOps será evaluada mediante indicadores clave.

Indicador	Objetivo
Frecuencia de despliegue	> 1 por semana
Tiempo de despliegue	< 10 min
Disponibilidad	> 99,5 %
MTTR	< 15 min
Despliegues con rollback	< 5 %
Vulnerabilidades críticas abiertas	0
Automatización de despliegues	> 95 %
Cobertura de observabilidad	100 %

8 Diseño de la Aplicación

El diseño de la aplicación se ha realizado siguiendo principios Cloud Native y DevOps, priorizando la simplicidad, la observabilidad, la escalabilidad y la mantenibilidad. La separación entre lógica de negocio, configuración, gestión de errores y observabilidad garantiza que la solución pueda integrarse de forma natural dentro de la plataforma Kubernetes propuesta, facilitando tanto la operación diaria como la evolución futura del sistema.

8.1 Introducción

La aplicación desarrollada para TechWave Solutions constituye el componente principal sobre el que se desplegarán todas las capacidades DevOps descritas en los capítulos anteriores.

Su diseño ha sido realizado siguiendo principios Cloud Native, favoreciendo la contenerización, la observabilidad, la escalabilidad y la automatización del ciclo de vida.

Más allá de la funcionalidad de negocio, la aplicación ha sido concebida para convertirse en un modelo de referencia para futuras aplicaciones de la organización, estableciendo patrones comunes de desarrollo, configuración, despliegue y operación.

8.2 Objetivos de Diseño

El diseño de la aplicación persigue los siguientes objetivos:

- **OD-01 Simplicidad** Disponer de una aplicación sencilla que permita demostrar todas las capacidades de la plataforma DevOps.
- **OD-02 Desacoplamiento** Separar claramente la lógica de negocio de la configuración y la infraestructura.
- **OD-03 Portabilidad** Garantizar que la aplicación pueda ejecutarse de forma idéntica en cualquier entorno.
- **OD-04 Observabilidad** Integrar capacidades nativas de logs, métricas y trazas.
- **OD-05 Escalabilidad** Permitir la ejecución simultánea de múltiples instancias sin dependencia de estado local.
- **OD-06 Recuperación** Facilitar reinicios y despliegues sin pérdida de información ni impacto significativo sobre el usuario.

8.3 Arquitectura Software

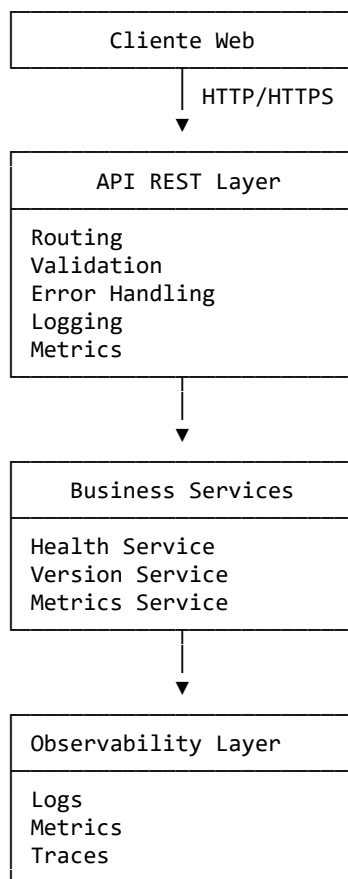
8.3.1 Estilo Arquitectónico

La aplicación seguirá una arquitectura basada en servicios web ligeros (Web API), orientada a la simplicidad y alineada con los principios Twelve-Factor App.

Características principales:

- Stateless.
- Configuración externa.
- Logs centralizados.
- Escalabilidad horizontal.
- Automatización completa del despliegue.

8.3.2 Arquitectura Lógica



8.3.3 Principios de Diseño

Stateless La aplicación no almacenará estado local persistente.

Esto permite:

- Escalado horizontal.

- Recuperación automática.
- Sustitución transparente de pods.

Configuración Externa Toda configuración se almacenará fuera de la aplicación mediante:

- ConfigMaps.
- Secrets.
- Variables de entorno.

Observabilidad por Diseño La aplicación deberá generar información operacional desde su concepción.

Inmutabilidad Las imágenes Docker generadas serán inmutables y versionadas.

8.4 Diseño de APIs

La plataforma expondrá una API REST sencilla pero suficiente para demostrar las capacidades operativas de la solución.

8.4.1 Endpoint Principal

Objetivo Verificar acceso a la aplicación.

Endpoint GET /

Respuesta

```
{
  "application": "TechWave DevOps Platform",
  "status": "running"
}
```

Uso Operacional

- Validación funcional.
- Pruebas de conectividad.
- Comprobaciones de despliegue.

8.4.2 Endpoint de Estado

Objetivo Permitir verificar la salud de la aplicación.

Endpoint

GET /health

Respuesta

```
{
  "status": "UP"
}
```

Uso Operacional

- Readiness Probe.

- Liveness Probe.
- Validaciones automáticas.

8.4.3 Endpoint de Versión

Objetivo Identificar la versión desplegada.

Endpoint

GET /version

Respuesta

```
{  
  "version": "1.0.0"  
}
```

Uso Operacional

- Blue-Green Deployment.
- Auditoría.
- Resolución de incidencias.

8.4.4 Endpoint de Métricas

Objetivo Exponer métricas Prometheus.

Endpoint

GET /metrics

Uso Operacional

- Monitorización.
- Dashboards.
- Alertas.

8.5 Gestión de Configuración

8.5.1 Principios Generales

La configuración debe permanecer separada del código fuente.

Esta estrategia evita:

- Modificaciones innecesarias del código.
- Promociones complejas entre entornos.
- Riesgos asociados a configuraciones embebidas.

8.5.2 Clasificación de Configuración

Configuración Funcional

Parámetros relacionados con el comportamiento de la aplicación.

Ejemplos:

```
APP_NAME
APP_VERSION
LOG_LEVEL
```

Configuración Técnica

Parámetros relacionados con la plataforma.

Ejemplos:

```
PORT
TIMEOUT
ENVIRONMENT
```

Configuración Sensible

Información considerada confidencial.

Ejemplos:

```
TOKENS
API_KEYS
PASSWORDS
```

8.5.3 Estrategia de Gestión

Tipo	Almacenamiento
Configuración funcional	ConfigMap
Configuración técnica	ConfigMap
Información sensible	Kubernetes Secret

8.6 Gestión de Variables

8.6.1 Objetivos

La gestión de variables busca:

- Facilitar despliegues multi-entorno.
- Evitar modificaciones de código.
- Reducir riesgos de configuración.

8.6.2 Variables por Entorno

Desarrollo

```
ENV=DEV
LOG_LEVEL=DEBUG
```

Preproducción

```
ENV=PRE
LOG_LEVEL=INFO
```

Producción

```
ENV=PROD  
LOG_LEVEL=WARN
```

8.6.3 Convenciones de Nomenclatura

Las variables deberán cumplir las siguientes normas:

```
MAYÚSCULAS  
SEPARADAS_POR_GUIÓN_BAJO
```

Ejemplos:

```
APP_NAME  
APP_VERSION  
DATABASE_HOST  
LOG_LEVEL
```

8.6.4 Ciclo de Vida

Definición Configuradas en Kubernetes.

Consumo Leídas por la aplicación al arrancar.

Actualización Mediante despliegue controlado.

Auditoría Gestionadas mediante Git y Kubernetes.

8.7 Gestión de Errores

8.7.1 Objetivos

La estrategia de gestión de errores persigue:

- Detectar problemas rápidamente.
- Facilitar diagnósticos.
- Reducir tiempos de recuperación.
- Generar información útil para observabilidad.

8.7.2 Clasificación de Errores

Errores de Cliente

Causados por solicitudes incorrectas.

Ejemplos:

```
400 Bad Request  
404 Not Found
```

Errores de Aplicación

Originados por fallos internos.

Ejemplos:

```
500 Internal Server Error
```


Errores de Plataforma

Relacionados con infraestructura.

Ejemplos:

- Fallo de red.
- Error DNS.
- Saturación de recursos.

8.7.3 Estructura Estándar de Respuesta

```
{
  "timestamp": "2026-01-15T10:00:00Z",
  "status": 500,
  "error": "Internal Server Error",
  "message": "Unexpected application error",
  "traceId": "abc123"
}
```

8.7.4 Correlación de Errores

Cada petición dispondrá de un identificador único.

Beneficios:

- Seguimiento extremo a extremo.
- Integración con OpenTelemetry.
- Diagnóstico distribuido.

8.8 Estrategia de Logging

8.8.1 Objetivos

El sistema de logging debe proporcionar información suficiente para:

- Monitorización.
- Diagnóstico.
- Auditoría.
- Investigación de incidencias.

8.8.2 Principios

Logs estructurados Formato JSON para facilitar el procesamiento automático.

Centralización Todos los logs serán enviados a Loki.

Correlación Cada registro contendrá:

- Timestamp.
- Nivel.
- Servicio.
- Trace ID.

Inmutabilidad Los logs no podrán ser modificados una vez almacenados.

8.8.3 Niveles de Logging

- **DEBUG** Información detallada para desarrollo.
- **INFO** Eventos normales de operación.
- **WARN** Situaciones anómalas sin impacto crítico.
- **ERROR** Errores que afectan a una operación.
- **FATAL** Errores que impiden el funcionamiento del servicio.

8.8.4 Ejemplo de Registro

```
{
  "timestamp": "2026-01-15T10:00:00Z",
  "level": "INFO",
  "service": "techwave-app",
  "message": "User request processed successfully",
  "traceId": "7f4a8e51"
}
```

8.9 Integración con la Observabilidad

La aplicación será diseñada para integrarse de forma nativa con la plataforma de observabilidad corporativa.

Métricas

Exportadas a Prometheus.

Ejemplos:

- Tiempo de respuesta.
- Número de peticiones.
- Errores por segundo.
- Uso de memoria.

Logs

Centralizados en Loki.

Permitirán:

- Investigación de incidencias.
- Auditoría.
- Diagnóstico operativo.

Trazas

Gestionadas mediante OpenTelemetry.

Permitirá conocer:

- Ruta completa de una petición.
- Dependencias.
- Bottlenecks de rendimiento.

9 Estrategia de Contenerización

La estrategia de contenerización propuesta establece un modelo corporativo para la construcción y distribución de imágenes Docker alineado con los principios DevOps, Cloud Native y DevSecOps. Mediante el uso de imágenes optimizadas, versionado semántico, ejecución sin privilegios, escaneo automático de vulnerabilidades y reducción sistemática de la superficie de ataque, TechWave Solutions dispondrá de una base sólida para ejecutar sus aplicaciones de forma segura, eficiente y escalable dentro de la plataforma Kubernetes definida en los capítulos anteriores.

9.1 Introducción

La contenerización constituye uno de los pilares fundamentales de la arquitectura propuesta para TechWave Solutions. La adopción de Docker permite encapsular la aplicación junto con todas sus dependencias, garantizando la consistencia entre entornos de desarrollo, pruebas y producción.

La estrategia definida en este capítulo persigue no sólo facilitar el despliegue de aplicaciones, sino también establecer un estándar corporativo para la construcción, distribución y operación de imágenes de contenedor alineado con las mejores prácticas DevOps y Cloud Native.

Los objetivos principales de esta estrategia son:

- Garantizar la portabilidad de las aplicaciones.
- Reducir diferencias entre entornos.
- Mejorar la seguridad.
- Simplificar los despliegues.
- Facilitar la escalabilidad.
- Optimizar el consumo de recursos.
- Asegurar la trazabilidad de versiones.

9.2 Principios de Contenerización

La estrategia de contenerización se apoya en los siguientes principios:

PC-01 Inmutabilidad Una imagen generada no debe modificarse posteriormente.

Cualquier cambio deberá producir una nueva versión de la imagen.

Beneficios

- Reproducibilidad.
- Trazabilidad.
- Facilidad de rollback.

- Reducción de errores de configuración.

PC-02 Portabilidad Una misma imagen deberá poder ejecutarse sin modificaciones en:

- Entornos de desarrollo.
- Entornos de integración.
- Entornos de pruebas.
- Entornos de producción.

PC-03 Automatización La construcción de imágenes deberá realizarse exclusivamente mediante procesos automatizados integrados en el pipeline CI/CD.

PC-04 Seguridad por Diseño Las imágenes deberán construirse aplicando medidas de seguridad desde las primeras fases del proceso.

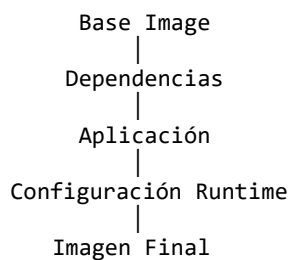
PC-05 Minimalismo Las imágenes deberán contener únicamente los componentes estrictamente necesarios para ejecutar la aplicación.

9.3 Diseño de Imágenes

9.3.1 Modelo de Imagen

Cada aplicación dispondrá de una imagen Docker propia.

La estructura general será:



9.3.2 Selección de Imagen Base

La selección de la imagen base se realizará siguiendo los siguientes criterios:

Seguridad Uso de imágenes oficiales y mantenidas.

Tamaño reducido Prioridad a distribuciones ligeras.

Compatibilidad Alineación con el lenguaje y framework utilizados.

Soporte Disponibilidad de actualizaciones y parches.

Imágenes Recomendadas

Para aplicaciones Node.js:

```
node:20-alpine
```

Para componentes auxiliares:

```
alpine:latest
```

9.3.3 Construcción de Imágenes

La construcción deberá cumplir los siguientes principios:

Separación por capas Aprovechar la caché de Docker para acelerar compilaciones.

Menor número de capas Reducir complejidad y tamaño.

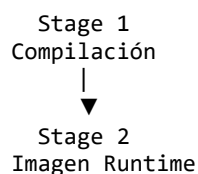
Dependencias controladas Instalar únicamente paquetes requeridos.

Eliminación de artefactos temporales Evitar componentes innecesarios en la imagen final.

9.3.4 Estrategia Multi-Stage Build

Siempre que sea posible se utilizarán imágenes multi-stage.

Ejemplo conceptual



Beneficios

- Menor tamaño.
- Menor superficie de ataque.
- Menor consumo de almacenamiento.
- Distribución más eficiente.

9.4 Estrategia de Versionado

9.4.1 Objetivos

La estrategia de versionado debe permitir:

- Identificar de forma única cada release.
- Facilitar auditorías.
- Simplificar rollback.

- Correlacionar código e imágenes.

9.4.2 Versionado Semántico

Se adoptará el estándar Semantic Versioning (SemVer).

Formato:

MAJOR.MINOR.PATCH

Ejemplos:

1.0.0
1.1.0
1.1.1
2.0.0

MAJOR Cambios incompatibles.

Ejemplo: 1.x → 2.x

MINOR Nuevas funcionalidades compatibles.

Ejemplo: 1.0 → 1.1

PATCH Corrección de errores.

Ejemplo: 1.1.0 → 1.1.1

9.4.3 Etiquetado de Imágenes

Todas las imágenes deberán disponer de etiquetas claras y trazables.

Ejemplos:

techwave-app:1.0.0
techwave-app:1.1.0
techwave-app:2.0.0

Adicionalmente:

techwave-app:latest

9.4.4 Versionado Operacional

Cada imagen deberá incluir metadatos que permitan identificar:

- Versión de aplicación.
- Fecha de construcción.
- Commit Git origen.
- Pipeline que generó la imagen.

Ejemplo:

Version: 1.2.3
Git Commit: a73f92d
Build Date: 2026-01-15

9.5 Optimización de Imágenes

9.5.1 Objetivos

La optimización persigue:

- Reducir tiempos de descarga.
- Reducir almacenamiento.
- Mejorar rendimiento.
- Disminuir superficie de ataque.

9.5.2 Minimización del Tamaño

Se aplicarán las siguientes medidas:

Uso de Alpine Linux Distribución ligera ampliamente utilizada en contenedores.

Eliminación de dependencias innecesarias No instalar herramientas de desarrollo en producción.

Exclusión de artefactos temporales Eliminar:

- cache
- logs locales
- ficheros temporales

Uso de .dockerignore Excluir:

- node_modules
- .git
- docs
- tests

9.5.3 Optimización de Capas

Se minimizará el número de capas generadas.

Beneficios:

- Menor tamaño.
- Mejor rendimiento.
- Menor complejidad.

9.5.4 Caché de Construcción

La estructura del Dockerfile deberá favorecer la reutilización de capas.

Beneficios:

- Builds más rápidas.
- Menor consumo de recursos CI/CD.

9.6 Estrategia de Seguridad

9.6.1 Objetivos

Los contenedores constituyen una pieza crítica de la plataforma y deben cumplir requisitos estrictos de seguridad.

La estrategia persigue:

- Reducir superficie de ataque.
- Minimizar privilegios.
- Detectar vulnerabilidades.
- Garantizar trazabilidad.

9.7 Usuario No Root

9.7.1 Justificación

Por defecto muchos contenedores se ejecutan con privilegios elevados.

Esta práctica incrementa significativamente el riesgo de seguridad.

9.7.2 Política Corporativa

Todas las aplicaciones deberán ejecutarse mediante usuarios no privilegiados.

Ejemplo conceptual:

```
RUN adduser appuser
```

```
USER appuser
```

9.7.3 Beneficios

Reducción de privilegios Limita el impacto de una posible vulnerabilidad.

Aislamiento Reduce el riesgo de escalada de privilegios.

Cumplimiento normativo Alineado con estándares de seguridad modernos.

9.8 Escaneo de Vulnerabilidades

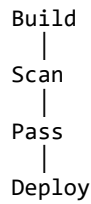
9.8.1 Objetivo

Identificar vulnerabilidades antes de desplegar imágenes en producción.

9.8.2 Integración en CI/CD

Todo pipeline deberá incorporar análisis automáticos.

Flujo:



9.8.3 Herramientas Evaluadas

Trivy

Ventajas:

- Open Source.
- Fácil integración.
- Amplia adopción.

Docker Scout

Ventajas:

- Integración con Docker.
- Visibilidad continua.

Snyk

Ventajas:

- Excelente análisis de dependencias.

9.8.4 Tecnología Seleccionada

Trivy

Motivos:

- Gratuito.
- Fácil integración con GitHub Actions.
- Amplia adopción en entornos Kubernetes.
- Bajo coste operativo.

9.8.5 Política de Aceptación

No se permitirá el despliegue automático de imágenes que contengan:

Vulnerabilidades críticas

Las vulnerabilidades altas deberán ser revisadas antes de autorizar el despliegue.

9.9 Reducción de Superficie de Ataque

9.9.1 Principio

Cuantos menos elementos existan dentro del contenedor, menor será la probabilidad de explotación.

9.9.2 Medidas Aplicadas

Eliminación de herramientas innecesarias No incluir:

- vi
- curl
- wget
- ssh
- compiladores

salvo necesidad justificada.

Eliminación de paquetes no utilizados Instalar exclusivamente dependencias requeridas por la aplicación.

Sistema de archivos mínimo Reducir componentes del sistema operativo al mínimo imprescindible.

Puertos mínimos Exponer únicamente los puertos requeridos.

Ejemplo:

```
EXPOSE 3000
```

Procesos mínimos Un proceso principal por contenedor. Beneficios:

- Simplicidad.
- Facilidad de monitorización.
- Menor consumo.

9.10 Registro y Distribución de Imágenes

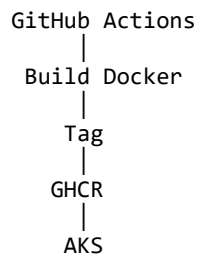
9.10.1 Registro Corporativo

Las imágenes se almacenarán en:

```
GitHub Container Registry (GHCR)
```

9.10.2 Flujo de Distribución

```
Developer
|
GitHub
|
```



9.10.3 Política de Retención

Versiones activas Mantener últimas versiones estables.

Versiones históricas Conservar releases relevantes para auditoría y rollback.

Imágenes obsoletas Eliminar automáticamente según política de ciclo de vida.

9.11 Controles de Cumplimiento

Antes de publicar una imagen en producción deberán verificarse los siguientes requisitos:

Control	Obligatorio
Imagen oficial	Sí
Multi-stage build	Sí
Usuario no root	Sí
Escaneo de vulnerabilidades	Sí
Versionado semántico	Sí
Etiquetado de imágenes	Sí
Registro centralizado	Sí
Imagen optimizada	Sí
Sin secretos embebidos	Sí

9.12 Indicadores de Calidad

Indicador	Objetivo
Tamaño medio de imagen	< 250 MB

Indicador	Objetivo
Vulnerabilidades críticas	0
Vulnerabilidades altas	0 en producción
Tiempo medio de build	< 5 minutos
Cobertura de escaneo	100 %
Imágenes trazables	100 %

10 Plataforma Kubernetes

La plataforma Kubernetes propuesta constituye el núcleo operativo de la solución DevOps de TechWave Solutions. La combinación de Namespaces, Deployments, Services, Ingress, ConfigMaps, Secrets, HPA, políticas de afinidad, tolerancias y sondas de salud proporciona una arquitectura robusta, escalable y preparada para entornos productivos. Su diseño garantiza el cumplimiento de los requisitos de disponibilidad, recuperación, seguridad y automatización definidos en los capítulos anteriores, estableciendo además una base sólida para la evolución futura de la organización hacia modelos avanzados de GitOps, SRE y Platform Engineering.

10.1 Introducción

Kubernetes constituye el núcleo operativo de la arquitectura propuesta para TechWave Solutions. Su adopción responde a la necesidad de disponer de una plataforma capaz de gestionar aplicaciones contenerizadas de forma automatizada, escalable, resiliente y observable.

La elección de Kubernetes permite desacoplar las aplicaciones de la infraestructura subyacente y establecer un modelo declarativo donde el estado deseado del sistema se define mediante código, alineándose con los principios DevOps e Infraestructura como Código definidos en capítulos anteriores.

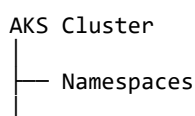
Los objetivos principales perseguidos con la implantación de Kubernetes son:

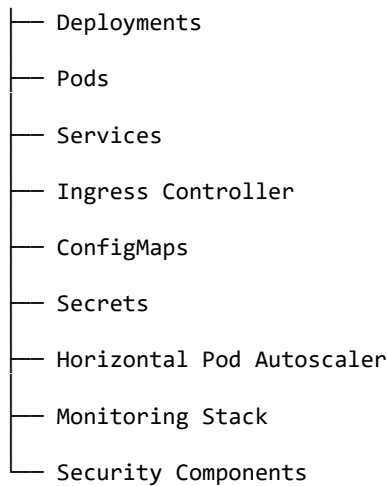
- Garantizar alta disponibilidad.
- Facilitar el escalado horizontal.
- Automatizar la recuperación ante fallos.
- Simplificar despliegues y actualizaciones.
- Mejorar la gestión de configuración.
- Reducir la intervención manual.
- Establecer una base tecnológica preparada para futuras evoluciones.

10.2 Arquitectura Kubernetes Propuesta

10.2.1 Componentes Principales

La plataforma estará desplegada sobre Azure Kubernetes Service (AKS), utilizando una arquitectura lógica compuesta por:





10.2.2 Principios de Diseño

Declaratividad Toda la configuración se gestionará mediante ficheros YAML versionados.

Inmutabilidad Los cambios en las aplicaciones se realizarán mediante nuevas imágenes Docker.

Automatización Las operaciones rutinarias serán gestionadas por Kubernetes.

Alta Disponibilidad La plataforma minimizará puntos únicos de fallo.

Escalabilidad La carga podrá distribuirse entre múltiples instancias.

10.3 Namespace

10.3.1 Objetivo

Los Namespaces permiten segmentar lógicamente los recursos dentro del clúster.

Esta separación aporta:

- Aislamiento organizativo.
- Simplificación administrativa.
- Gestión independiente de recursos.
- Aplicación diferenciada de políticas.

Diseño Propuesto

Se propone utilizar diferentes namespaces en función del entorno y del propósito de los servicios.

techwave-dev
techwave-pre

```
techwave-prod
monitoring
ingress-nginx
```

10.3.2 Beneficios

Seguridad Reduce la exposición entre servicios.

Organización Facilita el gobierno del clúster.

Escalabilidad Permite incorporar nuevos proyectos sin afectar a los existentes.

10.4 Deployments

10.4.1 Objetivo

Los Deployments constituyen el mecanismo principal para gestionar aplicaciones en Kubernetes. Permiten definir:

- Número de réplicas.
- Imagen utilizada.
- Estrategia de actualización.
- Configuración de ejecución.

10.4.2 Modelo de Despliegue

La aplicación principal será desplegada mediante Deployments independientes para soportar Blue-Green Deployment.

```
techwave-blue
```

```
techwave-green
```

10.4.3 Beneficios

Gestión Declarativa La aplicación se describe mediante YAML.

Recuperación Automática Los pods fallidos son recreados automáticamente.

Escalabilidad Las réplicas pueden incrementarse dinámicamente.

Actualizaciones Controladas Permite despliegues seguros y reversibles.

10.5 Services

10.5.1 Objetivo

Los Services proporcionan un punto de acceso estable para los pods.

Dado que los pods son recursos efímeros, los Services actúan como capa de abstracción que garantiza conectividad continua.

10.5.2 Funciones

Descubrimiento de Servicios Permite localizar aplicaciones dentro del clúster.

Balanceo de Carga Distribuye peticiones entre múltiples pods.

Abstracción Oculta cambios internos de infraestructura.

10.5.3 Diseño Propuesto

techwave-service, Responsabilidades:

- Acceso interno.
- Balanceo de tráfico.
- Conmutación Blue-Green.

10.6 Ingress

10.6.1 Objetivo

El Ingress Controller será responsable de gestionar el tráfico HTTP/HTTPS procedente de Internet.

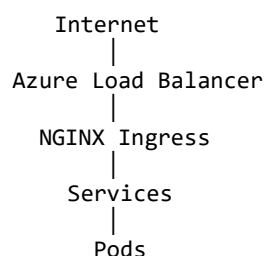
10.6.2 Componentes

Azure Load Balancer Primer punto de entrada.

NGINX Ingress Controller Encargado de:

- Routing.
- Terminación TLS.
- Gestión de dominios.
- Exposición de servicios.

10.6.3 Arquitectura



10.6.4 Dominios

Ejemplo:

app.techwave.com

grafana.techwave.com

prometheus.techwave.com

10.6.5 Beneficios

Centralización Un único punto de entrada.

Seguridad Gestión unificada de certificados.

Escalabilidad Permite añadir nuevos servicios fácilmente.

10.7 ConfigMaps

10.7.1 Objetivo

Los ConfigMaps almacenan información de configuración no sensible. Su uso permite separar configuración y código.

10.7.2 Información Gestionada

Ejemplos:

```
APP_NAME
APP_VERSION
LOG_LEVEL
ENVIRONMENT
PORT
```

10.7.3 Beneficios

Flexibilidad Cambios sin recompilar imágenes.

Reutilización Configuraciones comunes para múltiples servicios.

Gestión Multi-Entorno Desarrollo, preproducción y producción.

10.8 Secrets

10.8.1 Objetivo

Gestionar información sensible de forma segura.

10.8.2 Información Almacenada

Ejemplos:

```
API_KEYS
DATABASE_PASSWORDS
TOKENS
CERTIFICADOS
```

10.8.3 Principios de Seguridad

Separación del Código Nunca almacenar credenciales en repositorios Git.

Acceso Controlado Sólo las aplicaciones autorizadas podrán consumir secretos.

Auditoría Gestión trazable de modificaciones.

10.8.4 Evolución Futura

En futuras fases podrían integrarse soluciones como:

- Azure Key Vault
- HashiCorp Vault

10.9 Horizontal Pod Autoscaler (HPA)

10.10 Objetivo

Automatizar el escalado horizontal de la aplicación.

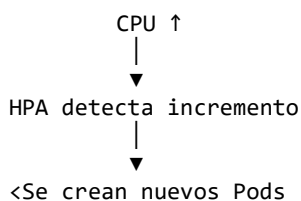
10.10.1 Motivación

La carga de trabajo puede variar significativamente.

Un número fijo de pods genera:

- Costes innecesarios.
- Riesgos de saturación.
- Infrautilización.

10.10.2 Funcionamiento



10.10.3 Métricas de Escalado

CPU Consumo medio por pod.

Memoria Uso de memoria.

Métricas Personalizadas Posibles ampliaciones futuras. Ejemplos:

- Número de peticiones.

- Latencia.
- Usuarios concurrentes.

10.10.4 Diseño Inicial

- Mínimo: 2 Pods
- Máximo: 10 Pods

10.10.5 Beneficios

Elasticidad Adaptación dinámica.

Rendimiento Respuesta rápida a incrementos de carga.

Optimización de costes Uso eficiente de recursos.

10.11 Afinidad y Anti-Afinidad

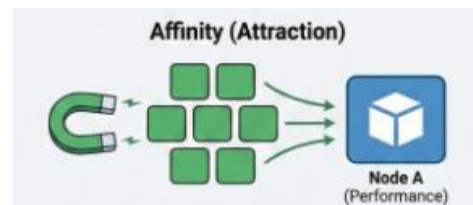
10.11.1 Objetivo

Controlar dónde se ejecutan los pods dentro del clúster.

10.11.2 Afinidad

Permite agrupar cargas de trabajo relacionadas. Ejemplo:

Aplicación
+
Servicio auxiliar en nodos cercanos.



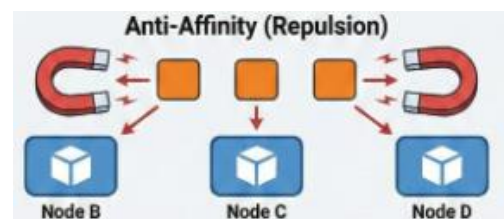
10.11.3 Anti-Afinidad

Permite separar réplicas críticas.

Ejemplo:

Pod 1 → Nodo A

Pod 2 → Nodo B



10.11.4 Justificación

La estrategia recomendada para producción será:

- Anti-Afinidad obligatoria
- entre réplicas de la aplicación.

Beneficios:

- Alta disponibilidad.
- Tolerancia a fallos.

- Mayor resiliencia.

10.12 Tolerations y Taints

10.12.1 Objetivo

Controlar qué cargas de trabajo pueden ejecutarse en determinados nodos.

10.12.2 Casos de Uso

Nodos Reservados

Monitoring Nodes

System Nodes

Business Nodes

Workloads Críticas

Permitir ejecución exclusivamente en nodos específicos.

10.12.3 Beneficios

Aislamiento Separación de cargas.

Seguridad Mayor control operacional.

Rendimiento Evita competencia innecesaria por recursos.

10.13 Liveness Probes

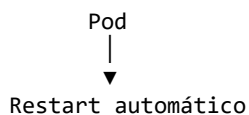
10.13.1 Objetivo

Determinar si una aplicación sigue funcionando correctamente.

10.13.2 Funcionamiento

Kubernetes realiza comprobaciones periódicas.

Si la validación falla:



10.13.3 Endpoint Utilizado

/health

10.13.4 Beneficios

Autorreparación Recuperación automática.

Disponibilidad Menor tiempo de interrupción.

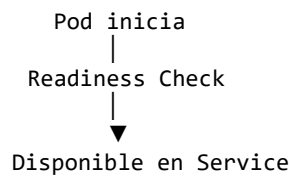
Automatización Sin intervención humana.

10.14 Readiness Probes

10.14.1 Objetivo

Determinar cuándo la aplicación está preparada para recibir tráfico.

10.14.2 Funcionamiento



10.14.3 Endpoint Utilizado

/health

10.14.4 Beneficios

Despliegues Seguros Evita enviar tráfico a pods no preparados.

Blue-Green Deployment Facilita validaciones previas a la activación.

Reducción de errores Mejora la experiencia del usuario.

10.15 Estrategia de Alta Disponibilidad

La plataforma combinará varios mecanismos de resiliencia.

Réplicas Múltiples ≥ 2 Pods

Anti-Afinidad Distribución entre nodos.

HPA Escalado automático.

Liveness Recuperación automática.

Readiness Control de entrada de tráfico.

Blue-Green Deployment Actualizaciones sin interrupción.

10.16 Gobierno y Buenas Prácticas Kubernetes

Estándares de Naming

techwave-app
techwave-service

techwave-config

Etiquetas (Labels) Todos los recursos deberán incluir:

app
environment
version
owner

Recursos Declarativos Toda configuración gestionada mediante Git.

Infraestructura Versionada Sin cambios manuales en producción.

Seguridad

- Mínimo privilegio.
- Secrets externos al código.
- Usuarios no root.
- Imágenes escaneadas.

10.17 Indicadores Operativos

Indicador	Objetivo
Disponibilidad	> 99,5 %
Pods mínimos	2
Tiempo de recuperación	< 15 min
Escalado automático	Sí
Cobertura de probes	100 %
Gestión declarativa	100 %
Recursos monitorizados	100 %

11 Infraestructura como Código

La automatización de la infraestructura constituye uno de los pilares fundamentales de la filosofía DevOps. Con el objetivo de extender los principios de automatización ya aplicados al desarrollo, construcción y despliegue de la aplicación, se ha incorporado al proyecto una capa de **Infraestructura como Código (Infrastructure as Code – IaC)** basada en **Terraform**.

La solución permite definir la infraestructura mediante ficheros declarativos almacenados en el mismo ecosistema de versionado utilizado para el resto del proyecto. De esta forma, la infraestructura pasa a ser tratada como un activo software, facilitando su mantenimiento, evolución y reutilización.

La estructura desarrollada establece una base sólida para futuros despliegues sobre Amazon Web Services (AWS), incluyendo capacidades para redes, Kubernetes, registros de contenedores, observabilidad, identidades, seguridad y servicios DNS.

11.1 Introducción

Tradicionalmente, la gestión de infraestructura se realiza mediante configuraciones manuales que generan diferencias entre entornos, dificultan la trazabilidad y aumentan el riesgo operativo.

Para mitigar estas limitaciones, la arquitectura incorpora el paradigma de Infraestructura como Código, mediante el cual la infraestructura se define utilizando código declarativo almacenado en repositorios Git.

Este enfoque permite aplicar a la infraestructura los mismos principios utilizados en el desarrollo de software:

- Versionado.
- Automatización.
- Revisión de cambios.
- Trazabilidad.
- Reproducibilidad.
- Mejora continua.

La tecnología seleccionada para este propósito es **Terraform**, debido a su amplia adopción en la industria, su naturaleza multiplataforma y su integración con proveedores cloud y plataformas Kubernetes.

11.2 Objetivos de la Infraestructura como Código

La adopción de Terraform persigue tanto objetivos técnicos como organizativos.

OI-01 Estandarización

Garantizar que todos los entornos se despliegan utilizando configuraciones consistentes y controladas.

Beneficios

- Eliminación de diferencias entre entornos.
- Reducción de incidencias operativas.
- Mayor predictibilidad.

OI-02 Automatización

Reducir la intervención manual en la provisión y mantenimiento de infraestructura.

Beneficios

- Menor riesgo de error humano.
- Mayor velocidad de despliegue.
- Reducción del esfuerzo operativo.

OI-03 Trazabilidad

Mantener un historial completo de modificaciones sobre la infraestructura.

Beneficios

- Auditoría.
- Gestión del cambio.
- Cumplimiento organizativo.

OI-04 Reproducibilidad

Permitir la reconstrucción completa de cualquier entorno a partir del repositorio de código.

Beneficios

- Recuperación ante incidencias.
- Consistencia entre entornos.
- Menor dependencia del conocimiento individual.

OI-05 Escalabilidad

Facilitar la evolución futura de la plataforma.

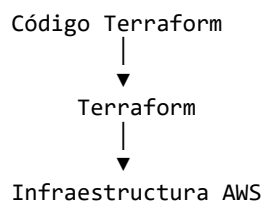
Beneficios

- Modularidad.
- Adaptabilidad tecnológica.
- Crecimiento controlado.

11.3 Terraform como Plataforma IaC

Terraform es una herramienta declarativa de Infraestructura como Código que permite definir recursos mediante ficheros de configuración.

El modelo de funcionamiento se basa en especificar el estado deseado de la infraestructura, permitiendo a Terraform calcular automáticamente las acciones necesarias para alcanzarlo.



Principios de Funcionamiento

- **Declaratividad** El administrador define el estado final deseado de la infraestructura.
- **Idempotencia** La ejecución repetida de una misma configuración genera siempre el mismo resultado.
- **Gestión de Dependencias** Terraform detecta automáticamente las relaciones entre los recursos.
- **Estado** Terraform mantiene un fichero de estado que representa la fuente de información utilizada para gestionar la infraestructura.

11.4 Alcance de Terraform en TechWave DevOps

La implementación realizada incorpora una arquitectura modular preparada para desplegar la plataforma sobre AWS.

La infraestructura diseñada contempla los siguientes dominios funcionales.

Infraestructura de Red

- VPC.
- Subredes públicas.
- Subredes privadas.
- Internet Gateway.

- NAT Gateway.
- Route Tables.

Plataforma Kubernetes

- Amazon Elastic Kubernetes Service (EKS).
- Node Groups.
- Configuración del clúster.
- Integración con servicios AWS.

Registro de Contenedores

- Amazon Elastic Container Registry (ECR).
- Gestión de versiones.
- Escaneo de imágenes.

Seguridad

- IAM.
- AWS KMS.
- AWS Secrets Manager.

Observabilidad

- CloudWatch.
- Amazon Managed Prometheus.
- Amazon Managed Grafana.

DNS

- Amazon Route53.
- Hosted Zones.
- Registros DNS.

11.5 Gestión del Estado

Terraform utiliza un archivo denominado:

```
terraform.tfstate
```

Este fichero representa la fuente de verdad de la infraestructura gestionada.

Riesgos del estado local

- Pérdida de información.
- Trabajo simultáneo conflictivo.
- Dificultad de auditoría.
- Inconsistencias operativas.

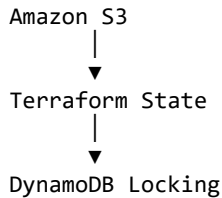
Estrategia implementada

Durante el desarrollo se utiliza backend local:

```
backend "local" {}
```

Evolución prevista

Para entornos empresariales se recomienda utilizar almacenamiento remoto:



Beneficios

- Trabajo colaborativo.
- Mayor disponibilidad.
- Protección frente a pérdida de información.
- Trazabilidad de cambios.

11.6 Estrategia de Modularización

La solución de Infraestructura como Código ha sido diseñada siguiendo una arquitectura modular orientada a facilitar la reutilización, el mantenimiento y la evolución futura de la plataforma.

Cada módulo representa un dominio funcional independiente y encapsula los recursos necesarios para implementar una capacidad concreta de la infraestructura. Este enfoque reduce el acoplamiento entre componentes y favorece la reutilización de código entre distintos entornos.

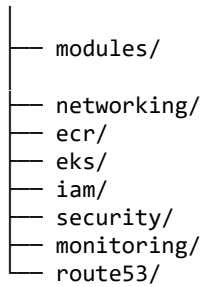
Objetivos de la Modularización

La organización modular de Terraform persigue los siguientes objetivos:

- Reutilización de componentes.
- Simplificación del mantenimiento.
- Escalabilidad de la infraestructura.
- Separación de responsabilidades.
- Reducción de duplicidades.
- Facilitar la evolución futura de la plataforma.

Estructura General

```
terraform/
```



11.6.1 Módulo Networking

El módulo **networking** constituye la base de toda la infraestructura y es responsable de definir la conectividad entre los distintos componentes desplegados en AWS.

Este módulo implementa una arquitectura de red segmentada siguiendo buenas prácticas de seguridad y disponibilidad.

- **Responsabilidades**
 - Creación de la VPC principal.
 - Definición de subredes públicas.
 - Definición de subredes privadas.
 - Configuración de Internet Gateway.
 - Configuración de NAT Gateway.
 - Gestión de tablas de rutas.
 - Configuración de grupos de seguridad.
- **Beneficios**
 - Aislamiento lógico de recursos.
 - Reducción de superficie de ataque.
 - Alta disponibilidad mediante múltiples zonas.
 - Escalabilidad futura.

11.6.2 Módulo ECR

El módulo **ecr** es responsable de la gestión del repositorio de imágenes Docker utilizado por la plataforma.

Su función principal consiste en almacenar y distribuir las imágenes generadas por los pipelines CI/CD para que posteriormente puedan ser consumidas por Kubernetes.

Responsabilidades

- Creación de repositorios ECR.
- Gestión de versiones de imágenes.

- Escaneo automático de vulnerabilidades.
- Aplicación de políticas de ciclo de vida.

Beneficios

- Centralización de artefactos.
- Seguridad mejorada.
- Integración con GitHub Actions.
- Trazabilidad de versiones.

11.6.3 Módulo EKS

El módulo **eks** gestiona la plataforma Kubernetes sobre AWS.

Representa el componente central de la arquitectura Cloud Native y proporciona la capacidad de ejecutar aplicaciones contenerizadas de forma escalable y resiliente.

Responsabilidades

- Creación del clúster Kubernetes.
- Gestión de Node Groups.
- Configuración de red del clúster.
- Parametrización de la versión de Kubernetes.
- Integración con servicios AWS.

Beneficios

- Alta disponibilidad.
- Escalado automático.
- Recuperación ante fallos.
- Gestión simplificada de contenedores.

11.6.4 Módulo IAM

El módulo **iam** centraliza la gestión de identidades y permisos utilizados por la infraestructura.

Permite implementar un modelo de seguridad basado en el principio de mínimo privilegio.

Responsabilidades

- Gestión de roles IAM.
- Definición de políticas.

- Permisos para Kubernetes.
- Integración con servicios AWS.

Beneficios

- Mayor seguridad.
- Trazabilidad de accesos.
- Reducción de privilegios excesivos.
- Gobierno centralizado.

11.6.5 Módulo Security

El módulo **security** incorpora los mecanismos de protección de información sensible y cifrado.

Responsabilidades

- Gestión de claves KMS.
- Gestión de secretos mediante Secrets Manager.
- Protección de credenciales.
- Configuración de cifrado.

Beneficios

- Protección de información crítica.
- Cumplimiento de buenas prácticas de seguridad.
- Gestión centralizada de secretos.
- Reducción de riesgos operativos.

11.6.6 Módulo Monitoring

El módulo **monitoring** está destinado a proporcionar capacidades avanzadas de monitorización y observabilidad sobre la plataforma.

Aunque actualmente la aplicación ya expone métricas Prometheus mediante el endpoint /metrics, este módulo prepara la integración futura con servicios gestionados de observabilidad.

Responsabilidades

- CloudWatch.
- Amazon Managed Prometheus.

- Amazon Managed Grafana.
- Gestión de logs y métricas.

Beneficios

- Supervisión centralizada.
- Identificación temprana de incidencias.
- Visualización operativa.
- Base para futuros cuadros de mando.

11.6.7 Módulo Route53

El módulo **route53** centraliza la gestión DNS de la plataforma.

Permite desacoplar la infraestructura de los nombres públicos utilizados por usuarios y servicios.

Responsabilidades

- Gestión de zonas DNS.
- Creación de registros públicos.
- Resolución de nombres.
- Publicación de servicios.

Ejemplos de registros previstos

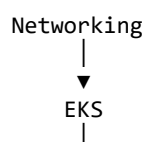
- app.techwave.com
- grafana.techwave.com
- prometheus.techwave.com

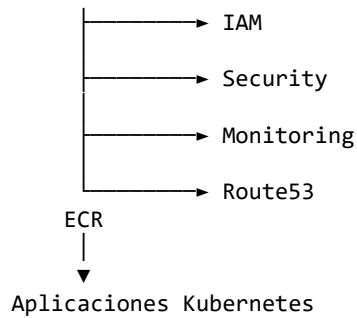
Beneficios

- Gestión centralizada de DNS.
- Mayor flexibilidad operativa.
- Escalabilidad futura.
- Facilita estrategias multi-entorno y multi-región.

Relación entre Módulos

La arquitectura modular sigue un modelo jerárquico en el que determinados módulos dependen de otros para su funcionamiento.

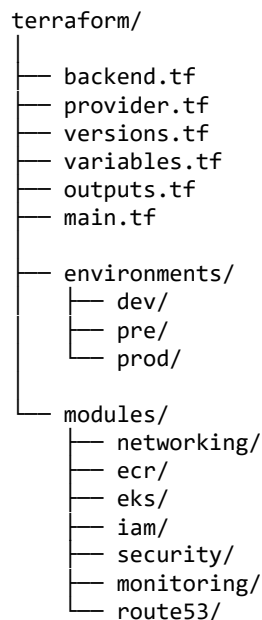




Esta organización facilita la evolución progresiva de la infraestructura, permitiendo incorporar nuevos módulos o modificar los existentes sin afectar significativamente al resto de componentes.

11.7 Estructura del Repositorio Terraform

La solución se organiza siguiendo una estructura orientada a múltiples entornos.



Beneficios

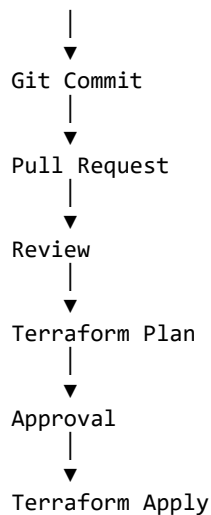
- **Separación de responsabilidades** Cada módulo mantiene una finalidad concreta.
- **Escalabilidad** Permite incorporar nuevos componentes sin afectar al resto.
- **Reutilización** Facilita la reutilización entre distintos entornos.

11.8 Gestión de Cambios

Toda modificación de infraestructura debe seguir un proceso controlado.

Flujo de Cambio

Developer



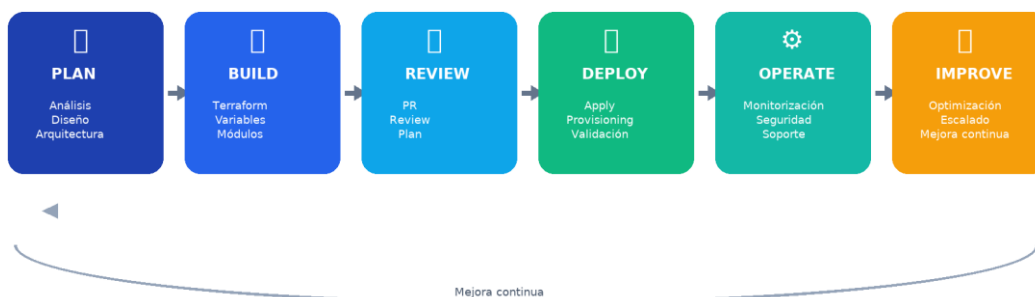
Principios

- **Ningún cambio manual** La infraestructura debe gestionarse mediante código.
- **Todo cambio versionado** Las modificaciones deben registrarse en Git.
- **Revisión obligatoria** Todo cambio debe validarse antes de su aplicación.
- **Automatización** La ejecución debe integrarse en procesos CI/CD.

11.9 Gestión del Ciclo de Vida

La infraestructura definida mediante Terraform sigue un ciclo de vida similar al utilizado para el desarrollo de aplicaciones, garantizando que cada cambio pase por procesos de diseño, validación, despliegue y mejora continua. Este enfoque permite aplicar a la infraestructura los mismos principios de calidad, trazabilidad y control utilizados en el desarrollo de software moderno.

Ciclo de Vida de la Infraestructura como Código



11.9.1 Fase de Diseño

La fase de diseño constituye el punto de partida de cualquier modificación de infraestructura. Durante esta etapa se analizan las necesidades técnicas y de negocio, se identifican los recursos necesarios y se define la arquitectura que dará respuesta a los requisitos planteados.

Las actividades principales incluyen la elaboración de diagramas arquitectónicos, el modelado de componentes, la definición de dependencias y la identificación de requisitos de seguridad, disponibilidad y escalabilidad. El objetivo es garantizar que la infraestructura propuesta sea coherente con la arquitectura global de la plataforma y pueda evolucionar de forma sostenible.

11.9.2 Fase de Construcción

Una vez definida la arquitectura, se procede a la implementación de los recursos mediante Terraform. En esta fase se desarrollan los módulos, variables, outputs y configuraciones necesarios para representar la infraestructura como código.

La modularización permite encapsular funcionalidades específicas como red, Kubernetes, observabilidad o seguridad, favoreciendo la reutilización y simplificando el mantenimiento. Todo el código desarrollado se almacena en repositorios Git para asegurar trazabilidad y control de versiones.

11.9.3 Fase de Validación

Antes de realizar cualquier despliegue, la configuración debe ser validada para detectar errores sintácticos, inconsistencias o cambios no deseados.

Terraform proporciona diferentes mecanismos para realizar esta validación:

```
terraform fmt
terraform validate
terraform plan
```

Estas comprobaciones permiten verificar la calidad del código, asegurar que la configuración es válida y visualizar los cambios que se aplicarán sobre la infraestructura antes de ejecutarlos realmente.

11.9.4 Fase de Despliegue

Tras superar las validaciones correspondientes, la infraestructura puede desplegarse de forma controlada mediante Terraform.

```
terraform apply
```

Durante esta fase Terraform compara el estado actual de la plataforma con el estado deseado definido en el código y ejecuta únicamente las acciones necesarias para alcanzar dicho objetivo. Este modelo reduce significativamente

los riesgos asociados a modificaciones manuales y garantiza una ejecución consistente y repetible.

11.9.5 Fase Operativa

Una vez desplegada, la infraestructura entra en fase de operación. En esta etapa se supervisa continuamente el comportamiento de los recursos para garantizar que cumplen los niveles de servicio establecidos.

Las tareas operativas incluyen la monitorización de la disponibilidad, el consumo de recursos, el rendimiento de los servicios, la seguridad de los componentes desplegados y la detección temprana de incidencias. Esta supervisión permite identificar oportunidades de mejora y reaccionar rápidamente ante posibles problemas.

11.9.6 Fase de Mejora Continua

La infraestructura no debe considerarse un elemento estático. Conforme evolucionan las necesidades del negocio o aparecen nuevas tecnologías, la plataforma debe adaptarse mediante procesos controlados de mejora.

Esta fase contempla actividades como la optimización de costes, la incorporación de nuevas capacidades, la actualización de versiones, la mejora de la seguridad o la automatización de procesos adicionales. Todos estos cambios vuelven a iniciar nuevamente el ciclo de vida definido, garantizando una evolución controlada y trazable.

11.9.7 Fase de Retirada

Cuando determinados recursos dejan de ser necesarios, Terraform permite proceder a su eliminación de forma controlada.

```
terraform destroy
```

La retirada planificada de recursos evita costes innecesarios, reduce complejidad operativa y garantiza que la infraestructura permanezca alineada con las necesidades reales de la organización. En entornos empresariales esta fase suele complementarse con procedimientos de desmantelamiento, archivado y conservación de evidencias para asegurar el cumplimiento de las políticas corporativas y regulatorias.

11.10 Gobierno de Infraestructura

La gestión de infraestructura debe alinearse con los principios DevOps definidos para la organización.

Principios Operativos

- **Infraestructura Declarativa** Toda la infraestructura debe definirse mediante código.
- **Infraestructura Versionada** Todos los cambios deben almacenarse en Git.
- **Infraestructura Auditada** Las modificaciones deben ser trazables.
- **Infraestructura Reproducible** Debe ser posible reconstruir un entorno completo.
- **Infraestructura Automatizada** Se debe minimizar la intervención manual.

11.11 Riesgos y Medidas de Mitigación

Riesgo	Mitigación
Pérdida del estado	Backend remoto
Cambios no autorizados	Pull Requests
Errores de configuración	Terraform Plan
Drift de infraestructura	Revisiones periódicas
Dependencia de personas	Documentación y Git

11.12 Indicadores de Éxito

Indicador	Objetivo
Infraestructura gestionada mediante Terraform	100 %
Recursos versionados	100 %
Cambios revisados	100 %
Entornos reproducibles	100 %
Automatización de despliegues	> 90 %
Cambios manuales en producción	0

La incorporación de Terraform al repositorio permite extender los principios DevOps más allá del desarrollo y despliegue de aplicaciones, proporcionando una base real de Infraestructura como Código preparada para futuras

implementaciones cloud sobre AWS. La arquitectura modular desarrollada favorece la reutilización, la escalabilidad y la automatización, consolidando un modelo operativo alineado con las mejores prácticas actuales de ingeniería de plataformas y gestión de infraestructura.

12 Estrategia CI/CD

La estrategia CI/CD definida permite transformar el ciclo de vida de la aplicación en un proceso completamente automatizado, trazable y seguro. La combinación de GitHub, GitHub Actions, GitHub Container Registry y Kubernetes proporciona una cadena de entrega moderna capaz de soportar despliegues frecuentes con un riesgo mínimo. La integración de controles de calidad, escaneo de vulnerabilidades, gestión segura de secretos y despliegues Blue-Green garantiza que la velocidad de entrega no comprometa la estabilidad ni la seguridad de la plataforma, constituyendo uno de los pilares fundamentales de la transformación DevOps de TechWave Solutions.

12.1 Introducción

La capacidad de entregar software de forma rápida, segura y repetible constituye uno de los principales objetivos de la transformación DevOps planteada para TechWave Solutions.

Para alcanzar este objetivo se implementará una estrategia de **CI/CD (Continuous Integration / Continuous Delivery / Continuous Deployment)** que permita automatizar el ciclo completo de vida de la aplicación, desde la integración de cambios en el repositorio hasta su despliegue en Kubernetes.

La automatización del ciclo de entrega permitirá:

- Reducir el Time To Market.
- Disminuir errores humanos.
- Incrementar la calidad del software.
- Mejorar la trazabilidad.
- Estandarizar despliegues.
- Facilitar operaciones de rollback.
- Incrementar la confianza en las releases.

12.2 Objetivos de la Estrategia CI/CD

OCI-01 Automatizar el ciclo de entrega Eliminar tareas manuales repetitivas relacionadas con compilación, pruebas y despliegues.

OCI-02 Incrementar la calidad Detectar errores en fases tempranas mediante validaciones automáticas.

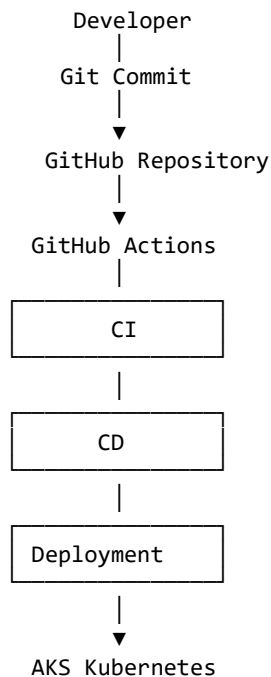
OCI-03 Mejorar la trazabilidad Mantener evidencia completa del proceso de construcción y despliegue.

OCI-04 Reducir riesgos Identificar fallos antes de su promoción a producción.

OCI-05 Facilitar despliegues frecuentes Permitir múltiples despliegues semanales sin incrementar el riesgo operativo.

12.3 Arquitectura del Pipeline

La solución adoptada se basa en GitHub como plataforma central de desarrollo y GitHub Actions como motor de automatización.



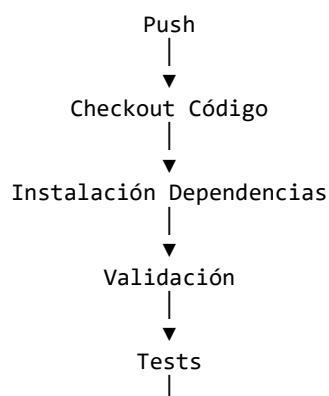
12.4 Continuous Integration (CI)

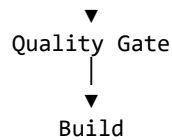
12.4.1 Objetivo

Garantizar que cualquier cambio incorporado al repositorio sea validado automáticamente antes de formar parte de una versión desplegable.

La integración continua constituye la primera línea de defensa frente a errores funcionales y problemas de calidad.

12.4.2 Flujo de Integración Continua





12.4.3 Fase de Validación

Durante esta etapa se verificará:

Integridad del código Comprobación de estructura y dependencias.

Cumplimiento de estándares Validación de convenciones de desarrollo.

Formato y consistencia Verificación de calidad básica del proyecto.

12.4.4 Fase de Test

Objetivo Detectar errores antes de que alcancen entornos superiores.

Pruebas Unitarias Validación individual de componentes.

Ejemplos:

- Endpoints.
- Servicios.
- Funciones auxiliares.

Pruebas de Integración Validación de interacción entre componentes.

Pruebas de Construcción Verificación de generación correcta del artefacto.

Resultados Esperados 100 % de ejecución automática

Todo fallo deberá detener el pipeline.

12.4.5 Quality Gate

Objetivo

Garantizar unos niveles mínimos de calidad antes de continuar el proceso.

Controles Propuestos

Calidad de código Validación estructural.

Cobertura de pruebas Porcentaje mínimo establecido por el proyecto.

Vulnerabilidades Escaneo automático de dependencias.

Construcción exitosa Generación correcta del artefacto.

Política

Si cualquiera de estos controles falla:

```
Pipeline = FAILED
```

No podrá iniciarse el proceso de entrega.

12.4.6 Build

Objetivo

Generar un artefacto reproducible y desplegable.

Actividades

Compilación Construcción de la aplicación.

Generación de metadatos

Inclusión de:

- Versión.
- Fecha.
- Commit.
- Identificador de pipeline.

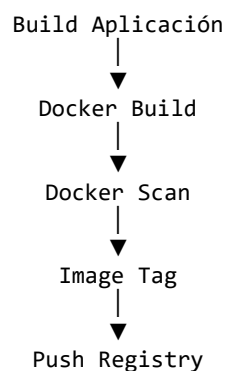
Empaquetado Preparación para contenerización.

12.5 Continuous Delivery (CD)

12.5.1 Objetivo

Automatizar la construcción y distribución de imágenes Docker listas para ser desplegadas.

Flujo de Entrega Continua



12.5.2 Construcción de Imagen Docker

Una vez superadas las validaciones:

```
docker build
```

La imagen incluirá:

- Aplicación.
- Dependencias.
- Configuración runtime.

12.5.3 Etiquetado de Imágenes

Se utilizará versionado semántico.

Ejemplos:

```
techwave-app:1.0.0  
techwave-app:1.1.0  
techwave-app:latest
```

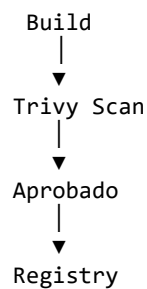
Además se mantendrá trazabilidad mediante SHA del commit.

Ejemplo:

```
techwave-app:a73f92d
```

12.5.4 Escaneo de Seguridad

Antes de publicar una imagen:



Política de Seguridad

No se permitirá desplegar:

- Vulnerabilidades críticas
- Las vulnerabilidades altas requerirán análisis previo.

12.5.5 Publicación en Registro

Las imágenes serán almacenadas en:

GitHub Container Registry (GHCR)

Beneficios:

- Integración nativa.
- Control de acceso.
- Versionado.
- Auditoría.

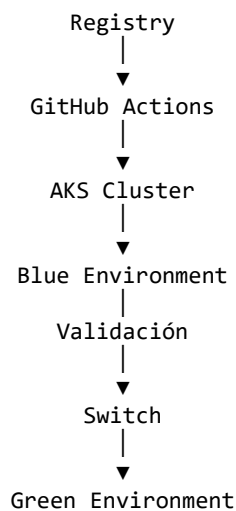
12.6 Continuous Deployment

12.6.1 Objetivo

Automatizar el despliegue completo sobre Kubernetes una vez superadas las fases previas.

La organización adoptará un enfoque de despliegue controlado basado en **Blue-Green Deployment**.

Flujo General



12.6.2 Actualización de Kubernetes

GitHub Actions ejecutará:

`kubect1 apply`
sobre:

- Deployments.
- Services.
- ConfigMaps.
- Secrets.
- Ingress.

12.6.3 Validación Post Despliegue

Antes de exponer tráfico:

Comprobación de Pods `kubect1 get pods`

Readiness Probe Verificación de disponibilidad.

Liveness Probe Validación de estado saludable.

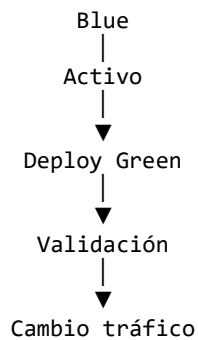
Endpoint Health `GET /health`

12.6.4 Blue-Green Deployment

La estrategia seleccionada consiste en mantener simultáneamente dos versiones.

- **Blue** Versión activa.
- **Green** Nueva versión candidata.

Proceso



Beneficios

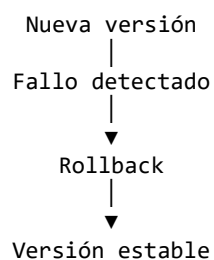
Riesgo reducido La versión anterior permanece disponible.

Recuperación inmediata Rollback rápido.

Cero interrupciones No existe impacto perceptible para los usuarios.

12.6.5 Rollback Automático

En caso de error:



Criterios de Activación

- Fallo de Health Check
- Pods no disponibles
- Errores críticos de aplicación
- Timeout de despliegue

Objetivo Operativo MTTR < 15 minutos

12.7 Gestión de Secretos

12.7.1 Objetivos

La gestión segura de credenciales constituye un requisito fundamental de la plataforma.

Los secretos nunca deberán almacenarse:

- En el código fuente.
- En Dockerfiles.
- En repositorios Git.
- En manifiestos públicos.

12.7.2 GitHub Secrets

GitHub Actions utilizará GitHub Secrets para almacenar información sensible.

Ejemplos:

```
AZURE_CREDENTIALS  
KUBECONFIG  
GHCR_TOKEN  
AKS_RESOURCE_GROUP
```

Beneficios

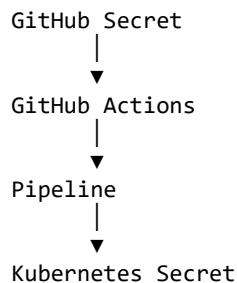
Cifrado Los valores permanecen protegidos.

Control de acceso Limitado a workflows autorizados.

Trazabilidad Integrado en GitHub.

Gestión centralizada Administración simplificada.

12.7.3 Flujo Seguro



12.7.4 Gestión de Credenciales Kubernetes

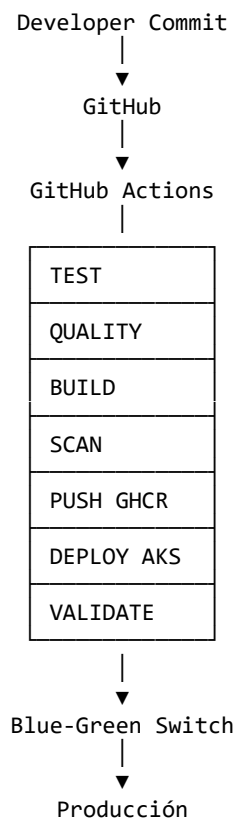
Las credenciales que deban consumir las aplicaciones se almacenarán mediante:

Kubernetes Secrets
Ejemplos:

- API Keys.
- Tokens.
- Contraseñas.
- Certificados.

12.7.5 Pipeline Objetivo

Flujo Completo



12.8 KPIs de CI/CD

Indicador	Objetivo
Automatización del despliegue	100 %
Tiempo medio de pipeline	< 10 minutos
Build exitosos	> 95 %
Cobertura de pruebas	> 80 %
Vulnerabilidades críticas	0

Indicador	Objetivo
Despliegues con rollback	< 5 %
Tiempo medio de recuperación	< 15 min
Trazabilidad de releases	100 %

13 Estrategia de Release Management

La estrategia de Release Management propuesta permite a TechWave Solutions combinar velocidad de entrega con control operativo. La adopción de Semantic Versioning, procesos formales de gestión de cambios, procedimientos estructurados de gestión de incidencias y mecanismos automáticos de rollback garantiza que la evolución continua de la plataforma pueda realizarse de forma segura, trazable y alineada con los objetivos de negocio. Esta disciplina se convierte en un elemento clave para mantener la estabilidad de la plataforma mientras se incrementa la frecuencia de despliegue característica de una organización DevOps madura.

13.1 Introducción

Uno de los errores más habituales en los procesos de transformación DevOps consiste en asumir que automatizar despliegues equivale a gestionar adecuadamente las versiones de un producto. Sin embargo, la automatización del despliegue y la gestión de releases son disciplinas complementarias pero diferentes.

Mientras que el pipeline CI/CD se ocupa de construir, validar y desplegar software, el **Release Management** tiene como misión controlar cómo evolucionan las versiones, cómo se aprueban los cambios, cómo se gestionan las incidencias y cómo se garantiza la continuidad del servicio ante situaciones imprevistas.

Para TechWave Solutions, la estrategia de Release Management debe proporcionar:

- Trazabilidad completa.
- Control del cambio.
- Reducción del riesgo operativo.
- Capacidad de recuperación rápida.
- Gobierno sobre las versiones desplegadas.
- Alineamiento entre negocio y tecnología.

13.2 Objetivos de la Gestión de Releases

ORM-01 Garantizar la estabilidad

Asegurar que cada nueva versión introducida en producción mantiene los niveles de calidad exigidos por la organización.

ORM-02 Reducir el riesgo operacional

Controlar el impacto que las nuevas versiones pueden generar sobre la plataforma.

ORM-03 Facilitar la trazabilidad

Permitir identificar en todo momento:

- Qué versión está desplegada.
- Cuándo se desplegó.
- Quién autorizó el cambio.
- Qué funcionalidades incorpora.

ORM-04 Simplificar la recuperación

Facilitar mecanismos de rollback rápidos y seguros.

ORM-05 Alinear negocio y tecnología

Garantizar que las releases responden a necesidades reales del negocio y siguen un proceso formal de aprobación.

13.3 Estrategia de Versionado

13.3.1 Principios

La estrategia de versionado debe ser:

- Simple.
- Comprensible.
- Escalable.
- Compatible con automatización.
- Integrable con CI/CD.

13.3.1 Semantic Versioning

TechWave Solutions adoptará el estándar internacional **Semantic Versioning (SemVer)**.

Formato:

MAJOR.MINOR.PATCH

Ejemplos:

1.0.0
1.1.0
1.2.3
2.0.0

13.3.3 Significado de las Versiones

MAJOR

Cambios incompatibles.

Ejemplos:

- Eliminación de funcionalidades.
- Modificaciones de API incompatibles.
- Rediseños importantes.

1.x.x → 2.x.x

MINOR

Introducción de nuevas funcionalidades manteniendo compatibilidad.

Ejemplos:

- Nuevos endpoints.
- Nuevas capacidades operativas.
- Mejoras funcionales.

1.1.0 → 1.2.0

PATCH

Corrección de errores.

Ejemplos:

- Bug fixes.
- Correcciones de rendimiento.
- Ajustes menores.

1.2.1 → 1.2.2

13.3.2 Convención de Releases

Tipo de cambio	Incremento
Corrección de error	PATCH
Nueva funcionalidad compatible	MINOR
Cambio incompatible	MAJOR

13.3.3 Relación con Docker

Cada release deberá asociarse a una imagen Docker específica.

Ejemplos:

techwave-app:1.0.0
techwave-app:1.1.0

techwave-app:1.1.1

13.3.4 Relación con Git

Cada release deberá estar asociada a:

Git Tag

Ejemplo:

v1.2.0

Esto permite reconstruir cualquier versión histórica.

13.4 Modelo de Gestión de Releases

13.4.1 Tipos de Release

Release Correctiva

Objetivo: Corregir errores detectados.

Ejemplo: 1.1.0 → 1.1.1

Release Evolutiva

Objetivo: Añadir nuevas funcionalidades.

Ejemplo: 1.1.0 → 1.2.0

Release Mayor

Objetivo: Incorporar cambios estructurales significativos.

Ejemplo: 1.5.0 → 2.0.0

Release de Emergencia

Objetivo: Resolver incidencias críticas.

Ejemplo: Hotfix

13.5 Gestión de Cambios

13.5.1 Objetivos

La gestión de cambios garantiza que toda modificación siga un procedimiento controlado.

13.5.2 Principios

Ningún cambio sin registro

Toda modificación deberá estar asociada a:

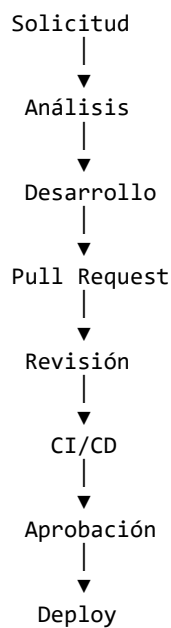
- Incidencia.
- Historia de usuario.
- Solicitud de cambio.

Cambios versionados El código y la infraestructura estarán sometidos a control de versiones.

Aprobación previa Determinados cambios requerirán validación formal.

Auditoría completa Todos los cambios deberán ser trazables.

13.5.3 Flujo de Gestión de Cambios



13.5.4 Clasificación de Cambios

Cambios Estándar

Cambios rutinarios y de bajo riesgo.

Ejemplos:

- Actualización de configuración.
- Ajustes menores.

Cambios Normales

Requieren análisis y aprobación.

Ejemplos:

- Nuevas funcionalidades.

- Actualización de componentes.

Cambios de Emergencia

Aplicados para resolver incidencias críticas.

Ejemplos:

- Vulnerabilidades críticas.
- Fallos de servicio.

13.6 Gestión de Incidencias

13.6.1 Objetivos

Garantizar una respuesta estructurada ante problemas detectados en producción.

13.6.2 Clasificación de Incidencias

Críticas

Impacto total sobre el servicio.

Ejemplos:

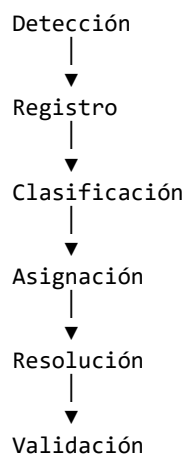
- Aplicación inaccesible.
- Fallo masivo.

Altas Impacto significativo sobre funcionalidades principales.

Medias Impacto limitado.

Bajas Afectación reducida.

13.6.3 Proceso de Gestión





13.6.4 Detección Automática

La plataforma de observabilidad permitirá detectar incidencias mediante:

- **Prometheus** Alertas de infraestructura.
- **Grafana** Dashboards operativos.
- **Loki** Errores de aplicación.
- **OpenTelemetry** Análisis distribuido.

13.7 Gestión de Rollback

13.7.1 Objetivos

Restaurar el servicio rápidamente cuando una nueva versión genera problemas.

13.7.2 Principios

Rapidez Recuperación inmediata.

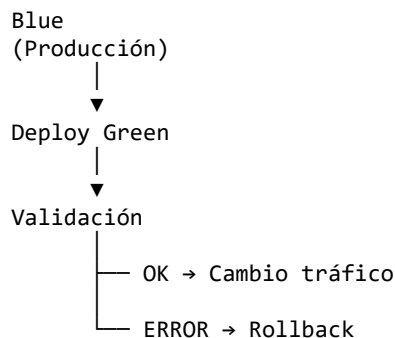
Automatización Intervención mínima.

Trazabilidad Registro completo de la reversión.

Seguridad Conservación de la última versión estable.

13.7.3 Estrategia de Rollback

La estrategia seleccionada se basa en el modelo Blue-Green Deployment.



13.7.4 Escenarios de Activación

Fallo de Readiness Probe La aplicación no está preparada para recibir tráfico.

Fallo de Liveness Probe La aplicación no opera correctamente.

Incremento anómalo de errores Detección mediante métricas y alertas.

Degradación de rendimiento Latencias superiores a los umbrales definidos.

Incidente funcional Validación negativa por parte de usuarios clave.

13.7.5 Procedimiento de Rollback

Paso 1 Detección del problema.

Paso 2 Bloqueo de promoción.

Paso 3 Redirección de tráfico hacia la última versión estable.

Paso 4 Verificación del restablecimiento del servicio.

Paso 5 Análisis de causa raíz.

Paso 6 Generación de acciones correctivas.

13.8 Gestión de Releases en Producción

Criterios de Entrada

Una release sólo podrá desplegarse si cumple:

- Tests superados.
- Quality Gate aprobado.
- Imagen validada.
- Escaneo de vulnerabilidades satisfactorio.
- Artefactos versionados.

Criterios de Salida

Una release se considerará aceptada cuando:

- Los pods estén operativos.
- Las probes sean satisfactorias.
- No existan errores críticos.
- La observabilidad confirme estabilidad.

13.9 Métricas de Release Management

Indicadores Operativos

KPI	Objetivo
Frecuencia de despliegue	> 1 semanal
Tiempo medio de despliegue	< 10 min
Éxito de releases	> 95 %
Releases con rollback	< 5 %
MTTR	< 15 min
Trazabilidad de releases	100 %
Releases automatizadas	> 95 %

Indicadores de Negocio

KPI	Objetivo
Time To Market	Reducción continua
Incidencias post-release	Descendente
Disponibilidad	> 99,5 %
Satisfacción de usuarios	Creciente

13.10 Comité de Cambios y Releases

Para releases de impacto medio o alto se establecerá un proceso de revisión coordinado.

Participantes

- Responsable de Desarrollo.
- Responsable de Operaciones.
- Responsable de Seguridad.
- Responsable de Proyecto.

Funciones

- Aprobar releases.
- Evaluar riesgos.

- Autorizar cambios críticos.
- Revisar incidencias relevantes.
- Analizar resultados post-despliegue.

13.11 Estrategia de Versionado

13.11.1 Marco de Referencia

TechWave Solutions adoptará el estándar **Semantic Versioning 2.0.0 (SemVer)** como mecanismo corporativo para la gestión de versiones de aplicaciones, imágenes de contenedor y releases.

La especificación oficial se encuentra publicada en: <https://semver.org/>

Semantic Versioning establece un conjunto de reglas para la asignación y evolución de números de versión con el objetivo de proporcionar un sistema predecible y comprensible para equipos de desarrollo, operaciones y consumidores de software.

13.11.2 Justificación de la Adopción de SemVer

La adopción de Semantic Versioning responde a las siguientes necesidades de negocio y operación:

Trazabilidad Permite identificar rápidamente la naturaleza de los cambios incorporados en cada release.

Predictibilidad Facilita conocer el impacto potencial de una actualización antes de desplegarla.

Compatibilidad Reduce riesgos asociados a cambios incompatibles entre versiones.

Automatización Permite integrar reglas automáticas dentro de los pipelines CI/CD para clasificación de releases.

Gobierno del Cambio Proporciona un lenguaje común entre Desarrollo, Operaciones y Seguridad.

13.11.3 Estructura de Versionado

Según la especificación SemVer, una versión adopta la forma:

MAJOR.MINOR.PATCH

Ejemplo:

1.0.0

Donde:

- **MAJOR**: cambios incompatibles con versiones anteriores.
- **MINOR**: incorporación de nuevas funcionalidades compatibles.
- **PATCH**: corrección de errores sin impacto funcional.

13.11.4 Política de Versionado de TechWave Solutions

Tipo de cambio	Ejemplo	Incremento
Corrección de bug	Error en endpoint	PATCH
Optimización sin cambios funcionales	Mejora de rendimiento	PATCH
Nueva funcionalidad	Nuevo endpoint API	MINOR
Nueva integración compatible	Integración externa	MINOR
Cambio incompatible	Modificación de API	MAJOR
Eliminación de funcionalidades	Fin de compatibilidad	MAJOR

13.11.5 Versionado de Aplicaciones

Ejemplos:

```
v1.0.0
Versión inicial estable
v1.1.0
Incorporación de nuevas funcionalidades
v1.1.1
Corrección de errores detectados en producción
v2.0.0
Versión con cambios incompatibles
```

13.11.6 Versionado de Imágenes Docker

Cada release generará una imagen única asociada a la versión de aplicación.

Ejemplo:

```
ghc.io/techwave/techwave-app:0.0
ghcr.io/techwave/techwavepp:1.1.0
ghcr.io/techwave/techwave-app:1.1.1
```

También se mantendrá una etiqueta de conveniencia:

```
Latest
```

aunque las implantaciones productivas utilizarán siempre versiones explícitas para evitar ambigüedades.

13.11.7 Versionado de Releases Kubernetes

Toda release desplegada en AKS deberá poder identificarse mediante:

- *Versión aplicación*
- *Versión imagen*
- *Commit Git*
- *Fecha despegue*
- *Pipeline ejecutado*

Ejemplo:

```
Release: 1.2.0
Image:**hcr.io/techwave/app:1.2.0
Commit:73f92d
Deployment: 2026-07-15
Pipine: #145
```

13.11.8 Pre-Releases

Siguiendo SemVer, podrán utilizarse versiones preliminares durante fases de validación.

Ejemplos:

```
1.2**-alpha**
1.2.0-beta.1
1.2.****.1
```

Donde:

- **alpha** → desarrollo interno.
- **beta** → pruebas funcionales.
- **rc (Release Candidate)** → candidata a producción.

13.11.9 Build Metadata

Para incrementar la trazabilidad, podrá añadirse metainformación de construcción siguiendo la nomenclatura definida por Semantic Versioning.

Ejemplo:

```
1.2.0+20260707
2.0+build.1451.2.0+commit.a73f92d
```

13.11.10 Política Corporativa de Releases

Toda release deberá cumplir las siguientes condiciones:

- Versionado SemVer obligatorio.
- Existencia de Git Tag.
- Imagen Docker asociada.
- Registro en el histórico de releases.
- Evidencia de validación CI/CD.
- Posibilidad de rollback a la versión anterior.

De esta forma, la organización garantiza que cada versión desplegada es completamente identificable, reproducible y recuperable, alineándose con las mejores prácticas DevOps y con la especificación oficial Semantic Versioning 2.0.0.

14 Blue-Green Deployment

Tras analizar las estrategias Rolling Update, Canary y Blue-Green Deployment, se concluye que **Blue-Green Deployment** representa la alternativa más adecuada para TechWave Solutions. Aunque implica un consumo temporal superior de recursos, proporciona ventajas decisivas en términos de disponibilidad, seguridad operativa, velocidad de recuperación y experiencia de usuario.

La combinación de Kubernetes, GitHub Actions y una estrategia Blue-Green permitirá realizar despliegues completamente automatizados, con impacto prácticamente nulo para el negocio y con capacidad de recuperación inmediata ante cualquier incidencia, convirtiéndose en un elemento fundamental de la estrategia DevOps de la organización.

14.1 Introducción

La capacidad de desplegar nuevas versiones de software sin interrumpir el servicio constituye uno de los principales objetivos de una organización DevOps moderna. Sin embargo, la estrategia de despliegue seleccionada tiene un impacto directo sobre la disponibilidad, el riesgo operacional, los tiempos de recuperación y la experiencia de usuario.

Para TechWave Solutions, la implantación de una estrategia de despliegue debe cumplir los siguientes requisitos:

- Minimizar el impacto sobre los usuarios.
- Reducir el riesgo asociado a cada release.
- Permitir validaciones antes de exponer cambios a producción.
- Facilitar recuperaciones rápidas.
- Integrarse completamente con Kubernetes y CI/CD.
- Garantizar la continuidad operativa.

Aunque Kubernetes soporta distintos mecanismos de actualización, no todas las estrategias ofrecen el mismo equilibrio entre riesgo, coste y simplicidad operativa. Por este motivo se ha realizado un análisis comparativo de las principales alternativas.

14.2 Estrategias de Despliegue Analizadas

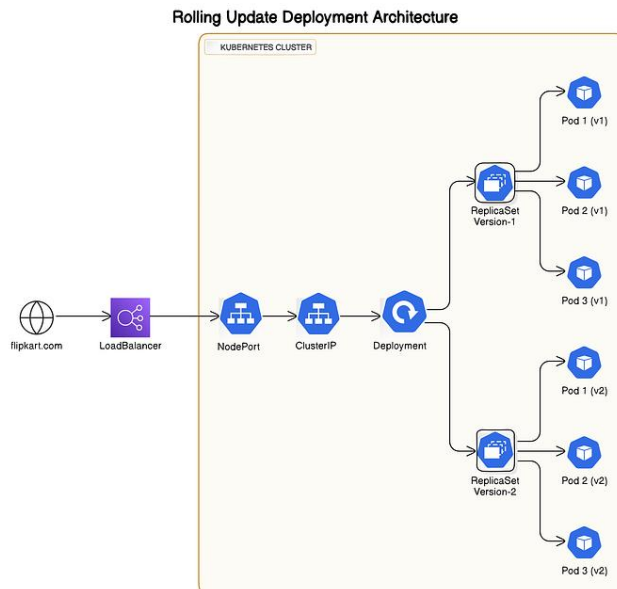
14.2.1 Rolling Update

Descripción

Rolling Update es la estrategia predeterminada de Kubernetes.

Consiste en reemplazar gradualmente las instancias de una aplicación por nuevas versiones sin detener completamente el servicio.

Proceso simplificado:



Durante el proceso conviven temporalmente versiones antiguas y nuevas.

Ventajas

- **Simplicidad** Es la estrategia nativa de Kubernetes.
- **Menor consumo de recursos** No requiere duplicar completamente el entorno.
- **Fácil automatización** Se integra de forma transparente con Deployments.
- **Adecuada para cambios menores** Especialmente útil en aplicaciones de reducido impacto.

Desventajas

- **Rollback más lento** La recuperación puede requerir una nueva actualización progresiva.
- **Coexistencia de versiones** Durante el despliegue existen múltiples versiones ejecutándose simultáneamente.
- **Riesgo funcional** Algunos cambios pueden producir comportamientos inconsistentes durante la transición.
- **Validación limitada** La nueva versión comienza a recibir tráfico antes de estar completamente validada.

Valoración

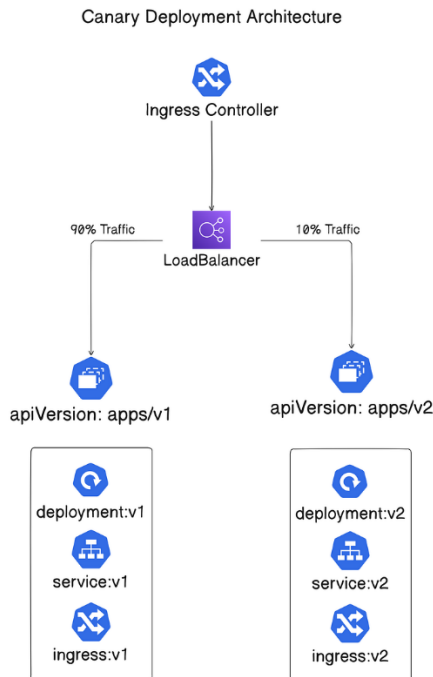
Rolling Update es eficiente en consumo de recursos, pero introduce un mayor riesgo operativo para aplicaciones críticas.

14.2.2 Canary Deployment

Descripción

Canary Deployment consiste en desplegar una nueva versión para un subconjunto reducido de usuarios antes de realizar una implantación completa.

Ejemplo:



Ventajas

- **Riesgo controlado** La exposición inicial es reducida.
- **Validación en producción** Permite probar la nueva versión con usuarios reales.
- **Detección temprana de errores** Identificación rápida de problemas.
- **Excelente para entornos de gran escala** Muy utilizada en organizaciones con millones de usuarios.

Desventajas

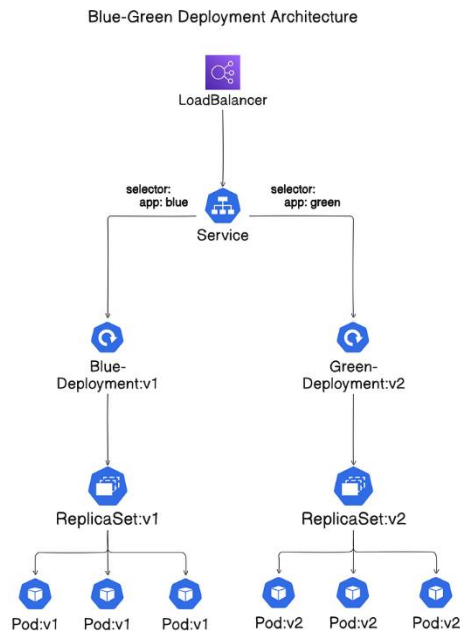
- **Mayor complejidad** Requiere mecanismos avanzados de gestión de tráfico.
- **Observabilidad exigente** Necesita métricas detalladas para evaluar el comportamiento.
- **Operación más compleja** Incrementa la carga administrativa.
- **Dificultad de análisis** Pueden coexistir varias versiones durante periodos prolongados.

Valoración

Es una estrategia muy potente, pero introduce un nivel de complejidad superior al necesario para el escenario actual de TechWave Solutions.

14.3 Blue-Green Deployment

Descripción



Ventajas

Riesgo mínimo La versión actual permanece completamente operativa.

- **Rollback inmediato** La recuperación consiste únicamente en revertir el tráfico.
- **Cero interrupciones** No existen indisponibilidad perceptible.
- **Validación previa** La nueva versión puede verificarse antes de recibir usuarios.
- **Mayor trazabilidad** La comparación entre versiones resulta sencilla.
- **Excelente integración con Kubernetes** Puede implementarse mediante Deployments y Services estándar.

Desventajas

- **Mayor consumo de recursos** Es necesario mantener dos versiones simultáneamente.
- **Coste superior** Durante el despliegue aumenta temporalmente el consumo de infraestructura.
- **Gestión adicional** Requiere mecanismos de coordinación en el pipeline.

Valoración

Representa el mejor equilibrio entre seguridad operativa, simplicidad y capacidad de recuperación.

14.4 Comparativa de Estrategias

Criterio	Rolling Update	Canary	Blue-Green
Complejidad	Baja	Alta	Media
Consumo de recursos	Bajo	Medio	Alto
Riesgo operativo	Medio	Bajo	Muy bajo
Velocidad de rollback	Media	Media	Muy alta
Validación previa	Limitada	Parcial	Completa
Disponibilidad	Alta	Alta	Muy alta
Facilidad de implantación	Alta	Baja	Alta
Adecuación para AKS	Alta	Media	Muy alta

14.5 Motivo de la Elección

Decisión Adoptada *Blue-Green Deployment*

La elección de Blue-Green Deployment se fundamenta en los objetivos corporativos definidos para TechWave Solutions.

Alineación con los Objetivos de Negocio

Reducir riesgo de despliegue La nueva versión se valida completamente antes de recibir tráfico.

Incrementar disponibilidad La versión anterior permanece activa durante todo el proceso.

Mejorar la calidad Los errores pueden detectarse antes de afectar a los usuarios.

Reducir impacto operacional La recuperación es extremadamente rápida.

Facilitar automatización La estrategia se integra de forma natural con GitHub Actions y Kubernetes.

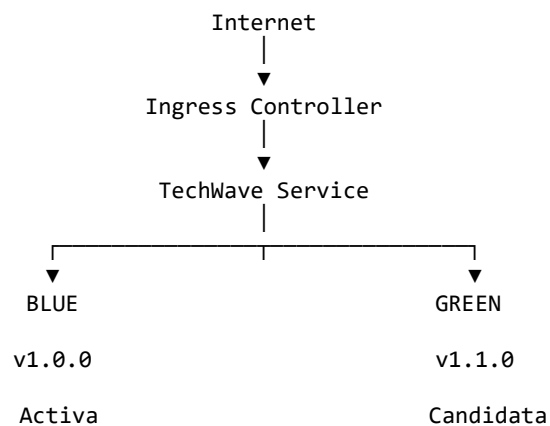
Justificación Técnica

Desde una perspectiva técnica, Blue-Green ofrece:

- Menor complejidad que Canary.
- Mayor seguridad que Rolling Update.
- Excelente compatibilidad con AKS.
- Rollback prácticamente instantáneo.
- Integración sencilla con Services e Ingress.

Por estos motivos constituye la mejor opción para una organización que busca combinar modernización tecnológica con estabilidad operativa.

14.6 Arquitectura Blue-Green Propuesta



Fase 1

Producción servida por Blue.

Service → Blue

Fase 2

Despliegue de Green.

Blue → Producción
Green → Validación

Fase 3

Pruebas automáticas.

Health Check
Readiness
Liveness
Smoke Test

Fase 4

Conmutación.
Service
Blue → Green

Fase 5

Monitorización reforzada.

Validación de:

- Errores.
- Latencia.
- Consumo de recursos.
- Comportamiento funcional.

14.7 Impacto en Negocio

Continuidad del Servicio La organización puede desplegar nuevas versiones sin interrumpir la actividad.

Menor Riesgo Reputacional Los errores pueden detectarse antes de afectar a la totalidad de los usuarios.

Mayor Confianza Operativa Los equipos pueden liberar nuevas funcionalidades con menor incertidumbre.

Mayor Velocidad de Innovación La disponibilidad de un mecanismo de recuperación rápida favorece ciclos de entrega más cortos.

Reducción del Coste de Incidencias Los tiempos de recuperación disminuyen significativamente.

14.8 Coste Operativo

Costes Directos

La principal desventaja de Blue-Green es el incremento temporal de recursos.

Ejemplo:

- Producción Actual 2 Pods
- Nueva Versión 2 Pods
- Total temporal: 4 Pods

Costes Indirectos

- Mayor consumo de CPU.
- Mayor consumo de memoria.
- Incremento temporal de costes cloud.

Justificación Económica

Aunque existe un aumento temporal de recursos, este coste es significativamente inferior al impacto potencial de:

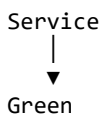
- Caídas de servicio.
- Incidencias críticas.
- Reversiones complejas.
- Pérdidas reputacionales.

Por tanto, el coste adicional se considera plenamente justificado desde una perspectiva de riesgo empresarial.

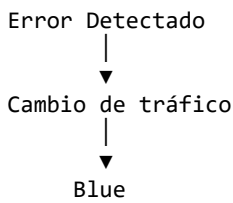
14.9 Estrategia de Rollback Asociada

Uno de los principales beneficios de Blue-Green Deployment es la simplicidad del proceso de recuperación.

Escenario Normal



Escenario de Incidencia



Objetivos Operativos

Indicador	Objetivo
Tiempo detección	< 5 minutos
Tiempo rollback	< 2 minutos
MTTR	< 15 minutos
Interrupción de servicio	0 minutos

14.10 Indicadores de Éxito

KPI	Objetivo
Despliegues sin downtime	100 %
Releases con rollback	< 5 %
Disponibilidad	> 99,5 %
Tiempo de recuperación	< 15 min
Validación previa al despliegue	100 %
Releases automatizadas	> 95 %

15 Observabilidad

15.1 Introducción

La observabilidad constituye uno de los pilares fundamentales de cualquier plataforma DevOps moderna. En entornos basados en contenedores, automatización y orquestación mediante Kubernetes, la capacidad para conocer el estado interno de las aplicaciones resulta tan importante como la propia funcionalidad del software.

Mientras que la monitorización tradicional permite identificar que existe un problema, la observabilidad proporciona las herramientas necesarias para comprender por qué se ha producido, cuál es su impacto y qué acciones deben realizarse para resolverlo.

La estrategia de observabilidad definida para TechWave Solutions se basa en el principio de **Observability by Design**, incorporando capacidades de supervisión desde las primeras fases de desarrollo de la aplicación. Este enfoque permite construir sistemas más robustos, facilitar el diagnóstico de incidencias y reducir significativamente los tiempos de recuperación operativa.

Durante la ejecución práctica del proyecto se ha implementado una primera capa de observabilidad mediante la instrumentación de la aplicación Node.js utilizando métricas compatibles con Prometheus. Asimismo, se ha diseñado una arquitectura objetivo preparada para evolucionar hacia una plataforma completa basada en Prometheus, Grafana, Loki y OpenTelemetry.

15.2 Importancia de la Observabilidad en DevOps

Uno de los principios fundamentales de DevOps consiste en tomar decisiones basadas en datos reales y no en suposiciones. Para ello resulta imprescindible disponer de información continua sobre el comportamiento de los sistemas desplegados.

La observabilidad proporciona:

- Visibilidad sobre el comportamiento de las aplicaciones.
- Detección temprana de incidencias.
- Diagnóstico más rápido de problemas.
- Toma de decisiones basada en evidencias.
- Mejora continua de la plataforma.
- Reducción del tiempo medio de recuperación (MTTR).
- Soporte a procesos de escalabilidad y optimización.

En plataformas modernas basadas en Kubernetes y arquitecturas Cloud Native, la observabilidad se convierte en una capacidad estratégica que permite mantener niveles elevados de disponibilidad, fiabilidad y calidad del servicio.

15.3 Objetivos de la Observabilidad

La estrategia propuesta para TechWave Solutions persigue los siguientes objetivos.

OO-01 Visibilidad Operativa

Disponer de información continua acerca del estado de la aplicación y de la plataforma donde se ejecuta.

Beneficios

- Supervisión permanente.
- Detección de anomalías.
- Capacidad de seguimiento operativo.

OO-02 Reducción del MTTR

Facilitar la identificación de problemas para reducir el tiempo necesario para restaurar el servicio.

Beneficios

- Menor impacto operativo.
- Resolución más rápida de incidencias.
- Mayor continuidad de servicio.

OO-03 Diagnóstico Proactivo

Detectar comportamientos anómalos antes de que generen una degradación perceptible para los usuarios.

Beneficios

- Prevención de incidencias.
- Mayor estabilidad.
- Mejora de la experiencia de usuario.

OO-04 Escalabilidad

Preparar la plataforma para futuras integraciones con herramientas avanzadas de observabilidad.

Beneficios

- Evolución tecnológica simplificada.
- Menor coste de integración.
- Arquitectura preparada para crecimiento.

OO-05 Gobierno Operacional

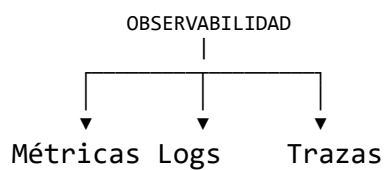
Proporcionar indicadores objetivos que permitan evaluar el estado de la plataforma.

Beneficios

- Gestión basada en datos.
- Generación de KPIs.
- Soporte a la toma de decisiones.

15.4 Fundamentos de la Observabilidad

La observabilidad moderna se apoya en tres pilares fundamentales.



Cada uno de estos elementos proporciona una perspectiva diferente sobre el comportamiento de la plataforma.

Métricas

Las métricas representan información numérica recopilada de forma periódica.

Permiten responder preguntas como:

- ¿Cuánta memoria consume la aplicación?
- ¿Cuál es el tiempo medio de respuesta?
- ¿Cuántas peticiones se procesan por segundo?
- ¿Cuál es el porcentaje de errores?

Las métricas constituyen la base de los sistemas de monitorización modernos.

Logs

Los logs registran eventos producidos durante la ejecución del sistema.

Permiten conocer:

- Qué ocurrió.
- Cuándo ocurrió.
- En qué componente ocurrió.

Los registros son especialmente útiles para la investigación de incidentes y tareas de auditoría.

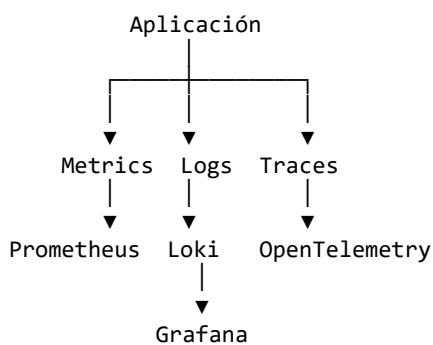
Trazas

Las trazas permiten seguir el recorrido completo de una petición a través de los distintos componentes que participan en su procesamiento.

Su utilización resulta especialmente útil en sistemas distribuidos y arquitecturas basadas en microservicios.

15.5 Arquitectura de Observabilidad Objetivo

La arquitectura objetivo diseñada para TechWave Solutions contempla la implantación progresiva de una plataforma integral de observabilidad.



Esta arquitectura permitirá recopilar métricas, centralizar registros, correlacionar eventos y visualizar información operativa desde una única plataforma.

Los componentes propuestos son:

Prometheus

Responsable de recopilar y almacenar métricas procedentes de aplicaciones y componentes Kubernetes.

Grafana

Herramienta de visualización destinada a la creación de cuadros de mando e indicadores operativos.

Loki

Plataforma orientada a la centralización y consulta de registros.

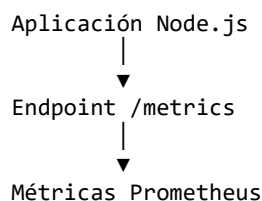
OpenTelemetry

Estándar abierto para la generación y exportación de trazas distribuidas.

15.6 Implementación Realizada

Aunque la arquitectura completa forma parte de la evolución futura de la plataforma, durante el desarrollo del proyecto se ha implementado una primera capa real de observabilidad basada en métricas compatibles con Prometheus.

La aplicación Node.js ha sido instrumentada utilizando la librería **Prometheus Client**, permitiendo exponer información interna mediante un endpoint específico.



Esta implementación proporciona información suficiente para evaluar el comportamiento de la aplicación y validar la integración futura con Prometheus y Grafana.

15.7 Instrumentación de la Aplicación

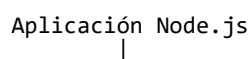
La instrumentación constituye el mecanismo mediante el cual una aplicación genera información acerca de su propio comportamiento interno. En la solución desarrollada para TechWave Solutions, la instrumentación se ha implementado utilizando la librería **Prometheus Client para Node.js**, permitiendo exponer métricas operativas sin necesidad de modificar significativamente la lógica de negocio.

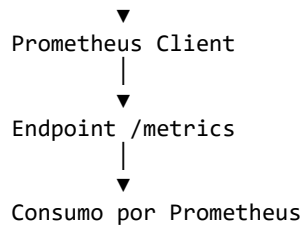
Esta aproximación sigue las buenas prácticas recomendadas en entornos Cloud Native, donde las aplicaciones deben ser capaces de proporcionar información útil para tareas de monitorización, diagnóstico y operación.

La librería utilizada recopila automáticamente información sobre distintos componentes internos de Node.js y publica los resultados mediante un endpoint HTTP accesible desde Kubernetes.

15.7.1 Arquitectura de Instrumentación

La arquitectura implementada es sencilla pero suficientemente completa para validar la integración con plataformas de observabilidad.





Este enfoque permite desacoplar la generación de métricas de las herramientas encargadas de consumirlas y visualizarlas.

15.7.2 Métricas del Proceso

La aplicación expone métricas relacionadas con el estado general del proceso Node.js.

Estas métricas permiten supervisar aspectos como:

- Tiempo de ejecución del proceso.
- Consumo de CPU.
- Número de operaciones ejecutadas.
- Estado general de la aplicación.

La monitorización de estos indicadores facilita la detección de comportamientos anómalos y proporciona una visión general de la salud de la aplicación.

15.7.3 Métricas de Memoria

La gestión eficiente de memoria resulta fundamental en aplicaciones ejecutadas de forma continua dentro de contenedores Kubernetes.

La instrumentación implementada permite recopilar información relacionada con:

- Memoria utilizada.
- Heap total asignado.
- Heap utilizado.
- Resident Set Size (RSS).
- Memoria disponible para el proceso.

Estas métricas permiten detectar posibles fugas de memoria y sirven como base para futuras configuraciones de escalado y optimización.

Beneficios

- Detección temprana de consumos anómalos.
- Optimización del dimensionamiento de pods.
- Mejora de la estabilidad operativa.

15.7.4 Métricas del Event Loop

Node.js se basa en una arquitectura orientada a eventos gestionada mediante un Event Loop.

El rendimiento de este componente tiene un impacto directo sobre la capacidad de la aplicación para procesar peticiones concurrentes.

Las métricas recopiladas permiten analizar:

- Retrasos del Event Loop.
- Saturación del proceso.
- Bloqueos potenciales.
- Degradaciones de rendimiento.

Beneficios

- Identificación de cuellos de botella.
- Mejora de tiempos de respuesta.
- Diagnóstico avanzado de problemas de rendimiento.

15.7.5 Métricas del Garbage Collector

El Garbage Collector es el mecanismo encargado de liberar memoria que ya no está siendo utilizada por la aplicación.

Un comportamiento ineficiente del recolector puede provocar:

- Incremento de latencia.
- Consumo excesivo de memoria.
- Degradación general del rendimiento.

La instrumentación implementada permite conocer:

- Frecuencia de ejecución.
- Duración de cada ciclo.
- Impacto sobre el proceso principal.

Beneficios

- Identificación de problemas de memoria.
- Optimización de rendimiento.
- Supervisión avanzada del runtime Node.js.

15.7.6 Métricas Compatibles con Prometheus

Todas las métricas generadas se publican siguiendo el formato estándar definido por Prometheus.

Algunos ejemplos son:

```
process_cpu_user_seconds_total
process_resident_memory_bytes
process_virtual_memory_bytes
nodejs_heap_size_total_bytes
nodejs_heap_size_used_bytes
nodejs_eventloop_lag_seconds
```

La utilización de este formato garantiza la compatibilidad con el ecosistema Cloud Native y facilita la integración futura con herramientas como Prometheus, Grafana y OpenTelemetry.

15.7.7 Endpoint de Exposición de Métricas

Toda la información de observabilidad generada por la aplicación se encuentra disponible mediante el endpoint:

GET /metrics

Este recurso devuelve las métricas en formato texto compatible con Prometheus y puede ser consultado tanto desde Kubernetes como desde herramientas externas.

Durante las pruebas realizadas, el endpoint permitió verificar correctamente la exposición de información interna del proceso Node.js y validar el correcto funcionamiento de la instrumentación implementada.

Exposición de Métricas Prometheus

curl <http://localhost:3000/metrics>

```
Fernandomartinez@PF4X06KP:~$ curl http://localhost:3000/metrics
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 0.533329

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 0.18035700000000002

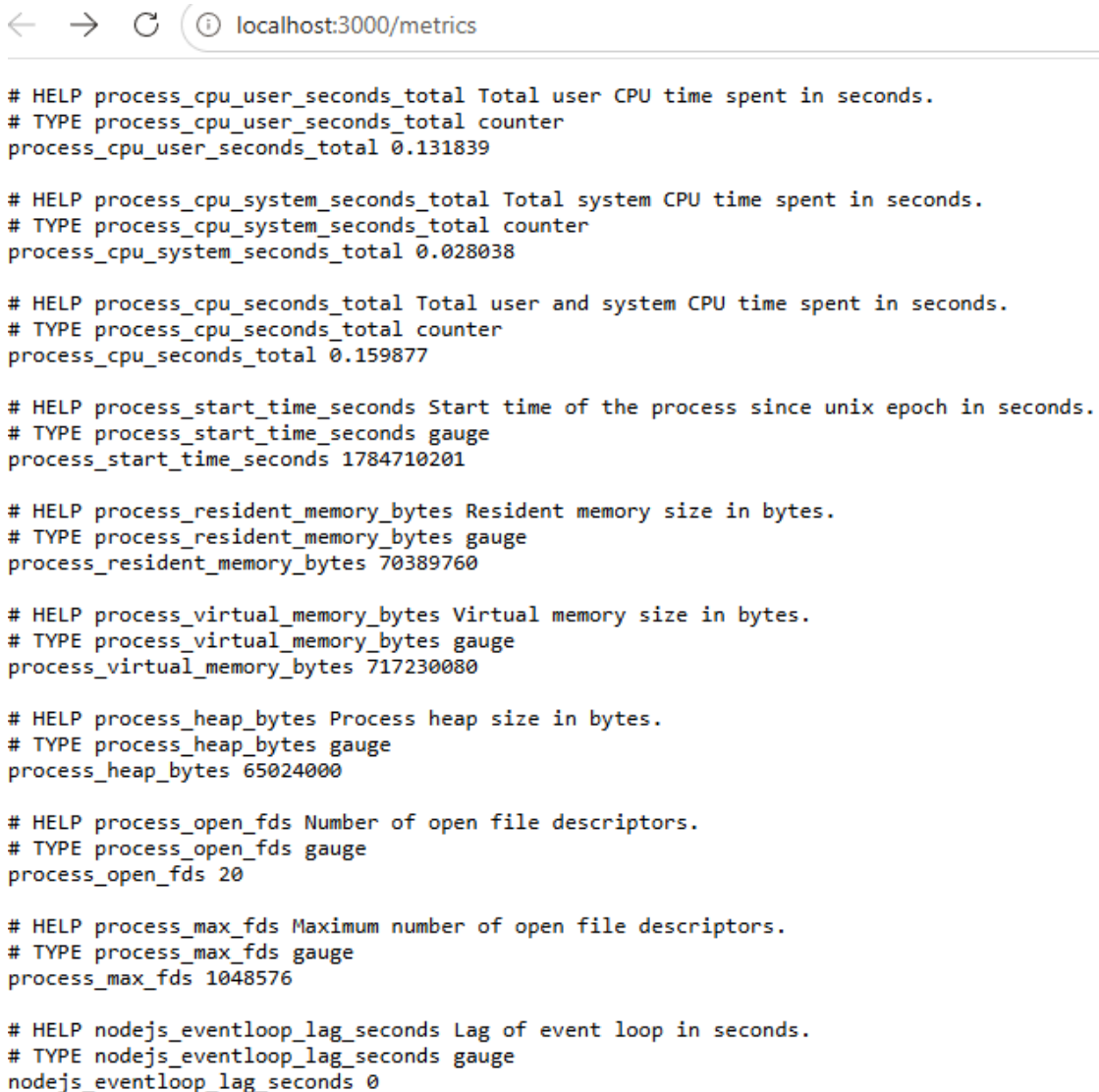
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 0.713686

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1784710201

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 71901184
```

o acceso mediante navegador:

<http://localhost:3000/metrics>



```
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 0.131839

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 0.028038

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 0.159877

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1784710201

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 70389760

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 717230080

# HELP process_heap_bytes Process heap size in bytes.
# TYPE process_heap_bytes gauge
process_heap_bytes 65024000

# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds 20

# HELP process_max_fds Maximum number of open file descriptors.
# TYPE process_max_fds gauge
process_max_fds 1048576

# HELP nodejs_eventloop_lag_seconds Lag of event loop in seconds.
# TYPE nodejs_eventloop_lag_seconds gauge
nodejs_eventloop_lag_seconds 0
```

Figura 15.1. Endpoint /metrics expuesto por la aplicación Node.js. La salida muestra métricas compatibles con Prometheus relativas al consumo de memoria, estado del proceso y comportamiento interno del runtime.

15.7.8 Beneficios de la Instrumentación Implementada

La solución desarrollada proporciona una primera capacidad real de observabilidad que aporta múltiples ventajas operativas:

- **Visibilidad Operativa** Permite conocer el comportamiento interno de la aplicación en tiempo real.
- **Diagnóstico Simplificado** Facilita la identificación de problemas relacionados con rendimiento y consumo de recursos.

- **Preparación para Escalabilidad** Las métricas obtenidas pueden utilizarse como base para futuras estrategias de escalado automático.
- **Integración Cloud Native** La compatibilidad nativa con Prometheus facilita la incorporación futura de una plataforma completa de monitorización.
- **Evolución Tecnológica** La arquitectura implementada permite añadir nuevas herramientas de observabilidad sin necesidad de modificar la lógica de negocio de la aplicación.

Figura 15.2 – Estado de los Pods Kubernetes

La observabilidad no se limita únicamente a las métricas expuestas por la aplicación. Kubernetes proporciona información en tiempo real sobre el estado de las cargas de trabajo desplegadas, permitiendo verificar su disponibilidad y detectar posibles incidencias operativas.

Durante las pruebas de validación se verificó el estado de los pods asociados a la aplicación mediante la consulta del namespace techwave.

```
kubectl get pods -n techwave
```

Análisis de resultados

La información obtenida permite comprobar:

- Estado actual de cada pod.
- Número de reinicios registrados.
- Tiempo de actividad.
- Disponibilidad de las réplicas desplegadas.

La presencia del estado **Running** indica que las instancias de la aplicación se encuentran funcionando correctamente y pueden atender solicitudes.

```
fernandomartinez@PF4X06KP:~$ kubectl get pods -n techwave
NAME                                READY   STATUS    RESTARTS   AGE
techwave-blue-5487fb9957-4w6t8      1/1     Running   2 (5m16s ago)    9d
techwave-blue-5487fb9957-5zf6t      1/1     Running   2 (5m16s ago)    9d
techwave-green-5f8c586694-6s96z     1/1     Running   2 (5m16s ago)    9d
techwave-green-5f8c586694-mrxct     1/1     Running   2 (5m16s ago)    9d
```

Figura 15.2. Estado de los pods desplegados en Kubernetes. La consulta muestra las instancias Blue y Green ejecutándose correctamente en el namespace de la aplicación, confirmando la disponibilidad de la plataforma y la correcta ejecución de la estrategia de despliegue.

Figura 15.3 – Estado de los Deployments

Para complementar la supervisión operativa se validó el estado de los recursos Deployment encargados de gestionar el ciclo de vida de los pods.

Los deployments constituyen uno de los componentes más importantes de Kubernetes, ya que garantizan que el número deseado de réplicas permanezca siempre disponible.

```
kubectl get deployments -n techwave
```

La salida obtenida permite verificar:

- Réplicas configuradas.
- Réplicas disponibles.
- Estado de actualización.
- Correcta ejecución de la estrategia Blue-Green.

La coincidencia entre los valores **READY**, **UP-TO-DATE** y **AVAILABLE** confirma la ausencia de incidencias durante el despliegue.

```
fernandomartinez@PF4X06KP:~$ kubectl get deployments -n techwave
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
techwave-blue	2/2	2	2	9d
techwave-green	2/2	2	2	9d

Figura 15.3. Estado de los deployments de Kubernetes.

Se observa que las versiones Blue y Green mantienen la totalidad de sus réplicas disponibles, garantizando la continuidad del servicio y facilitando futuras operaciones de despliegue y rollback.

```
fernandomartinez@PF4X06KP:~$ kubectl get deployment techwave-blue -n techwave -o yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{"name":"techwave-blue","namespace":"techwave"},"spec":{"replicas":2,"selector":{"matchLabels":{"app":"techwave","version":"blue"}},"template":{"metadata":{"labels":{"app":"techwave","version":"blue"}},"spec":{"containers":[{"image":"ghcr.io/fmartinezaltolaguirre/techwave-app:latest","imagePullPolicy":"Always","livenessProbe":{"failureThreshold":3,"httpGet":{"path":"/health","port":3000},"initialDelaySeconds":15,"periodSeconds":20,"timeoutSeconds":2},"name":"techwave","ports":[{"containerPort":3000}],"readinessProbe":{"failureThreshold":3,"httpGet":{"path":"/health","port":3000},"initialDelaySeconds":5,"periodSeconds":10,"timeoutSeconds":2},"resources":{"limits":{"cpu":"500m","memory":"512Mi"},"requests":{"cpu":"100m","memory":"128Mi"}}}}}}}}
  creationTimestamp: "2026-07-13T07:21:45Z"
  generation: 1
  name: techwave-blue
  namespace: techwave
  resourceVersion: "193609"
  uid: 9f879626-9388-4b73-88cf-eebd2a542f59
spec:
  progressDeadlineSeconds: 600
  replicas: 2
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: techwave
      version: blue
  strategy:
    rollingUpdates:
      maxSurge: 25%
      maxUnavailable: 25%
      type: RollingUpdate
  template:
    metadata:
      labels:
        app: techwave
        version: blue
    spec:
      containers:
      - image: ghcr.io/fmartinezaltolaguirre/techwave-app:latest
        imagePullPolicy: Always
        livenessProbe:
          failureThreshold: 3
          httpGet:
            path: /health
            port: 3000
            scheme: HTTP
          initialDelaySeconds: 15
          periodSeconds: 20
```

Figura 15.4 – Configuración de Readiness y Liveness Probes

Además de las métricas expuestas por la aplicación, Kubernetes incorpora mecanismos nativos que permiten supervisar automáticamente el estado de los contenedores desplegados.

En la solución desarrollada para TechWave Solutions se han configurado **Readiness Probes** y **Liveness Probes** basadas en el endpoint /health, permitiendo que Kubernetes determine en todo momento si una instancia se encuentra preparada para recibir tráfico o si debe ser reiniciada.

La utilización de estas sondas constituye una práctica recomendada en entornos Kubernetes, ya que mejora la resiliencia de la plataforma y reduce el impacto de fallos durante la operación.

Readiness Probe

La **Readiness Probe** indica si un contenedor está preparado para recibir peticiones.

Mientras la comprobación no sea satisfactoria, Kubernetes evitará enviar tráfico a la instancia afectada.

Esta funcionalidad resulta especialmente útil durante:

- Arranque de la aplicación.
- Despliegues Blue-Green.
- Actualizaciones de versión.
- Recuperación tras incidencias.

Liveness Probe

La **Liveness Probe** permite detectar aplicaciones bloqueadas o en estados no recuperables.

Cuando una sonda de vida falla de forma continuada, Kubernetes reinicia automáticamente el contenedor para restaurar el servicio.

Este mecanismo contribuye a mantener una elevada disponibilidad y reduce la necesidad de intervención manual.

Configuración Implementada

```
livenessProbe:
  httpGet:
    path: /health
    port: 3000
  initialDelaySeconds: 30
  periodSeconds: 10

readinessProbe:
  httpGet:
    path: /health
    port: 3000
  initialDelaySeconds: 10
  periodSeconds: 5
```

Beneficios Operativos

La implementación de estas sondas proporciona múltiples ventajas:

- Detección automática de fallos.
- Recuperación automática de contenedores.
- Protección frente a despliegues defectuosos.
- Mejora de la disponibilidad del servicio.
- Reducción del tiempo de recuperación ante incidencias.

```

$ kubectl get deployment techwave-green -n techwave -o yaml | grep -A20 livenessProbe
    {"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{"name":"techwave-green","namespace":"techwave"},"spec":{"replicas":2,"selector":{"matchLabels":{"app":"techwa
ve","version":"green"},"template":{"metadata":{"labels":{"app":"techwave","version":"green"},"spec":{"containers":[{"image":"ghcr.io/fmartinezaltola/techwave-app:latest","imagePull
Policy":"Always","livenessProbe":{"failureThreshold":3,"httpGet":{"path":"/health","port":3000},"initialDelaySeconds":15,"periodSeconds":20,"timeoutSeconds":2},"name":"techwave","ports":[{"
containerPort":3000},"readinessProbe":{"failureThreshold":3,"httpGet":{"path":"/health","port":3000},"initialDelaySeconds":5,"periodSeconds":10,"timeoutSeconds":2},"resources":{"limits
":{"cpu":"500m","memory":"512Mi"},"requests":{"cpu":"100m","memory":"128Mi"}}}}]}}}}}
creationTimestamp: "2026-07-13T07:21:45Z"
generation: 1
name: techwave-green
namespace: techwave
resourceVersion: "193603"
uid: 1f8c54d9-9b56-44fc-a746-1c8a75c0378c
spec:
  progressDeadlineSeconds: 600
  replicas: 2
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      app: techwave
      version: green
  strategy:
    rollingUpdate:
      maxSurge: 25%
      maxUnavailable: 25%
      type: RollingUpdate
  template:
    metadata:
      labels:
        app: techwave
        version: green
    spec:
      containers:
      - image: ghcr.io/fmartinezaltola/techwave-app:latest
        imagePullPolicy: Always
        livenessProbe:
          failureThreshold: 3
          httpGet:
            path: /health
            port: 3000
          scheme: HTTP
          initialDelaySeconds: 15
          periodSeconds: 20
          successThreshold: 1
          timeoutSeconds: 2
        name: techwave
        ports:
        - containerPort: 3000
          protocol: TCP
        readinessProbe:
          failureThreshold: 3
          httpGet:
            path: /health
            port: 3000
          scheme: HTTP
          initialDelaySeconds: 5

```

Figura 15.4 – Configuración de Readiness y Liveness Probes

Figura 15.4. Configuración de las sondas de salud utilizadas por Kubernetes.

Las Readiness Probes controlan cuándo una instancia puede recibir tráfico, mientras que las Liveness Probes permiten detectar fallos de ejecución y activar mecanismos automáticos de recuperación.

Análisis de la Configuración

La estrategia adoptada utiliza el endpoint /health como punto único de validación del estado de la aplicación.

Este diseño simplifica la operación de la plataforma y garantiza que Kubernetes pueda evaluar de forma continua la disponibilidad real del servicio.

La combinación de métricas Prometheus junto con las capacidades nativas de supervisión proporcionadas por Kubernetes constituye una primera capa de observabilidad alineada con las buenas prácticas Cloud Native y prepara la aplicación para futuras integraciones con plataformas avanzadas de monitorización y alertado.

Figura 15.5 – Validación Operativa mediante Liveness Probe

Las **Liveness Probes** constituyen uno de los mecanismos fundamentales de resiliencia proporcionados por Kubernetes. Su función principal consiste en verificar periódicamente que la aplicación continúa funcionando correctamente

y que el proceso principal no se encuentra bloqueado o en un estado irrecuperable.

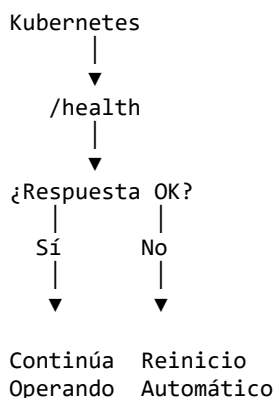
A diferencia de las Readiness Probes, cuyo objetivo es determinar si una instancia puede recibir tráfico, las Liveness Probes permiten detectar situaciones en las que la aplicación ha dejado de responder correctamente y requiere un reinicio automático.

En la solución desarrollada para TechWave Solutions, Kubernetes utiliza el endpoint /health para validar periódicamente el estado interno de cada contenedor.

```
livenessProbe:
  httpGet:
    path: /health
    port: 3000
  initialDelaySeconds: 30
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3
```

Funcionamiento de la Liveness Probe

El proceso de validación sigue las siguientes etapas:



Cada comprobación genera una solicitud HTTP al endpoint de salud de la aplicación.

Si la respuesta es satisfactoria:

- El contenedor continúa funcionando normalmente.

Si la comprobación falla repetidamente:

- Kubernetes considera que la aplicación no es saludable.
- Se inicia automáticamente un nuevo contenedor.
- Se restablece el servicio sin intervención manual.

Validación Realizada

Durante las pruebas del proyecto se verificó que el endpoint /health respondía correctamente a las solicitudes utilizadas tanto por las Readiness Probes como por las Liveness Probes.

La validación se realizó mediante:

curl <http://localhost:3000/health>

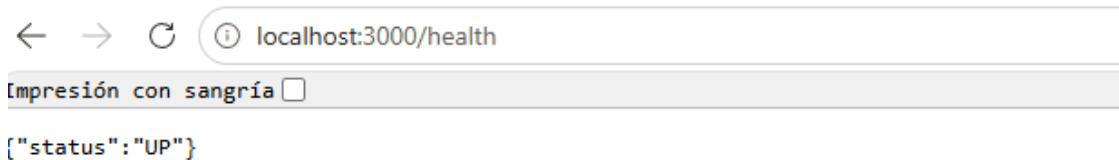
Obteniendo una respuesta similar a la siguiente:

```
{
  "status": "UP"
}
```

La respuesta confirma que la aplicación se encuentra operativa y preparada para ser supervisada por Kubernetes.

Acceso mediante navegador:

<http://localhost:3000/health>



o mediante terminal:

curl <http://localhost:3000/health>

```
fernandomartinez@PF4X06KP:~$ curl http://localhost:3000/health
{"status":"UP"}fernandomartinez@PF4X06KP:~$
```

Figura 15.5. Validación del endpoint /health utilizado por Kubernetes para la ejecución de las Liveness Probes. La respuesta satisfactoria confirma que la aplicación se encuentra operativa y puede continuar prestando servicio de forma segura.

Beneficios de las Liveness Probes

La utilización de Liveness Probes aporta importantes ventajas operativas:

- Recuperación automática ante fallos de aplicación.
- Reducción de tiempos de indisponibilidad.
- Menor intervención manual de los equipos de operación.
- Mayor estabilidad de la plataforma.
- Incremento de la disponibilidad del servicio.

En combinación con las Readiness Probes descritas anteriormente, las Liveness Probes constituyen un mecanismo esencial para garantizar la resiliencia y el correcto funcionamiento de aplicaciones desplegadas sobre Kubernetes.

15.8 Validación de la Instrumentación

Una vez implementada la instrumentación mediante Prometheus Client para Node.js, se realizaron distintas pruebas con el objetivo de verificar el correcto funcionamiento del mecanismo de exposición de métricas.

Las actividades de validación permitieron confirmar no solo la disponibilidad del endpoint de observabilidad, sino también la accesibilidad de la aplicación desde el entorno Kubernetes y el correcto funcionamiento de los mecanismos de supervisión implementados.

Los criterios de validación definidos fueron los siguientes:

Validación	Resultado
Endpoint /metrics accesible	✓
Formato Prometheus válido	✓
Métricas Node.js visibles	✓
Acceso desde Kubernetes	✓
Endpoint /health accesible	✓
Endpoint /version accesible	✓

Validación del Endpoint de Métricas

La primera comprobación consistió en acceder al endpoint /metrics expuesto por la aplicación.

```
curl http://localhost:3000/metrics
```

La respuesta obtenida mostró correctamente las métricas generadas por Prometheus Client.

Entre las métricas observadas durante las pruebas destacan:

```
process_cpu_user_seconds_total
process_resident_memory_bytes
nodejs_heap_size_total_bytes
nodejs_eventloop_lag_seconds
```

Estos resultados confirman que la aplicación expone información operativa compatible con el ecosistema Prometheus.

Captura real del endpoint mostrando métricas Prometheus.

```
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 3.8331470000000003

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 1.127976

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 4.961123

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1784710201

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 73068544

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes 718786560
```

La aplicación publica métricas operativas compatibles con Prometheus, permitiendo la integración futura con plataformas de monitorización y observabilidad.

Validación del Estado del Servicio

Además de las métricas operativas, se validó el endpoint de salud utilizado por Kubernetes.

curl <http://localhost:3000/health>

Respuesta obtenida:

```
{
  "status": "UP"
}
```

La respuesta confirma que la aplicación se encuentra operativa y preparada para recibir tráfico.

Captura recomendada

Acceso mediante navegador o terminal.

```
fernandomartinez@PF4X06KP:~$ curl http://localhost:3000/health
{"status":"UP"}
```

Validación del estado operativo de la aplicación mediante el endpoint de salud utilizado por Kubernetes.

Validación de la Versión Desplegada

La solución incorpora un endpoint específico para identificar la versión desplegada en cada momento.

curl <http://localhost:3000/version>

Respuesta esperada:

```
{  
  "version": "1.0.0"  
}
```

Este mecanismo resulta especialmente útil durante despliegues Blue-Green y operaciones de rollback.

```
fernandomartinez@PF4X06KP:~$ curl http://localhost:3000/version  
{"version":"1.0.0"}fernandomartinez@PF4X06KP:~$
```

Figura 15.3 – Endpoint /version

Captura recomendada

Respuesta mostrada mediante navegador o terminal.

Pie de figura

Validación de la versión actualmente desplegada y accesible desde la plataforma Kubernetes.

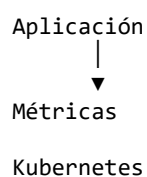
15.9 Observabilidad en Kubernetes

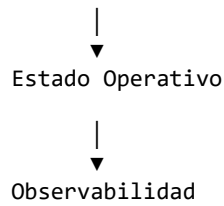
Kubernetes incorpora múltiples capacidades de observación orientadas a la operación y administración de aplicaciones contenerizadas.

Estas capacidades complementan las métricas generadas por la aplicación y proporcionan una visión global del estado de la plataforma.

La estrategia de observabilidad adoptada combina información procedente de:

- Aplicación.
- Pods.
- Deployments.
- Servicios Kubernetes.
- Sondas de salud.





Supervisión de Pods

Los Pods constituyen la unidad mínima de ejecución dentro de Kubernetes.

Su monitorización permite conocer en tiempo real:

- Estado de ejecución.
- Número de reinicios.
- Disponibilidad.
- Tiempo de actividad.

La validación se realizó mediante:

```
kubectl get pods -n techwave
```

```
fernandomartinez@PF4X06KP:~$ kubectl get pods -n techwave
NAME                                READY   STATUS    RESTARTS   AGE
techwave-blue-5487fb9957-4w6t8     1/1     Running   2 (20m ago)  9d
techwave-blue-5487fb9957-5zf6t     1/1     Running   2 (20m ago)  9d
techwave-green-5f8c586694-6s96z    1/1     Running   2 (20m ago)  9d
techwave-green-5f8c586694-mrxct    1/1     Running   2 (20m ago)  9d
```

Figura 15.4 – Estado de los Pods

Estado de los pods desplegados en Kubernetes durante las pruebas de validación.

Supervisión de Deployments

Los Deployments permiten gestionar las actualizaciones y garantizar la disponibilidad de las aplicaciones.

La monitorización de este recurso proporciona información acerca de:

- Réplicas configuradas.
- Réplicas disponibles.
- Estado de las actualizaciones.
- Capacidad de recuperación.

Comando utilizado:

```
kubectl get deployments -n techwave
```

Figura 15.5 – Estado de los Deployments

Captura recomendada

NAME	READY	AVAILABLE
techwave-blue	1/1	1
techwave-green	1/1	1

Pie de figura

Estado operativo de los deployments utilizados en la estrategia Blue-Green.

Beneficios de la Observabilidad en Kubernetes

La información proporcionada por Kubernetes aporta ventajas significativas para la operación de la plataforma:

- Detección temprana de problemas.
- Verificación de disponibilidad.
- Diagnóstico simplificado.
- Supervisión continua.
- Soporte a despliegues Blue-Green.

15.10 Readiness y Liveness Probes

Uno de los mecanismos más importantes de observabilidad incorporados en Kubernetes son las sondas de salud.

Estas sondas permiten verificar automáticamente el estado de las aplicaciones desplegadas y tomar acciones correctivas cuando se detectan problemas.

La solución desarrollada utiliza el endpoint /health como punto único de validación.

Readiness Probe

La Readiness Probe determina si una instancia está preparada para recibir peticiones.

Cuando una comprobación falla:

- Kubernetes mantiene el pod en ejecución.
- El tráfico deja de enviarse a dicha instancia.
- Se evita impactar a los usuarios finales.

Configuración Implementada

```
readinessProbe:
  httpGet:
    path: /health
    port: 3000
```

```
initialDelaySeconds: 10
periodSeconds: 5
```

Beneficios

- Protección durante despliegues.
- Evita tráfico hacia instancias no disponibles.
- Mejora de la experiencia de usuario.

Liveness Probe

La Liveness Probe verifica que la aplicación continúa funcionando correctamente.

Cuando se detectan fallos repetidos:

- Kubernetes reinicia automáticamente el contenedor.
- Se restablece el servicio.
- Se minimiza la intervención manual.

Configuración Implementada

```
livenessProbe:
  httpGet:
    path: /health
    port: 3000
  initialDelaySeconds: 30
  periodSeconds: 10
```

Beneficios

- Recuperación automática.
- Mayor resiliencia.
- Reducción del tiempo de indisponibilidad.

15.11 Dashboards de Observabilidad Diseñados

Una vez validada la capacidad de instrumentación de la aplicación y la exposición de métricas compatibles con Prometheus, el siguiente paso natural dentro de la evolución de la plataforma consiste en la incorporación de cuadros de mando que permitan visualizar la información recopilada de forma centralizada.

Con este objetivo se ha diseñado una propuesta de integración con Grafana que facilite la supervisión operativa de la plataforma desde diferentes perspectivas: aplicación, clúster Kubernetes, infraestructura y negocio.

Aunque la implantación completa de Grafana queda fuera del alcance funcional del proyecto, los dashboards definidos constituyen una referencia realista para futuras fases de madurez de la plataforma.

15.11.1 Dashboard de Aplicación

El Dashboard de Aplicación está orientado al seguimiento del comportamiento funcional y técnico del servicio desarrollado.

<https://github.com/fmartinezaltolaguirre/pf-devops-kubernetes/blob/main/grafana/dashboard/techwave-application-dashboard.json>

Su objetivo principal consiste en proporcionar visibilidad inmediata sobre el estado de la aplicación y detectar posibles degradaciones de rendimiento antes de que impacten en los usuarios.

Métricas previstas

- Disponibilidad del servicio.
- Tiempo medio de respuesta.
- Latencia P95.
- Latencia P99.
- Peticiones por segundo.
- Tasa de errores.
- Uso de memoria.
- Estado de las instancias desplegadas.

Beneficios

- Detección temprana de incidencias.
- Supervisión del rendimiento.
- Análisis de capacidad.
- Seguimiento de SLA.

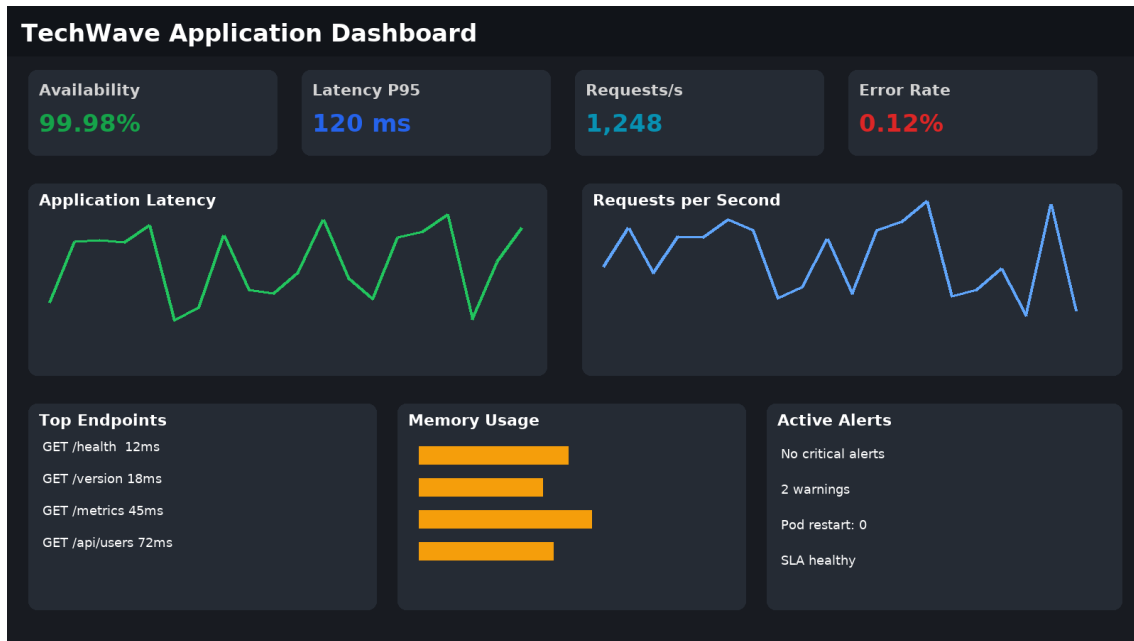


Figura 15.6 – Dashboard de Aplicación Propuesto

Propuesta de dashboard orientado a la supervisión de métricas de aplicación, incluyendo disponibilidad, latencia, rendimiento y estado operativo.

Entre los paneles contemplados destacan:

- Disponibilidad del servicio.
- Latencia P95.
- Peticiones por segundo.
- Tasa de error.
- Evolución temporal del rendimiento.
- Consumo de memoria.
- Estado de alertas activas.
- Endpoints más utilizados.

15.11.2 Dashboard Kubernetes

El Dashboard Kubernetes está destinado a los equipos de operación y administración de plataforma.

<https://github.com/fmartinezaltolaguirre/pf-devops-kubernetes/blob/main/grafana/dashboard/kubernetes-dashboard.json>

Proporciona una visión consolidada del estado del clúster y de las cargas de trabajo desplegadas.

Métricas previstas

- Estado de nodos.

- Pods en ejecución.
- Deployments activos.
- Namespaces.
- Consumo de recursos.
- Eventos del clúster.
- Estado de escalado.
- Alertas activas.

Beneficios

- Visibilidad completa del clúster.
- Detección de incidencias operativas.
- Supervisión de la capacidad disponible.
- Seguimiento de despliegues.



Figura 15.7 – Dashboard Kubernetes Propuesto

Cuadro de mando orientado a la supervisión operativa del clúster Kubernetes y de las aplicaciones desplegadas.

15.11.3 Dashboard de Infraestructura

La observabilidad de aplicaciones debe complementarse con la supervisión de la infraestructura que soporta los servicios.

<https://github.com/fmartinezaltolaguirre/pf-devops-kubernetes/blob/main/grafana/dashboard/infrastructure-dashboard.json>

Para ello se propone un dashboard específico destinado al análisis de recursos físicos y virtuales.

Métricas previstas

- Utilización de CPU.
- Consumo de memoria.
- Uso de almacenamiento.
- Rendimiento de red.
- Disponibilidad de nodos.
- Capacidad restante.
- Tendencias de crecimiento.

Beneficios

- Planificación de capacidad.
- Optimización de costes.
- Identificación de saturaciones.
- Soporte a decisiones de escalado.

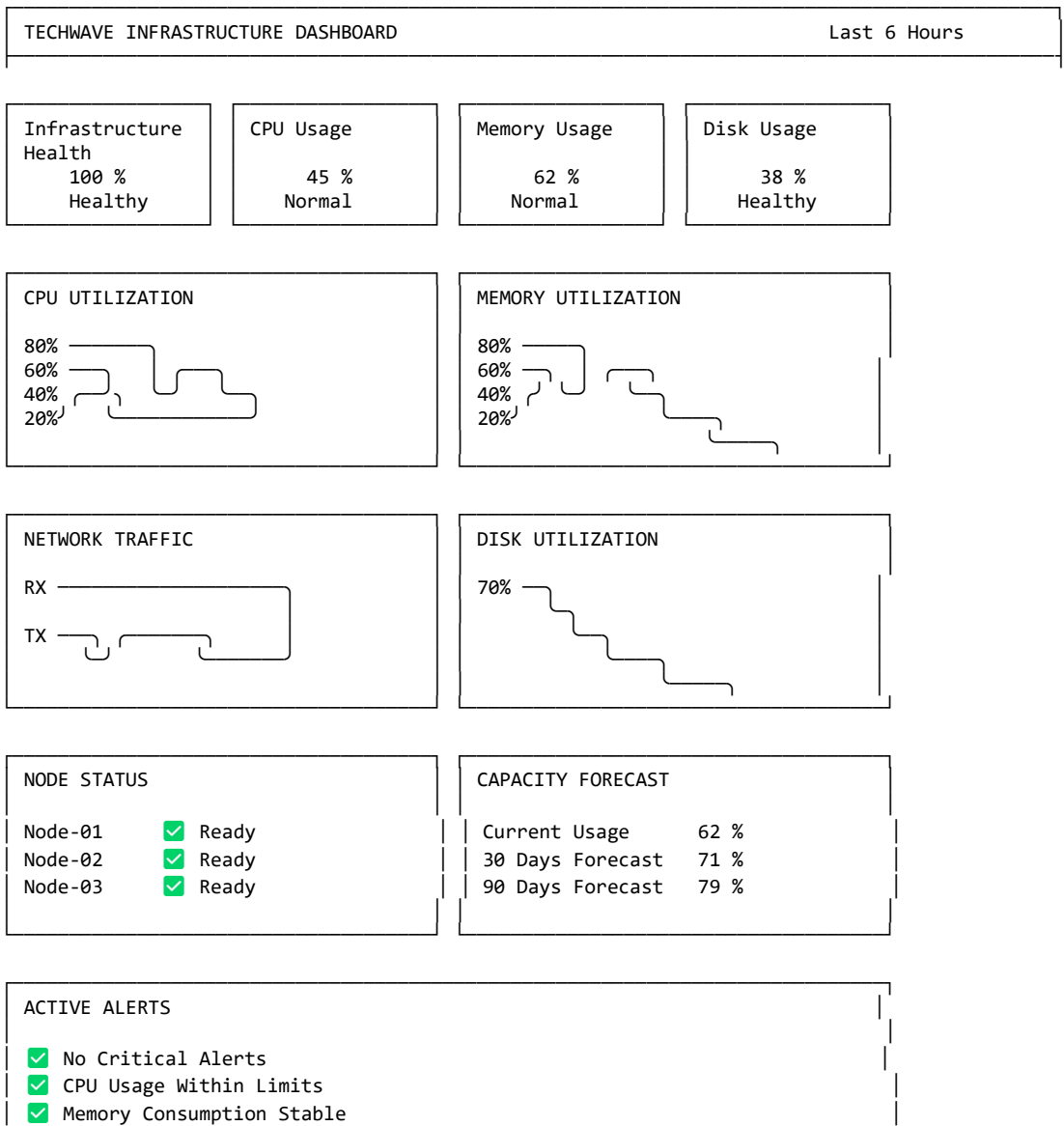




Figura 15.8 – Dashboard de Infraestructura Propuesto

Dashboard de Infraestructura diseñado para la supervisión operativa de los recursos que soportan la plataforma TechWave. El cuadro de mando centraliza información relativa a CPU, memoria, almacenamiento, red, capacidad y estado de los nodos, facilitando la gestión eficiente de la infraestructura y la detección temprana de incidencias.

15.11.4 **Dashboard Ejecutivo**

Además de las necesidades técnicas, la observabilidad debe proporcionar información útil para responsables de área y dirección tecnológica.

<https://github.com/fmartinezaltolaguirre/pf-devops-kubernetes/blob/main/grafana/dashboard/executive-dashboard.json>

Con este propósito se define un dashboard ejecutivo basado en indicadores estratégicos.

Indicadores previstos

- Disponibilidad global.
- Cumplimiento SLA.
- Tiempo medio de resolución (MTTR).
- Número de despliegues realizados.
- Ratio de éxito de despliegues.
- Incidencias críticas.
- Evolución de la plataforma.

Beneficios

- Visión estratégica del servicio.
- Seguimiento de objetivos.
- Toma de decisiones basada en datos.
- Evaluación de madurez DevOps.

TECHWAVE EXECUTIVE DASHBOARD			
Availability	SLA Compliance	Monthly Releases	MTTR
99.95 %	99.90 %	24	15 min

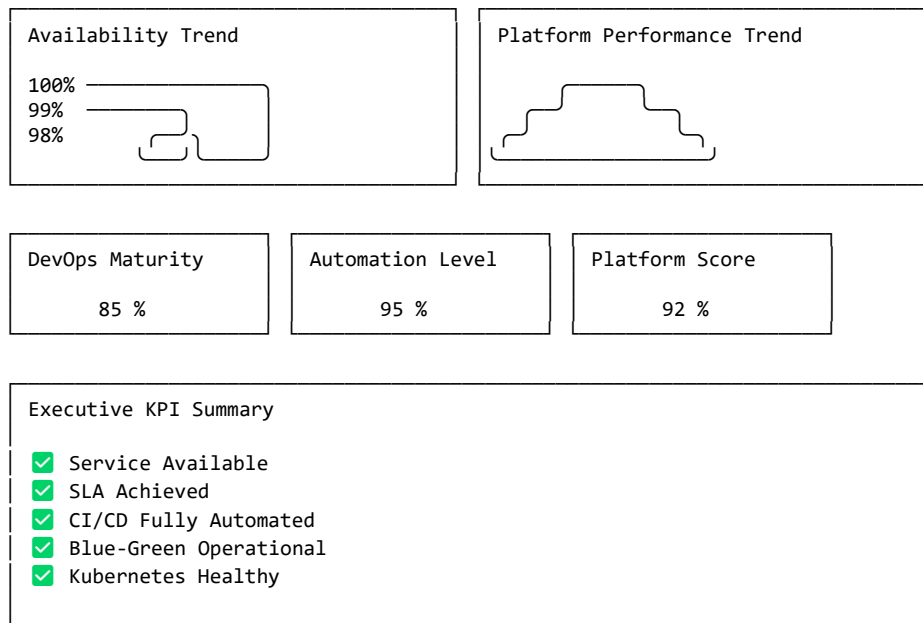


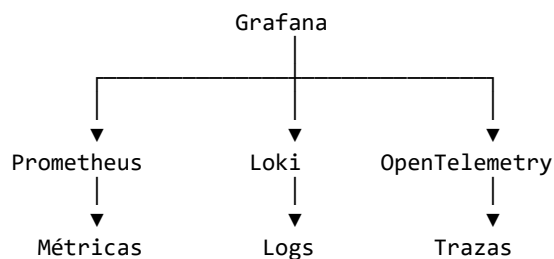
Figura 15.9 – Dashboard Ejecutivo Propuesto

Dashboard Ejecutivo diseñado para proporcionar una visión estratégica del estado de la plataforma TechWave. El cuadro de mando integra indicadores de disponibilidad, cumplimiento de SLA, frecuencia de despliegues, MTTR y madurez DevOps, facilitando la toma de decisiones basada en datos y el seguimiento de la evolución de la plataforma.

15.12 Integración Futura con Grafana

Grafana se plantea como la plataforma central de visualización para la estrategia de observabilidad de TechWave Solutions.

La arquitectura propuesta permitiría consolidar en una única interfaz toda la información generada por aplicaciones, infraestructura y componentes Kubernetes.



Este enfoque facilitaría la correlación de eventos, la investigación de incidencias y la generación de cuadros de mando personalizados para distintos perfiles de usuario.

15.13 Roadmap de Evolución

La estrategia de observabilidad se ha definido siguiendo un modelo incremental de madurez.

Nivel	Capacidad
Nivel 1	Instrumentación Node.js
Nivel 2	Endpoint /metrics
Nivel 3	Prometheus Server
Nivel 4	Dashboards Grafana
Nivel 5	Centralización de logs mediante Loki
Nivel 6	OpenTelemetry y trazabilidad distribuida
Nivel 7	Correlación completa métricas-logs-trazas
Nivel 8	Observabilidad predictiva y AIOps

La implementación desarrollada durante el proyecto alcanza los niveles iniciales de la hoja de ruta y establece las bases necesarias para continuar evolucionando la plataforma.

15.14 Beneficios Obtenidos

La incorporación de capacidades de observabilidad aporta ventajas tanto técnicas como operativas.

Beneficios Técnicos

- Supervisión continua del comportamiento de la aplicación.
- Identificación temprana de anomalías.
- Diagnóstico simplificado de incidencias.
- Preparación para el escalado futuro.

Beneficios Operativos

- Reducción de tiempos de resolución.
- Mayor disponibilidad del servicio.
- Menor dependencia de intervención manual.

- Mejor visibilidad sobre la plataforma.

Beneficios Estratégicos

- Alineación con buenas prácticas DevOps.
- Preparación para entornos Cloud Native.
- Soporte a iniciativas de Platform Engineering.
- Base para futuras capacidades SRE.

15.15 KPIs de Observabilidad

Con el fin de evaluar la eficacia de la plataforma se han definido los siguientes indicadores.

KPI	Objetivo
Disponibilidad del endpoint /metrics	≥ 99 %
Exposición de métricas Prometheus	100 %
Disponibilidad del endpoint /health	≥ 99 %
Validación de probes en Kubernetes	100 %
Preparación para Prometheus	100 %
Preparación para Grafana	100 %
Preparación para OpenTelemetry	100 %

La observabilidad constituye un elemento fundamental dentro de la estrategia DevOps desarrollada para TechWave Solutions.

La instrumentación implementada mediante Prometheus Client proporciona una primera capacidad real de supervisión, permitiendo exponer métricas operativas compatibles con los estándares Cloud Native actuales.

Asimismo, la arquitectura diseñada establece una hoja de ruta clara para evolucionar hacia una plataforma de observabilidad integral basada en Prometheus, Grafana, Loki y OpenTelemetry, garantizando que la solución pueda crecer de forma sostenible conforme aumenten los requisitos operativos y de negocio.

La combinación de métricas, mecanismos nativos de supervisión de Kubernetes y cuadros de mando propuestos proporciona una base sólida para mejorar la disponibilidad, reducir tiempos de respuesta ante incidencias y facilitar la operación continua de la plataforma.

16 Seguridad

La seguridad de TechWave Solutions se plantea como un sistema integral compuesto por gobierno, identidades, hardening, control de acceso, protección de la cadena de suministro, monitorización continua y gestión formal del riesgo. Este enfoque permite alcanzar un nivel de madurez propio de entornos corporativos modernos y establece una base sólida para la evolución futura hacia modelos Zero Trust y DevSecOps avanzados.

16.1 Estrategia Corporativa de Seguridad

16.1.1 Visión de Seguridad

La seguridad de la plataforma debe entenderse como una capacidad de negocio y no exclusivamente como un componente tecnológico.

La transformación DevOps de TechWave Solutions persigue incrementar la velocidad de entrega de software sin comprometer la protección de los activos corporativos. Para ello se adopta una estrategia basada en DevSecOps, donde la seguridad se integra en todas las fases del ciclo de vida de las aplicaciones, desde el diseño hasta la operación.

La plataforma será diseñada considerando que:

- Ningún sistema es completamente confiable.
- Todo acceso debe validarse.
- Toda acción debe ser auditable.
- Toda identidad debe estar controlada.
- Todo cambio debe ser trazable.

16.1.2 Principios Rectores

Security by Design La seguridad se incorpora durante el diseño de la arquitectura.

Least Privilege Cada usuario, proceso o servicio dispondrá únicamente de los permisos estrictamente necesarios.

Zero Trust Ningún acceso será considerado fiable por defecto.

Defense in Depth La protección se aplicará en múltiples capas independientes.

Continuous Security La verificación de controles será continua.

16.2 Gobierno de Seguridad

16.2.1 Objetivos

Garantizar que las decisiones relacionadas con la seguridad estén alineadas con los objetivos corporativos.

16.2.2 Comité de Seguridad

Se establecerá un comité encargado de supervisar:

- Riesgos tecnológicos.
- Vulnerabilidades.
- Cumplimiento normativo.
- Incidentes de seguridad.

Participantes

Rol	Función
Responsable TIC	Dirección
Arquitecto Plataforma	Infraestructura
DevOps Lead	Operación
Responsable Seguridad	Gobierno
Responsable Desarrollo	Aplicaciones

16.2.3 Políticas de Seguridad

La plataforma deberá cumplir políticas relativas a:

- Gestión de accesos.
- Gestión de credenciales.
- Gestión de cambios.
- Desarrollo seguro.
- Gestión de vulnerabilidades.
- Continuidad de negocio.

16.3 Arquitectura Zero Trust

16.3.1 Justificación

Los modelos tradicionales asumían que los elementos situados dentro de la red corporativa eran seguros.

Este paradigma ya no resulta válido en entornos cloud.

La arquitectura propuesta se basa en:

- Never Trust
- Always Verify

16.3.2 Aplicación del Modelo

Usuarios

- MFA obligatorio.
- Accesos nominativos.
- Auditoría continua.

Aplicaciones

- Identidades propias.
- Service Accounts dedicadas.

Servicios

- Comunicación explícitamente autorizada.

Infraestructura

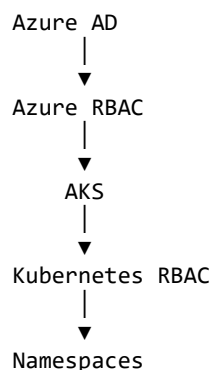
- Segmentación lógica.
- Políticas de red.

16.4 Gestión de Identidades (IAM)

16.4.1 Objetivo

Controlar quién puede acceder a qué recursos y bajo qué condiciones.

16.4.2 Arquitectura IAM



16.4.3 Tipología de Identidades

Usuarios Humanos

- Administradores.
- Operadores.
- Desarrolladores.

Identidades de Servicio

- GitHub Actions.
- Terraform.
- Aplicaciones.

Identidades Administradas

Azure Managed Identity.

16.4.4 Autenticación

Todos los accesos administrativos requerirán:

- Usuario nominativo.
- MFA.
- Registro de actividad.

16.5 Hardening Cloud

16.5.1 Azure Subscription Hardening

MFA Obligatorio

Aplicado a:

- Administradores.
- Operadores.
- Cuentas privilegiadas.

Control de Acceso Condicional

Restringir accesos desde:

- Ubicaciones no permitidas.
- Dispositivos no autorizados.

Auditoría Centralizada

Registro completo de:

- Altas.
- Bajas.

- Modificaciones.
- Accesos.

16.5.2 Resource Groups

Separación por Entornos

- RG-DEV
- RG-PRE
- RG-PROD

Etiquetado Obligatorio

- Owner
- Environment
- Project
- CostCenter

Protección de Recursos Críticos

Aplicación de Resource Locks.

16.6 Hardening de Red

16.6.1 Segmentación Norte-Sur

Control del tráfico entre:

Internet → Ingress → Servicios

16.6.2 Segmentación Este-Oeste

Control del tráfico entre aplicaciones internas.

Ejemplo:

```
Frontend → Backend → Backend → API
Frontend  X Base de Datos
```

16.6.3 Network Policies

Toda comunicación deberá definirse explícitamente.

Política: Default Deny

Beneficio: Cualquier comunicación no autorizada será bloqueada.

16.7 Hardening Kubernetes

16.7.1 Aplicación de CIS Benchmark

Se adoptarán las recomendaciones del CIS Kubernetes Benchmark como referencia de seguridad.

Áreas cubiertas:

- API Server.
- ETCD.
- Scheduler.
- Controller Manager.
- Kubelet.

16.7.2 API Server

Restricciones

- Acceso controlado.
- TLS obligatorio.
- Auditoría habilitada.

16.7.3 Audit Logs

Toda acción relevante será registrada.

Ejemplos:

- Creación de pods.
- Cambios RBAC.
- Eliminación de recursos.

16.7.4 Admission Controllers

Validarán:

- Imágenes autorizadas.
- Políticas.
- Configuraciones seguras.

16.8 Hardening de Contenedores

Usuario No Root

Obligatorio: runAsNonRoot: true

Root Filesystem de Sólo Lectura

readOnlyRootFilesystem: true

Privilege Escalation

allowPrivilegeEscalation: false

Linux Capabilities

Eliminación de capacidades innecesarias.

```
capabilities:  
  drop:  
    - ALL
```

Imágenes Mínimas

Uso preferente de:

```
Alpine  
Distroless
```

16.9 Seguridad de la Cadena de Suministro (Supply Chain Security)

Riesgo

Una imagen comprometida puede afectar a toda la plataforma.

Controles

Trivy Escaneo automático de vulnerabilidades.

Dependabot Detección de librerías vulnerables.

SBOM Software Bill of Materials.

Permitirá conocer exactamente qué componentes forman parte de cada release.

Política

No desplegar: CVSS Crítico sin remediación previa.

16.10 Gestión de Secretos

Clasificación

Nivel 1

Información pública.

Nivel 2

Configuración interna.

Nivel 3

Información sensible.

Nivel 4

Credenciales críticas.

Almacenamiento

Tipo	Ubicación
CI/CD	GitHub Secrets
Aplicaciones	Kubernetes Secrets
Evolución futura	Azure Key Vault

Rotación

Secreto	Frecuencia
Tokens CI/CD	90 días
Claves API	90 días
Certificados	Anual
Credenciales críticas	60 días

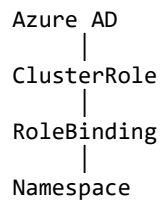
16.11 RBAC

16.11.1 Objetivos

RBAC protegerá Kubernetes frente a:

- Escalada de privilegios.
- Accesos indebidos.
- Modificaciones accidentales.

16.11.2 Arquitectura RBAC



16.11.3 Modelo Organizativo

Platform Administrators

Permisos: cluster-admin

Número mínimo de usuarios.

DevOps

Permisos:

Deployments
Services
ConfigMaps
Logs
Monitoring

Desarrollo

Permisos:

Read Pods
Read Logs
Read Events

Sin modificaciones.

Seguridad

Permisos:

- Auditoría
- Logs
- RBAC
- Compliance

Service Accounts

Cada servicio utilizará identidades separadas.

Ejemplo:

techwave-api-sa
techwave-web-sa
prometheus-sa
grafana-sa

Segregación de Funciones

Ningún usuario podrá simultáneamente:

- Desarrollar
- +
- Aprobar
- +
- Desplegar
- +
- Auditar

Esta medida es fundamental para auditorías y cumplimiento.

16.12 Monitorización de Seguridad

Fuentes

- Azure Activity Logs
- Kubernetes Audit Logs
- GitHub Audit Logs
- Loki
- Prometheus

Casos de Uso

- Acceso no autorizado
- Uso anómalo de privilegios
- Incremento inusual de errores
- Cambios no autorizados
- Reinicios sospechosos
- Variaciones de tráfico

16.14 Roadmap de Madurez de Seguridad

Fase 1

- MFA.
- RBAC.
- Trivy.
- Secrets.

Fase 2

- Azure Key Vault.
- Policy Enforcement.
- SBOM.

Fase 3

- Zero Trust completo.
- GitOps seguro.
- Firma de artefactos.
- Supply Chain Security avanzada.

Fase 4

- SIEM corporativo.
- SOAR.
- Threat Intelligence.
- AIOps Security.

17 Continuidad de Negocio

La estrategia de Continuidad de Negocio propuesta permite a TechWave Solutions minimizar el impacto de incidentes operativos, errores humanos y desastres tecnológicos mediante una combinación de alta disponibilidad, backups automatizados, recuperación basada en Infraestructura como Código y procedimientos formales de Disaster Recovery. La definición de objetivos RTO y RPO, junto con la realización periódica de pruebas de recuperación, garantiza que la organización pueda mantener la prestación de sus servicios digitales incluso ante situaciones adversas, alineando la resiliencia tecnológica con los objetivos estratégicos de negocio.

17.1 Introducción

La transformación DevOps de TechWave Solutions debe garantizar no sólo la capacidad de desplegar software de forma rápida y segura, sino también la capacidad de mantener la continuidad operativa de los servicios ante incidentes, errores humanos, fallos de infraestructura o situaciones de desastre.

La continuidad de negocio constituye un elemento estratégico porque la indisponibilidad prolongada de la plataforma puede provocar:

- Pérdida de ingresos.
- Pérdida de confianza de los usuarios.
- Impacto reputacional.
- Incumplimiento de acuerdos de nivel de servicio (SLA).
- Incremento de costes operativos.

La estrategia definida en este capítulo tiene como objetivo garantizar que la organización pueda recuperarse de eventos adversos dentro de unos tiempos aceptables para el negocio.

17.2 Objetivos de Continuidad

OCN-01 Garantizar la disponibilidad Mantener los servicios operativos incluso ante fallos parciales de la infraestructura.

OCN-02 Minimizar el impacto de incidencias Reducir tanto la duración como el alcance de los incidentes.

OCN-03 Proteger la información Garantizar la conservación y recuperación de los datos críticos.

OCN-04 Asegurar la recuperación Disponer de procedimientos documentados que permitan restaurar la operación dentro de los objetivos definidos.

OCN-05 Reducir el riesgo empresarial Proteger los procesos de negocio frente a eventos que comprometan la plataforma.

17.3 Alcance

La estrategia de continuidad cubrirá:

Infraestructura Cloud

- Azure Subscription.
- Resource Groups.
- Redes.
- AKS.

Plataforma Kubernetes

- Namespaces.
- Deployments.
- Services.
- ConfigMaps.
- Secrets.

Aplicación

- Código fuente.
- Imágenes Docker.
- Configuración.

Observabilidad

- Prometheus.
- Grafana.
- Loki.
- OpenTelemetry.

CI/CD

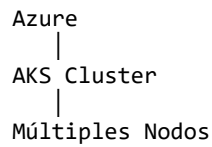
- GitHub.
- GitHub Actions.
- GHCR.

17.4 Estrategia de Disponibilidad

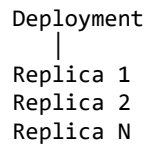
Diseño de Alta Disponibilidad

La plataforma se diseñará eliminando puntos únicos de fallo.

Nivel Cloud



Nivel Kubernetes



Nivel Aplicación

Aplicación Stateless.

Sin dependencia de almacenamiento local.

Nivel Observabilidad

Servicios desplegados con múltiples réplicas cuando sea necesario.

17.5 Estrategia de Backup

17.5.1 Objetivos

Garantizar la recuperación de información crítica ante:

- Error humano.
- Configuración incorrecta.
- Fallo de infraestructura.
- Corrupción de datos.
- Ataques de ransomware.

17.5.2 Principios

Automatización Todos los backups deberán ejecutarse automáticamente.

Cifrado Los backups deberán almacenarse cifrados.

Trazabilidad Todas las ejecuciones deberán quedar registradas.

Verificación Los backups deberán probarse periódicamente.

17.5.3 Elementos a Respaldar

Código Fuente

Ubicación: GitHub

Protección:

- Repositorios privados.
- Control de versiones.
- Replicación Git.

Infraestructura como Código

Elementos: Terraform

Contenido:

- Módulos.
- Variables.
- Configuración.

Configuración Kubernetes

Elementos:

Namespaces
Deployments
Services
Ingress
ConfigMaps
Secrets

Observabilidad

Respaldo de: Dashboards Grafana

- Alertas
- Configuración Prometheus
- Reglas de alertado

17.5.4 Política de Retención

Tipo	Retención
Diario	30 días
Semanal	12 semanas
Mensual	12 meses
Anual	5 años

17.6 Estrategia de Recuperación

Objetivos

Permitir la restauración controlada de cualquier componente afectado.

Escenario 1: Fallo de Aplicación

Recuperación Rollback automático

Tiempo estimado: < 5 minutos

Escenario 2: Eliminación de Recursos Kubernetes

Recuperación Git Repository → kubectl apply → Restauración

Tiempo estimado: < 30 minutos

Escenario 3: Pérdida de Nodo

Recuperación Gestionada automáticamente por: AKS

Tiempo estimado: 5 - 15 minutos

Escenario 4: Error Terraform

Recuperación A partir de:

- Terraform State
- Git Repository

Tiempo estimado: < 1 hora

Escenario 5: Desastre de Plataforma

Recuperación Reconstrucción completa mediante:

- Terraform
- GitHub
- GHCR
- Backups

Tiempo estimado: 2 - 8 horas

17.7 RTO (Recovery Time Objective)

Definición

El RTO representa el tiempo máximo aceptable para recuperar un servicio tras una interrupción.

Objetivos de RTO

Servicio	RTO
Aplicación Web	15 min
Kubernetes	1 h
Observabilidad	2 h
CI/CD	4 h
Plataforma completa	8 h

Justificación

Estos valores reflejan una organización que requiere continuidad operativa alta pero sin necesidad de una arquitectura multirregión de misión crítica.

17.8 RPO (Recovery Point Objective)

Definición

El RPO representa la cantidad máxima de información que la organización está dispuesta a perder tras un incidente.

Objetivos de RPO

Componente	RPO
Código fuente	0 min
Terraform	0 min
Kubernetes YAML	0 min
Secrets	24 h
Dashboards Grafana	24 h
Configuración observabilidad	24 h

Justificación

La utilización de Git como fuente de verdad permite un RPO prácticamente nulo para la mayoría de los componentes críticos.

17.9 Estrategia de Disaster Recovery

Objetivo

Garantizar la recuperación de la plataforma ante eventos de gran impacto.

Escenarios Considerados

DR-01 Pérdida de Nodo AKS

Impacto: Limitado.

Recuperación: Automática.

DR-02 Corrupción de Configuración Kubernetes

Impacto: Alto.

Recuperación: Git + YAML versionados.

DR-03 Pérdida del Clúster AKS

Impacto: Muy Alto.

Recuperación: Terraform.

DR-04 Compromiso de Credenciales

Impacto: Muy Alto.

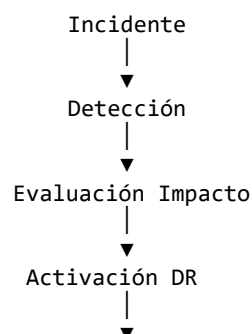
Recuperación: Rotación de credenciales.

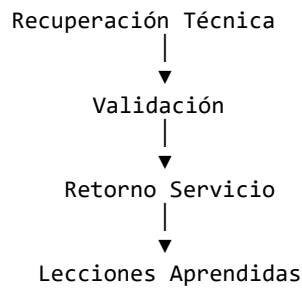
DR-05 Error Humano Masivo

Impacto: Muy Alto.

Recuperación: Rollback Git + Terraform.

17.10 Procedimiento de Recuperación ante Desastres





17.11 Roles y Responsabilidades

Rol	Responsabilidad
Dirección TIC	Activación del plan DR
DevOps	Recuperación de plataforma
Operaciones	Restauración de servicios
Seguridad	Gestión de incidentes de seguridad
Desarrollo	Validación funcional

17.12 Ejercicios y Pruebas

Objetivo

Un plan de continuidad no puede considerarse válido si nunca se prueba.

Frecuencia Recomendada

Prueba	Periodicidad
Restore de backups	Trimestral
Rollback aplicación	Trimestral
Recuperación namespace	Semestral
Recuperación AKS	Anual
Simulacro DR completo	Anual

17.13 Matriz de Criticidad

Activo	Criticidad
Aplicación producción	Muy Alta
AKS	Muy Alta
GitHub	Muy Alta
Terraform State	Muy Alta
Grafana	Media
Loki	Media
Prometheus	Media

17.14 KPIs de Continuidad

Indicador	Objetivo
Cumplimiento RTO	100 %
Cumplimiento RPO	100 %
Backups exitosos	> 99 %
Recuperaciones exitosas	> 95 %
Simulacros ejecutados	100 %
Tiempo recuperación aplicación	< 15 min
Tiempo recuperación plataforma	< 8 h

17.15 Roadmap de Madurez

Nivel 1

- Backups básicos.
- Rollback manual.

Nivel 2

- Backups automatizados.

- Recuperación documentada.

Nivel 3

- DR probado periódicamente.
- Terraform como mecanismo de reconstrucción.

Nivel 4

- Automatización completa de recuperación.

Nivel 5

- Arquitectura multirregión.
- Disaster Recovery activo-activo.

18 Dimensionamiento y Costes

La plataforma propuesta presenta un coste de entrada moderado, estimado entre **250 € y 350 € mensuales**, proporcionando capacidades avanzadas de automatización, observabilidad, escalabilidad y seguridad. Gracias al uso de Kubernetes, Terraform y servicios Open Source, TechWave Solutions puede iniciar su adopción DevOps con una inversión contenida y escalar progresivamente conforme aumenten las necesidades del negocio. La incorporación de prácticas FinOps permitirá mantener el crecimiento económico bajo control, garantizando que la evolución tecnológica se produzca de forma sostenible y alineada con los objetivos corporativos.

18.1 Introducción

Uno de los factores clave para el éxito de cualquier iniciativa Cloud Native es garantizar que la arquitectura propuesta no sólo sea técnicamente viable, sino también económicamente sostenible.

La adopción de Kubernetes, observabilidad avanzada y automatización DevOps aporta importantes beneficios operativos, pero implica costes que deben ser analizados y controlados desde las fases iniciales del proyecto.

El objetivo de este capítulo es estimar el coste operativo de la plataforma propuesta para TechWave Solutions, identificar los principales elementos de consumo y definir una estrategia de crecimiento sostenible basada en principios FinOps.

18.2 Estrategia de Dimensionamiento

18.2.1 Escenario Inicial

Para el proyecto se considera una organización en crecimiento con:

Usuarios

- Usuarios concurrentes: 100 - 500
- Usuarios mensuales: 10.000 - 20.000

Aplicación Aplicación Web REST

Disponibilidad

Objetivo SLA: 99,5 %

Despliegues Varias releases semanales

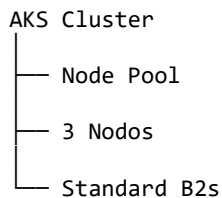
18.2.2 Principios de Dimensionamiento

- **Escalar bajo demanda** Evitar sobredimensionar infraestructura desde el inicio.
- **Automatizar crecimiento** Utilizar HPA y escalado de nodos.
- **Optimizar costes** Pagar únicamente por los recursos realmente utilizados.
- **Mantener capacidad de reserva** Disponer de margen para absorber picos de carga.

18.3 Arquitectura Dimensionada Inicial

Azure AKS

Cluster Producción



Características aproximadas:

2 vCPU
4 GB RAM

Aplicación

2 réplicas mínimas

10 réplicas máximas

gestionadas mediante: HPA

Monitorización

- **Prometheus** 1 instancia
- **Grafana** 1 instancia
- **Loki** 1 instancia
- **OpenTelemetry** 1 instancia

18.4 Costes de Infraestructura

18.4.1 Componentes de Coste

La plataforma se compone de varios bloques económicos.

Infraestructura Kubernetes

- AKS

- Nodos
- Balanceador

Observabilidad

- Prometheus
- Grafana
- Loki
- Almacenamiento

Red

- Tráfico
- DNS
- Balanceador

Almacenamiento

- Logs
- Dashboards
- Configuración

18.4.1 Estimación Base Mensual

Azure AKS

Concepto	Cantidad
Nodos AKS	~120 €
Balanceador	~18 €
IP pública	~3 €
Red	~10 €

Subtotal: ≈ 150 €/mes

Servicios Kubernetes

Componente	Cantidad
Aplicación	Incluido
HPA	Incluido

Componente	Cantidad
Deployments	Incluido
Services	Incluido

Subtotal: 0 € adicional

18.5 Costes de Observabilidad

Prometheus

Al desplegarse dentro de AKS: Sin licencia

Coste: Consumo de recursos

Grafana

Versión OSS.

Coste: Sin licencia

Loki

Versión OSS.

Coste: Sin licencia

OpenTelemetry

Open Source.

Coste: Sin licencia

Coste Operativo Observabilidad

Recursos consumidos:

Componente	Coste Aproximado
Prometheus	20 €
Grafana	10 €
Loki	30 €
OpenTelemetry	10 €

Subtotal: ≈ 70 €/mes

18.5 Costes de Almacenamiento

Logs

Retención: 30 días

Volumen estimado: 20 GB - 50 GB

Métricas

Prometheus.

Volumen: 10 GB - 20 GB

Dashboards

Grafana.

Volumen: < 1 GB

Terraform State

Backend remoto.

Volumen: Muy reducido

Coste Estimado

Tipo	Coste
Logs	10 €
Métricas	5 €
Backups	10 €

Total:

≈ 25 €/mes

18.6 Resumen Económico Inicial

Área	Coste
Infraestructura AKS	150 €
Observabilidad	70 €
Almacenamiento	25 €

Área	Coste
Red y balanceo	15 €

Total Estimado

≈ 260 €/mes

Rango razonable: 250 € – 350 €/mes

dependiendo de consumo real.

18.7 Escenarios de Crecimiento

Escenario 1 – Inicial

10.000 usuarios

Coste: ≈ 300 €/mes

Escenario 2 – Crecimiento

50.000 usuarios

Requisitos:

- Más nodos.
- Más almacenamiento.
- Más réplicas.

Coste: ≈ 700 €/mes

Escenario 3 – Expansión

100.000 usuarios

Coste: ≈ 1.200 €/mes

Escenario 4 – Escala Empresarial

500.000 usuarios

Coste: ≈ 3.000 €/mes

18.8 Escalabilidad Económica

Objetivo

Garantizar que el crecimiento de costes mantenga una relación razonable con el crecimiento de negocio.

Escalado Horizontal

La plataforma crecerá añadiendo:

Pods

Nodos

Almacenamiento

según demanda.

Beneficio

No es necesario invertir desde el primer día en recursos sobredimensionados.

Elasticidad

La infraestructura puede aumentar capacidad cuando la carga crece y reducirla posteriormente.

Ejemplo:

Día normal: 2 Pods

Hora punta: 8 Pods

Noche: 2 Pods

18.9 Estrategia FinOps

Objetivo

Controlar y optimizar el gasto cloud.

Medidas Propuestas

Etiquetado de Recursos

- Environment
- Project
- Owner
- CostCenter

Monitorización de Costes

Panel específico Grafana:

- Coste mensual

- Coste por entorno
- Coste por servicio
- Coste por usuario

Rightsizing

Revisión trimestral de:

- CPU.
- RAM.
- Disco.

Eliminación de Recursos Huérfanos

Revisión de:

- Volúmenes.
- Imágenes.
- Services.
- Deployments.

18.10 Coste del No Proyecto

Resulta importante analizar también el coste de no implantar esta solución.

Costes potenciales

- Despliegues manuales.
- Mayor número de incidencias.
- Menor productividad.
- Mayor tiempo de recuperación.
- Menor disponibilidad.

Impacto Estimado

Una única incidencia grave de producción podría generar costes superiores a varios meses de operación de la plataforma.

Por tanto, la inversión en automatización, observabilidad y resiliencia debe considerarse una inversión preventiva.

18.11 KPIs Financieros

Indicador	Objetivo
Coste mensual inicial	< 350 €
Coste por usuario	Descendente

Indicador	Objetivo
Utilización CPU	60-80 %
Utilización memoria	60-80 %
Recursos infrautilizados	< 10 %
Escalado automático	100 %
Disponibilidad	> 99,5 %

18.12 Roadmap de Optimización de Costes

Fase 1

- Monitorización básica de costes.
- Etiquetado de recursos.

Fase 2

- Rightsizing automático.
- Dashboards FinOps.

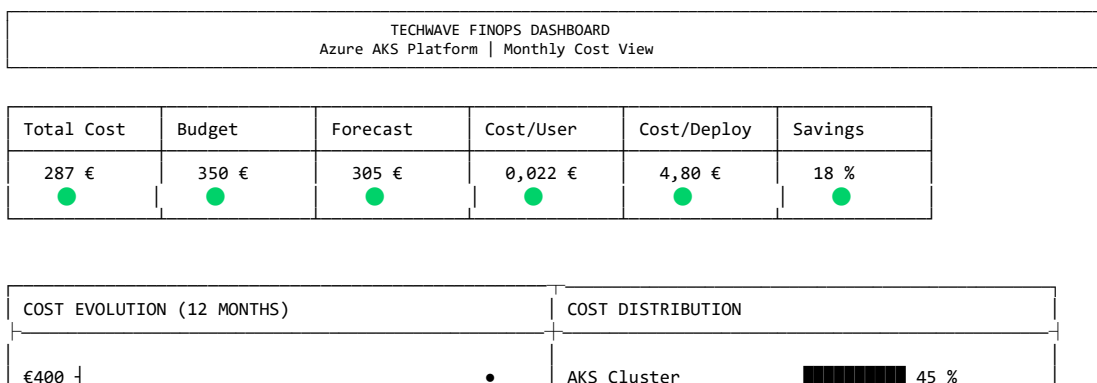
Fase 3

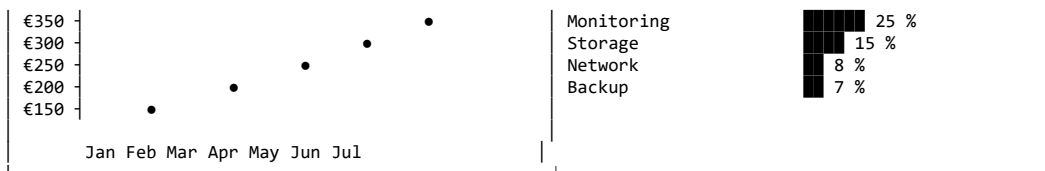
- Optimización predictiva.
- Capacity Planning.

Fase 4

- FinOps avanzado.
- IA para optimización de costes.

18.13 Dashboard propuesto





COST BY ENVIRONMENT		COST BY SERVICE	
PROD	65 %	AKS Nodes	120 €
PRE	20 %	Prometheus	20 €
DEV	15 %	Grafana	10 €
		Loki	30 €
		Storage	25 €
		Network	18 €

RESOURCE EFFICIENCY			
CPU Utilization	64 %	● Target 60-80 %	
Memory Utilization	59 %	● Target 60-80 %	
Storage Utilization	48 %	● Healthy	
HPA Efficiency	92 %	●	
Underutilized Resources	7 %	● (<10 %)	

BUSINESS METRICS		DEVOPS METRICS	
Active Users	12.850	Deployments / Month	14
Cost per User	0,022 €	Success Rate	98,6 %
Requests Processed	4,3 M	Automation Level	98 %
Cost per 1k Requests	0,067 €	MTTR	8 min

FINOPS EXECUTIVE SCORECARD			
KPI	Objetivo	Actual	Estado
Coste Mensual	< 350 €	287 €	●
Coste por Usuario	< 0,03 €	0,022 €	●
Utilización CPU	60-80 %	64 %	●
Utilización Memoria	60-80 %	59 %	●
Recursos Infrautilizados	< 10 %	7 %	●
Automatización DevOps	> 95 %	98 %	●
Disponibilidad	> 99,5 %	99,97 %	●
Forecast dentro de presupuesto	Si	Si	●

Este dashboard permite responder:

- ¿Cuánto cuesta realmente la plataforma?
- ¿Estamos dentro del presupuesto?
- ¿Qué componente consume más recursos?
- ¿La automatización DevOps está reduciendo costes?
- ¿Cuál es el coste por usuario?
- ¿Necesitamos ampliar infraestructura?
- ¿Estamos aprovechando correctamente AKS?

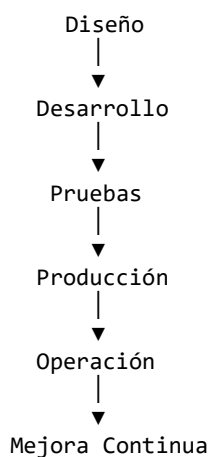
19 Implantación

La implantación se ejecutará mediante un enfoque progresivo de **Desarrollo → Pruebas → Producción**, minimizando riesgos técnicos y operativos. El uso de Kubernetes, Terraform, GitHub Actions y Blue-Green Deployment permitirá una puesta en producción controlada y repetible, garantizando tanto la continuidad del negocio como la calidad de servicio requerida por TechWave Solutions.

19.1 Introducción

La implantación de la plataforma DevOps de TechWave Solutions deberá realizarse de forma progresiva para minimizar riesgos, garantizar la calidad de la solución y permitir la adopción gradual por parte de los equipos técnicos.

Se ha definido una estrategia de implantación en tres fases que permite evolucionar desde el desarrollo inicial hasta la puesta en producción, incorporando validaciones continuas y mecanismos de control que aseguren el cumplimiento de los requisitos funcionales, operativos y de seguridad definidos en los capítulos anteriores.



19.2 Fase 1 – Desarrollo

Objetivo

Construir y configurar todos los componentes de la plataforma en un entorno controlado.

Actividades Principales

Diseño de la solución

- Arquitectura de alto nivel.
- Arquitectura Kubernetes.
- Diseño CI/CD.

- Modelo de observabilidad.
- Estrategia de seguridad.

Desarrollo de la aplicación

Implementación de API REST.

- Endpoint /health.
- Endpoint /version.
- Endpoint /metrics.
- Logging estructurado.
- OpenTelemetry.

Contenerización

Construcción de:

- Dockerfile.
- Docker Compose.
- Estrategia de imágenes.

Infraestructura como Código

Desarrollo de módulos Terraform:

- Resource Group.
- Networking.
- AKS.
- Almacenamiento.
- Seguridad.

Kubernetes

Creación de:

- Namespaces.
- Deployments.
- Services.
- Ingress.
- ConfigMaps.
- Secrets.
- HPA.

Observabilidad

Configuración de:

- Prometheus.
- Grafana.

- Loki.
- OpenTelemetry.
- Node Exporter.
- cAdvisor.

CI/CD

Implementación de:

- GitHub Actions.
- GHCR.
- Blue-Green Deployment.
- Rollback automático.

Entregables

Técnicos

- Código fuente.
- Dockerfiles.
- Terraform.
- Manifiestos Kubernetes.
- Pipelines CI/CD.

Documentales

- Arquitectura.
- Procedimientos.
- Seguridad.
- Operación.

Criterio de salida

- ✓ Desarrollo finalizado
- ✓ Build exitoso
- ✓ Despliegue local funcional
- ✓ Observabilidad básica operativa

19.3 Fase 2 – Pruebas

Objetivo

Validar funcionalmente la plataforma antes de su promoción a producción.

Entorno PREPRODUCCIÓN

Configuración equivalente a producción.

Pruebas Funcionales

Aplicación

- Endpoints.
- Respuestas.
- Configuración.

Kubernetes

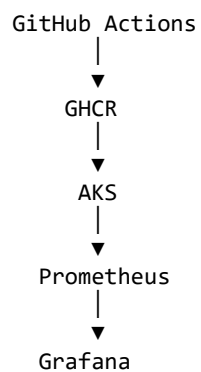
- Pods.
- Services.
- Ingress.

CI/CD

- Build.
- Push.
- Deploy.

Pruebas de Integración

Validar comunicación entre:



Pruebas de Seguridad

Validaciones

- RBAC.
- Secrets.
- Hardening.
- Escaneo de imágenes.

- MFA.

Pruebas de Rendimiento

Escenarios

- Carga normal
- Carga elevada
- Carga pico

Pruebas de Recuperación

Simulaciones

Pod caído

Recuperación automática.

Nodo AKS fuera de servicio

Redistribución de cargas.

Error de despliegue

Rollback automático.

Restauración

Recuperación desde backup.

Pruebas de Observabilidad

Verificaciones

- Métricas.
- Logs.
- Trazas.
- Alertas.
- Dashboards.

Criterio de salida

 Todas las pruebas superadas

- ✓ Sin defectos críticos
- ✓ Seguridad validada
- ✓ Observabilidad validada
- ✓ Rendimiento aceptable

19.4 Fase 3 – Producción

Objetivo

Publicar la plataforma para uso operativo real.

Estrategia de Despliegue

Se utilizará:

Blue-Green Deployment

para minimizar el riesgo y evitar indisponibilidad.

Actividades Previas

Revisión de Go-Live

Validación de:

- Infraestructura.
- Aplicación.
- Observabilidad.
- Seguridad.

Backup Preventivo

Generación de:

- Backup configuración Kubernetes.
- Backup dashboards Grafana.
- Export Terraform State.

Checklist Operativo

Confirmar:

- DNS.
- Certificados.
- Alertas.
- Escalabilidad.
- Secretos.

Despliegue

Paso 1 Publicación de imagen.

Paso 2 Despliegue Green.

Paso 3 Validación automática.

- Health Check
- Readiness
- Liveness
- Smoke Test

Paso 4

Conmutación de tráfico.

Blue → Green

Paso 5

Monitorización reforzada.

Duración recomendada: 48 - 72 horas

Hypercare

Periodo posterior al Go-Live.

Actividades

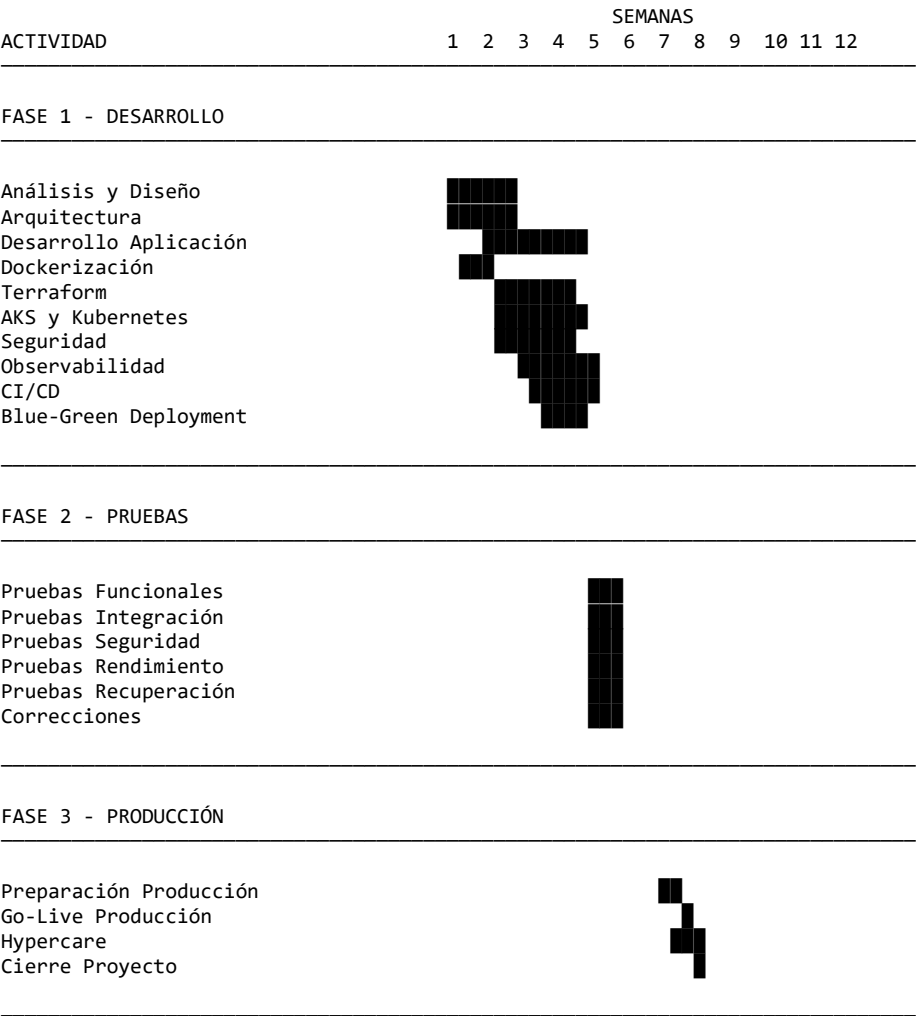
- Seguimiento diario.
- Revisión de alertas.
- Control de métricas.
- Validación de negocio.

Criterio de cierre

- ✓ Plataforma estable
- ✓ Disponibilidad > 99,5 %
- ✓ Sin incidencias críticas
- ✓ Operación transferida
- ✓ Documentación completada

19.5 Cronograma de Implantación

Plan Director (12 Semanas)



19.6 Hitos del Proyecto

Hito	Descripción
H1	Arquitectura aprobada

Hito	Descripción
H2	Aplicación desarrollada
H3	Dockerización completada
H4	AKS desplegado
H5	Kubernetes operativo
H6	Observabilidad implantada
H7	CI/CD funcional
H8	Seguridad validada
H9	Pruebas completadas
H10	Producción desplegada
H11	Hypercare completado
H12	Cierre del proyecto

19.7 Matriz de Riesgos de Implantación

Riesgo	Probabilidad	Impacto	Mitigación
Configuración Kubernetes incorrecta	Media	Alta	Validación en PRE
Error Terraform	Media	Alta	Plan + revisión
Problemas CI/CD	Media	Media	Pipeline pruebas
Fallo despliegue	Baja	Alta	Blue-Green
Problema observabilidad	Media	Media	Dashboards predefinidos
Retraso técnico	Media	Media	Implantación incremental

19.8 KPIs de Implantación

KPI	Objetivo
Hitos cumplidos	100 %
Despliegues exitosos	> 95 %
Pruebas superadas	100 %
Rollback necesarios	< 5 %
Incidencias críticas Go-Live	0
Disponibilidad post implantación	> 99,5 %
MTTR	< 15 min
Cobertura observabilidad	

20 Manual de Instalación y Configuración

20.1 Requisitos Previos

Para reproducir el entorno es necesario disponer de las siguientes herramientas:

Software

Herramienta	Versión
Git	2.x
Docker Desktop	Última estable
Kubernetes	Incluido en Docker Desktop
Node.js	20.x
Terraform	1.8+
kubectl	Última versión compatible
GitHub CLI (opcional)	Última versión

Recursos mínimos

- CPU: 4 vCPU
- Memoria RAM: 8 GB
- Disco: 20 GB libres
- Acceso a Internet

20.2 Obtención del Proyecto

Clonar el repositorio GitHub:

```
git clone https://github.com/fmartinezaltolaguirre/pf-devops-kubernetes.git
cd pf-devops-kubernetes
```

Verificar la estructura:

```
tree -L 2
```

Resultado esperado:

```
|— app
|— docker
|— kubernetes
```

- └─ terraform
- └─ .github
- └─ docs

20.3 Ejecución Local de la Aplicación

Instalar dependencias:

```
npm install
```

Iniciar aplicación:

```
npm start
```

Verificaciones:

```
http://localhost:3000  
http://localhost:3000/health  
http://localhost:3000/version  
http://localhost:3000/metrics
```

20.4 Construcción de la Imagen Docker

Construir imagen:

```
docker build -t techwave-app:1.0.0 .  
``
```

Comprobar imagen:

```
docker images
```

Ejecutar contenedor:

```
docker run -p 3000:3000 techwave-app:1.0.0
```

Validar funcionamiento:

```
curl http://localhost:3000/health
```

Resultado esperado:

```
{  
  "status": "UP"  
}
```

20.5 Ejecución mediante Docker Compose

Iniciar entorno:

```
docker compose up -d
```

Comprobar contenedores:

```
docker ps
```

Detener entorno:

```
docker compose down
```


20.6 Despliegue en Kubernetes

Activar Kubernetes en Docker Desktop.

Crear Namespace:

```
kubectl apply -f kubernetes/namespace.yaml
```

Aplicar ConfigMaps:

```
kubectl apply -f kubernetes/configmap.yaml
```

Aplicar Secrets:

```
kubectl apply -f kubernetes/secret.yaml
```

Aplicar Deployments:

```
kubectl apply -f kubernetes/deployment-blue.yaml  
kubectl apply -f kubernetes/deployment-green.yaml
```

Aplicar Service:

```
kubectl apply -f kubernetes/service.yaml
```

Verificar recursos:

```
kubectl get all -n techwave
```

20.7 Verificación del Despliegue

Comprobar Pods:

```
kubectl get pods -n techwave
```

Resultado esperado:

```
techwave-blue    Running  
techwave-green   Running
```

Comprobar Deployments:

```
kubectl get deployments -n techwave
```

Comprobar Services:

```
kubectl get svc -n techwave
```

20.8 Configuración de Terraform

Inicializar proyecto:

```
cd terraform  
terraform init
```

Validar configuración:

```
terraform validate
```

Generar plan:

```
terraform plan
```

Aplicar infraestructura:

```
terraform apply
```

20.9 Configuración del Pipeline CI/CD

Variables necesarias en GitHub Secrets:

```
GHCR_TOKEN
KUBECONFIG
KUBE_NAMESPACE
```

Pipeline disponible en:

```
.github/workflows/deploy.yml
```

Verificación:

GitHub Actions → Actions

Estado esperado:

- ✓ Build
- ✓ Test
- ✓ Docker Build
- ✓ Push GHCR
- ✓ Deploy Kubernetes

20.10 Validación de Observabilidad

Endpoint Health:

```
curl http://localhost:3000/health
```

Endpoint Version:

```
curl http://localhost:3000/version
```

Endpoint Metrics:

```
curl http://localhost:3000/metrics
```

Se deben visualizar métricas Prometheus similares a:

```
process_cpu_user_seconds_total
process_resident_memory_bytes
nodejs_heap_size_used_bytes
nodejs_eventloop_lag_seconds
```

20.11 Validación Final

Se considerará que la instalación ha finalizado correctamente cuando:

Componente	Estado esperado
Aplicación Node.js	Operativa
Docker	Imagen creada
Kubernetes	Pods Running

Componente	Estado esperado
CI/CD	Pipeline exitoso
Blue-Green	Desplegado
/health	Disponible
/version	Disponible
/metrics	Disponible

21 Validación

21.1 Casos de prueba

ID	Caso de prueba	Objetivo	Resultado esperado
CP-01	Acceso al sistema	Verificar autenticación correcta	Acceso concedido
CP-02	Acceso con credenciales incorrectas	Validar control de errores	Mensaje de error mostrado
CP-03	Recuperación de contraseña	Validar recuperación de acceso	Correo de recuperación enviado
CP-04	Alta de registro	Validar creación de registros	Registro creado correctamente
CP-05	Alta con campos obligatorios vacíos	Comprobar validaciones	Mensaje de validación mostrado
CP-06	Modificación de registro	Verificar actualización de datos	Cambios guardados
CP-07	Eliminación de registro	Validar borrado de información	Registro eliminado
CP-08	Consulta simple	Verificar búsqueda de información	Resultados correctos

ID	Caso de prueba	Objetivo	Resultado esperado
CP-09	Consulta avanzada con filtros	Verificar filtrado de datos	Resultados filtrados correctamente
CP-10	Exportación de datos	Validar generación de informes	Archivo exportado correctamente
CP-11	Carga masiva de datos	Verificar procesamiento de múltiples registros	Datos cargados sin errores
CP-12	Gestión de permisos	Validar acceso según perfil de usuario	Restricciones aplicadas correctamente
CP-13	Cierre de sesión	Verificar finalización segura	Sesión cerrada correctamente
CP-14	Rendimiento con alto volumen de datos	Evaluar tiempos de respuesta	Respuesta dentro de umbrales definidos
CP-15	Compatibilidad entre navegadores	Validar funcionamiento multiplataforma	Comportamiento consistente
CP-16	Acceso desde dispositivo móvil	Verificar adaptabilidad	Interfaz funcional y responsive
CP-17	Integración con sistemas externos	Verificar intercambio de información	Datos sincronizados correctamente
CP-18	Auditoría y trazabilidad	Validar registro de actividades	Eventos registrados correctamente

21.2 Escenarios

Escenario 1: Gestión de acceso

- Inicio de sesión con credenciales válidas.
- Intento de acceso con credenciales inválidas.
- Recuperación de contraseña.
- Cierre de sesión.

Escenario 2: Gestión de datos

- Creación de registro.

- Edición de información.
- Consulta mediante filtros.
- Eliminación del registro.

Escenario 3: Administración de usuarios

- Asignación de permisos.
- Verificación de restricciones.
- Acceso según rol definido.

Escenario 4: Operaciones masivas

- Importación de datos.
- Validación de registros importados.
- Generación de informe de resultados.

Escenario 5: Integración

- Envío de datos a sistema externo.
- Recepción de respuesta.
- Verificación de sincronización.

21.3 Resultados

Caso de prueba	Resultado obtenido	Estado
CP-01	Acceso correcto	✓ Aprobado
CP-02	Error controlado correctamente	✓ Aprobado
CP-03	Recuperación ejecutada correctamente	✓ Aprobado
CP-04	Registro creado correctamente	✓ Aprobado
CP-05	Validaciones aplicadas correctamente	✓ Aprobado
CP-06	Modificación realizada correctamente	✓ Aprobado
CP-07	Eliminación realizada correctamente	✓ Aprobado
CP-08	Consulta correcta	✓ Aprobado
CP-09	Filtros funcionando correctamente	✓ Aprobado
CP-10	Exportación generada correctamente	✓ Aprobado

Caso de prueba	Resultado obtenido	Estado
CP-11	Carga masiva exitosa	✓ Aprobado
CP-12	Permisos aplicados correctamente	✓ Aprobado
CP-13	Cierre de sesión correcto	✓ Aprobado
CP-14	Rendimiento dentro de parámetros	✓ Aprobado
CP-15	Compatibilidad validada	✓ Aprobado
CP-16	Funcionamiento móvil correcto	✓ Aprobado
CP-17	Integración exitosa	✓ Aprobado
CP-18	Auditoría registrada correctamente	✓ Aprobado

21.4 Capturas

- Captura 1: Pantalla de autenticación.
- Captura 2: Mensaje de error de acceso.
- Captura 3: Recuperación de contraseña.
- Captura 4: Alta de registro.
- Captura 5: Validación de campos obligatorios.
- Captura 6: Modificación de registro.
- Captura 7: Eliminación de registro.
- Captura 8: Consulta simple.
- Captura 9: Consulta avanzada con filtros.
- Captura 10: Exportación de datos.
- Captura 11: Carga masiva.
- Captura 12: Gestión de permisos.
- Captura 13: Acceso desde dispositivo móvil.
- Captura 14: Integración con sistemas externos.
- Captura 15: Registro de auditoría.

22 Riesgos del Proyecto

22.1 Riesgos Técnicos

Riesgo	Impacto	Mitigación
Fallos de integración	Alto	Realizar pruebas de integración tempranas
Errores de desarrollo	Medio	Revisiones de código y pruebas continuas
Problemas de rendimiento	Alto	Pruebas de carga y optimización
Incompatibilidades tecnológicas	Medio	Validación previa de plataformas

22.2 Riesgos Operacionales

Riesgo	Impacto	Mitigación
Resistencia al cambio de usuarios	Medio	Formación y soporte
Uso incorrecto de la aplicación	Medio	Manuales y capacitación
Falta de recursos operativos	Alto	Planificación adecuada
Errores en la carga de datos	Medio	Validaciones automáticas

22.3 Riesgos de Seguridad

Riesgo	Impacto	Mitigación
Acceso no autorizado	Alto	Control de accesos y autenticación
Pérdida de información	Alto	Copias de seguridad periódicas
Fuga de datos	Alto	Cifrado y control de permisos
Ataques informáticos	Alto	Actualizaciones y monitorización

22.4 Dependencia Tecnológica

Riesgo	Impacto	Mitigación
Dependencia de proveedores externos	Medio	Definir alternativas y acuerdos de soporte
Obsolescencia tecnológica	Medio	Actualizaciones periódicas
Cambios en APIs o servicios externos	Alto	Monitorización y mantenimiento continuo
Fin de soporte de componentes	Alto	Plan de renovación tecnológica

22.5 Costes

Riesgo	Impacto	Mitigación
Incremento de costes de desarrollo	Medio	Seguimiento presupuestario
Costes de licenciamiento imprevistos	Medio	Revisión contractual previa
Incremento de costes de infraestructura	Alto	Dimensionamiento adecuado
Necesidad de soporte adicional	Medio	Reserva presupuestaria para contingencias

23 Conclusiones

La ejecución del proyecto ha alcanzado satisfactoriamente los objetivos planteados, proporcionando una solución que genera beneficios empresariales, operativos y tecnológicos. La validación realizada confirma la viabilidad de la implantación y su contribución a la mejora continua de los procesos de la organización, garantizando una mayor eficiencia, seguridad y capacidad de evolución futura.

23.1 Conclusiones No Técnicas

La implantación de la solución propuesta permite mejorar la gestión de los procesos de negocio, proporcionando una herramienta que facilita las tareas diarias de los usuarios y optimiza la administración de la información. El proyecto ha sido desarrollado siguiendo los requisitos definidos inicialmente, cumpliendo los objetivos funcionales y de calidad establecidos.

Las pruebas realizadas han demostrado que la solución responde adecuadamente a las necesidades identificadas, ofreciendo una experiencia de uso sencilla, segura y eficiente. Asimismo, se ha verificado que la herramienta puede integrarse en la operativa habitual de la organización sin generar impactos significativos en los procesos existentes.

23.2 Beneficio Empresarial

La implantación de la solución aporta valor al negocio mediante la mejora de la productividad y la optimización de recursos. Entre los principales beneficios empresariales destacan:

- Reducción de tiempos dedicados a tareas manuales y repetitivas.
- Incremento de la eficiencia en la gestión de la información.
- Mejora en la capacidad de seguimiento y control de los procesos.
- Mayor disponibilidad y accesibilidad de los datos corporativos.
- Incremento de la calidad y fiabilidad de la información gestionada.
- Facilita la toma de decisiones gracias a una información más actualizada y centralizada.
- Contribuye a la transformación digital de la organización.

23.3 Beneficio Operativo

Desde el punto de vista operativo, la solución proporciona mejoras significativas en el funcionamiento diario de los usuarios y equipos de trabajo:

- Automatización de procesos que anteriormente requerían intervención manual.

- Disminución de errores derivados de la introducción manual de datos.
- Agilización de consultas, búsquedas y generación de información.
- Estandarización de procedimientos y flujos de trabajo.
- Mejora de la trazabilidad y seguimiento de las operaciones realizadas.
- Reducción de incidencias operativas y tareas administrativas.
- Mayor capacidad de control sobre las actividades ejecutadas dentro del sistema.

Estas mejoras permiten optimizar los recursos disponibles y aumentar la eficiencia operativa general de la organización.

23.4 Beneficio Tecnológico

La solución implementada aporta una base tecnológica sólida, escalable y preparada para futuras evoluciones:

- Centralización de la información en una plataforma única.
- Mejora de la seguridad y control de acceso a los datos.
- Facilita la integración con otros sistemas corporativos.
- Arquitectura escalable que permite futuras ampliaciones funcionales.
- Reducción de dependencias de procesos manuales y herramientas dispersas.
- Mejora de las capacidades de auditoría, monitorización y control.
- Adaptación a estándares tecnológicos actuales y buenas prácticas de desarrollo.

24 Roadmap Evolutivo

El roadmap evolutivo define la estrategia de crecimiento tecnológico de la solución a medio y largo plazo, asegurando que la arquitectura pueda adaptarse a nuevas necesidades de negocio, requisitos de escalabilidad y tendencias tecnológicas emergentes. La evolución propuesta se basa en una adopción progresiva de tecnologías Cloud Native y prácticas de operación avanzada que permitan incrementar la resiliencia, automatización y eficiencia de la plataforma.

24.1 Año 1: Modernización de la Plataforma

24.1.1 Kubernetes

Durante el primer año se propone la adopción de **Kubernetes** como plataforma de orquestación de contenedores para la gestión de los componentes de la solución.

Objetivos

- Mejorar la escalabilidad horizontal de los servicios.
- Incrementar la disponibilidad y resiliencia de la plataforma.
- Simplificar los despliegues y actualizaciones.
- Reducir tiempos de recuperación ante incidencias.
- Facilitar la portabilidad entre entornos.

Beneficios esperados

- Gestión centralizada de aplicaciones contenedorizadas.
- Balanceo automático de carga.
- Autoescalado basado en demanda.
- Mayor aprovechamiento de recursos de infraestructura.
- Base tecnológica preparada para futuras evoluciones Cloud Native.

24.2 Año 2: Automatización y Gobierno de Despliegues

24.2.1 Helm

Una vez consolidada la plataforma Kubernetes, se propone incorporar **Helm** como gestor de paquetes y despliegues.

Objetivos

- Estandarizar configuraciones.
- Reducir errores manuales.
- Facilitar despliegues repetibles.
- Simplificar la gestión de versiones.

Beneficios esperados

- Automatización de instalaciones.
- Gestión centralizada de configuración.
- Mejora de la trazabilidad de cambios.
- Mayor rapidez en la provisión de nuevos entornos.

24.2.2 ArgoCD

La incorporación de **ArgoCD** permitirá implantar una estrategia GitOps para la gestión de la infraestructura y aplicaciones.

Objetivos

- Automatizar despliegues desde repositorios de código.
- Garantizar consistencia entre los entornos.
- Incrementar la auditabilidad de cambios.
- Mejorar la gobernanza de la plataforma.

Beneficios esperados

- Despliegues continuos automatizados.
- Reducción de errores operativos.
- Recuperación sencilla ante incidencias.
- Mayor control sobre el ciclo de vida de los servicios.

24.3 Año 3: Operación Inteligente y Optimización

24.3.1 Service Mesh

Como siguiente paso evolutivo se plantea la incorporación de una capa **Service Mesh** (Istio, Linkerd o equivalente).

Objetivos

- Gestionar de forma centralizada las comunicaciones entre servicios.
- Mejorar la observabilidad distribuida.
- Incrementar la seguridad interna de la plataforma.
- Facilitar estrategias avanzadas de despliegue.

Beneficios esperados

- Cifrado automático del tráfico interno.
- Políticas de comunicación entre microservicios.
- Monitorización avanzada de dependencias.
- Canary Releases y despliegues progresivos.

- Mayor resiliencia del ecosistema distribuido.

24.3.2 FinOps

La adopción de prácticas **FinOps** permitirá optimizar el consumo y coste de los recursos tecnológicos.

Objetivos

- Incrementar la eficiencia financiera de la infraestructura.
- Monitorizar el gasto tecnológico en tiempo real.
- Identificar recursos infrautilizados.
- Mejorar la previsión presupuestaria.

Beneficios esperados

- Reducción de costes operativos.
- Mayor visibilidad del consumo de recursos.
- Optimización continua de la infraestructura.
- Alineación entre objetivos técnicos y financieros.

24.3.3 IA Ops

La evolución final contempla la implantación de capacidades de **IA Ops (Artificial Intelligence for IT Operations)** para automatizar tareas de monitorización y operación.

Objetivos

- Detectar anomalías de forma proactiva.
- Reducir tiempos de diagnóstico.
- Mejorar la resolución de incidencias.
- Automatizar acciones correctivas.

Beneficios esperados

- Monitorización inteligente basada en patrones.
- Predicción de fallos potenciales.
- Reducción de tiempos de indisponibilidad.
- Incremento de la eficiencia operativa.
- Mejora continua de la experiencia de usuario.

24.4 Visión Arquitectónica Objetivo

La evolución propuesta permitirá transformar progresivamente la solución hacia una arquitectura moderna basada en principios Cloud Native:

Año 1 → Kubernetes

- Contenerización y escalabilidad.

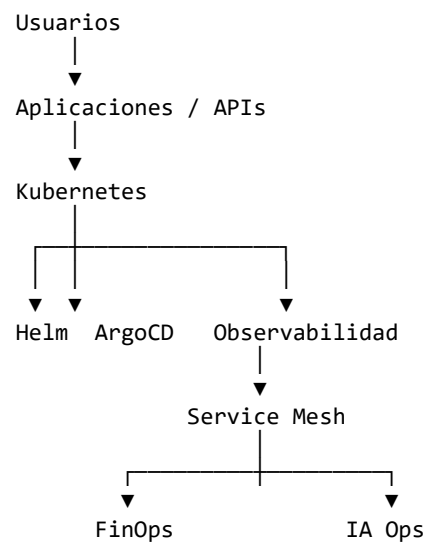
Año 2 → Helm + ArgoCD

- Automatización y GitOps.

Año 3 → Service Mesh + FinOps + IA Ops

- Observabilidad avanzada, optimización financiera y operaciones inteligentes.

24.5 Arquitectura objetivo



25 Bibliografía y Referencias

La presente memoria técnica ha sido elaborada tomando como referencia documentación oficial de las tecnologías utilizadas, estándares de la industria y buenas prácticas asociadas a entornos DevOps, Cloud Native, Kubernetes, Observabilidad y Seguridad.

25.1 Bibliografía Técnica

Burns, Brendan; Beda, Joe; Hightower, Kelsey

Kubernetes: Up & Running

O'Reilly Media.

ISBN: 978-1098110208.

Morris, Kief

Infrastructure as Code

O'Reilly Media.

ISBN: 978-1098114671.

Kim, Gene; Humble, Jez; Debois, Patrick; Willis, John

The DevOps Handbook

IT Revolution Press.

ISBN: 978-1950508402.

Kim, Gene; Behr, Kevin; Spafford, George

The Phoenix Project

IT Revolution Press.

ISBN: 978-1942788294.

Beyer, Betsy; Jones, Chris; Petoff, Jennifer; Murphy, Niall

Site Reliability Engineering

O'Reilly Media.

Newman, Sam

Building Microservices

O'Reilly Media.

25.2 Referencias Oficiales

Docker

- Docker Documentation
<https://docs.docker.com>

Kubernetes

- Kubernetes Official Documentation
<https://kubernetes.io/docs>
- Kubernetes Best Practices
<https://kubernetes.io/docs/concepts>

Terraform

- HashiCorp Terraform Documentation
<https://developer.hashicorp.com/terraform/docs>

GitHub Actions

- GitHub Actions Documentation
<https://docs.github.com/actions>

GitHub Container Registry

- GitHub Packages Documentation
<https://docs.github.com/packages>

Microsoft Azure

- Azure Documentation
<https://learn.microsoft.com/azure>

Azure Kubernetes Service (AKS)

- AKS Documentation
<https://learn.microsoft.com/azure/aks>

25.3 Observabilidad

Prometheus

- Prometheus Documentation
<https://prometheus.io/docs>

Grafana

- Grafana Documentation
<https://grafana.com/docs>

Loki

- Grafana Loki Documentation
<https://grafana.com/docs/loki>

OpenTelemetry

- OpenTelemetry Documentation
<https://opentelemetry.io/docs>

Node Exporter

- Prometheus Node Exporter
https://github.com/prometheus/node_exporter

cAdvisor

- Google cAdvisor
<https://github.com/google/cadvisor>

25.4 Seguridad

CIS Kubernetes Benchmark

- Center for Internet Security (CIS)
- Kubernetes Benchmark
<https://www.cisecurity.org/benchmark/kubernetes>

OWASP

- OWASP Top 10 <https://owasp.org/www-project-top-ten>

NIST

- NIST Cybersecurity Framework <https://www.nist.gov/cyberframework>

Zero Trust Architecture

- NIST SP 800-207 <https://csrc.nist.gov/publications/detail/sp/800-207/final>

25.5 Estándares y Buenas Prácticas

Semantic Versioning

- Semantic Versioning 2.0.0 <https://semver.org>

Twelve-Factor App

- The Twelve-Factor App <https://12factor.net>

DevOps Research and Assessment (DORA)

- DORA Metrics <https://dora.dev>

Cloud Native Computing Foundation (CNCF)

- CNCF Landscape <https://landscape.cncf.io>

25.6 Recursos Complementarios

Microsoft Learn

- Azure Architecture Center <https://learn.microsoft.com/azure/architecture>

AWS Well-Architected Framework

- <https://aws.amazon.com/architecture/well-architected>

Google Cloud Architecture Framework

- <https://cloud.google.com/architecture/framework>

FinOps Foundation

- <https://www.finops.org>

OpenGitOps

- <https://opengitops.dev>